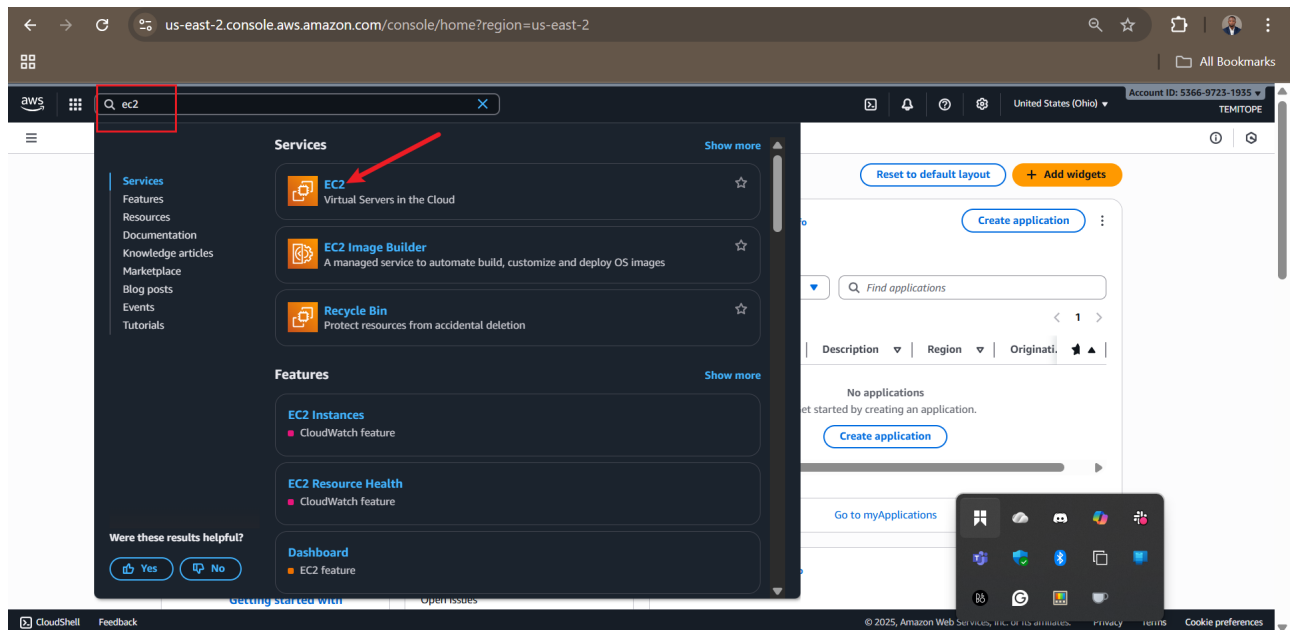


Configuring Auto Scaling with Application Load Balancer using Launch Template

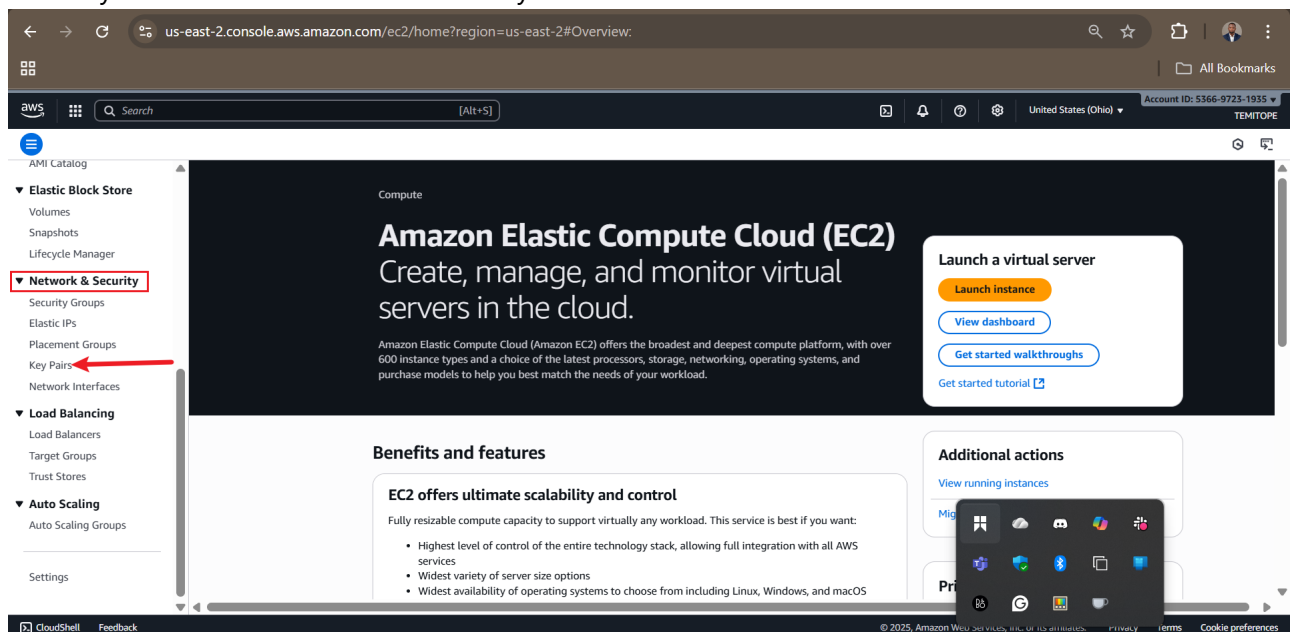
This project demonstrates the step-by-step process of configuring an Auto Scaling group (ASG) integrated with an Application Load Balancer (ALB) using a Launch Template in AWS EC2. The steps are organized sequentially based on the numbered screenshots provided.

Step-by-Step Guide

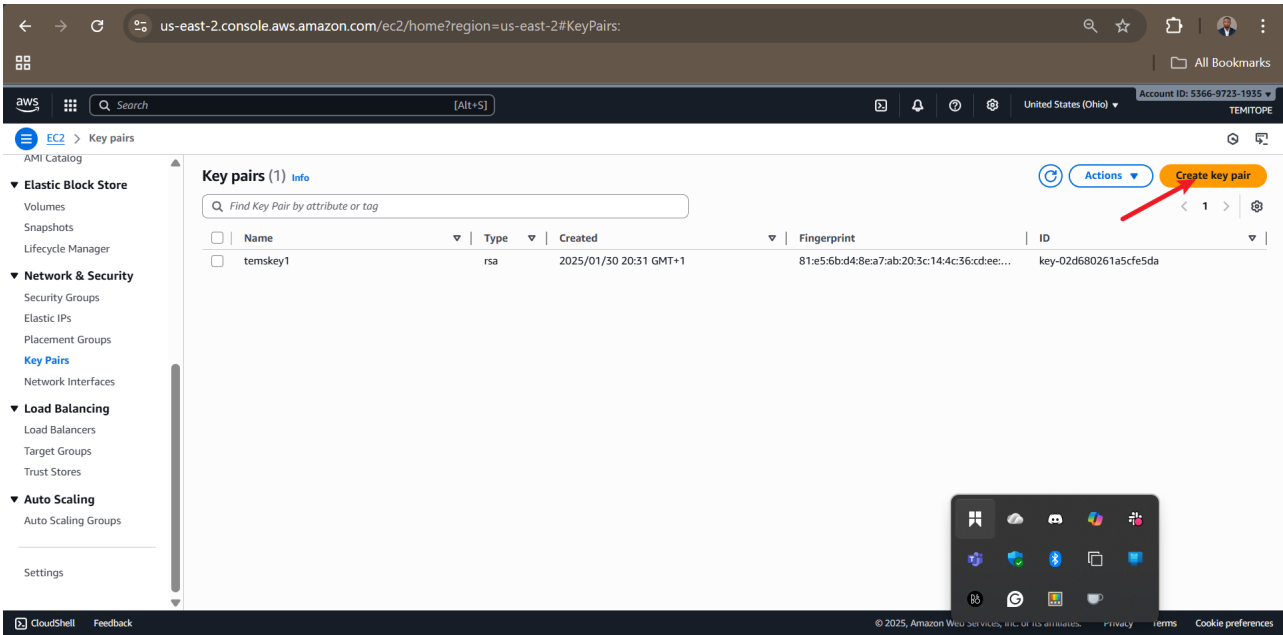
1. Search for EC2 and click on it.



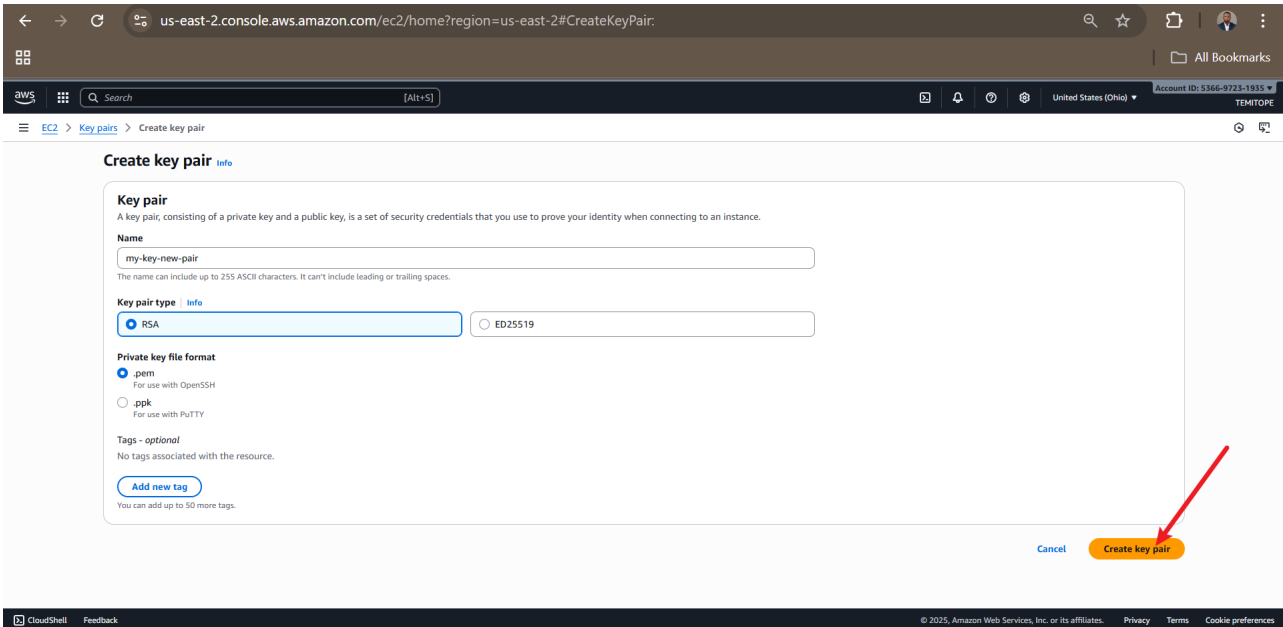
2. Find Key Pairs under Network and Security on the left sidebar and click on it.



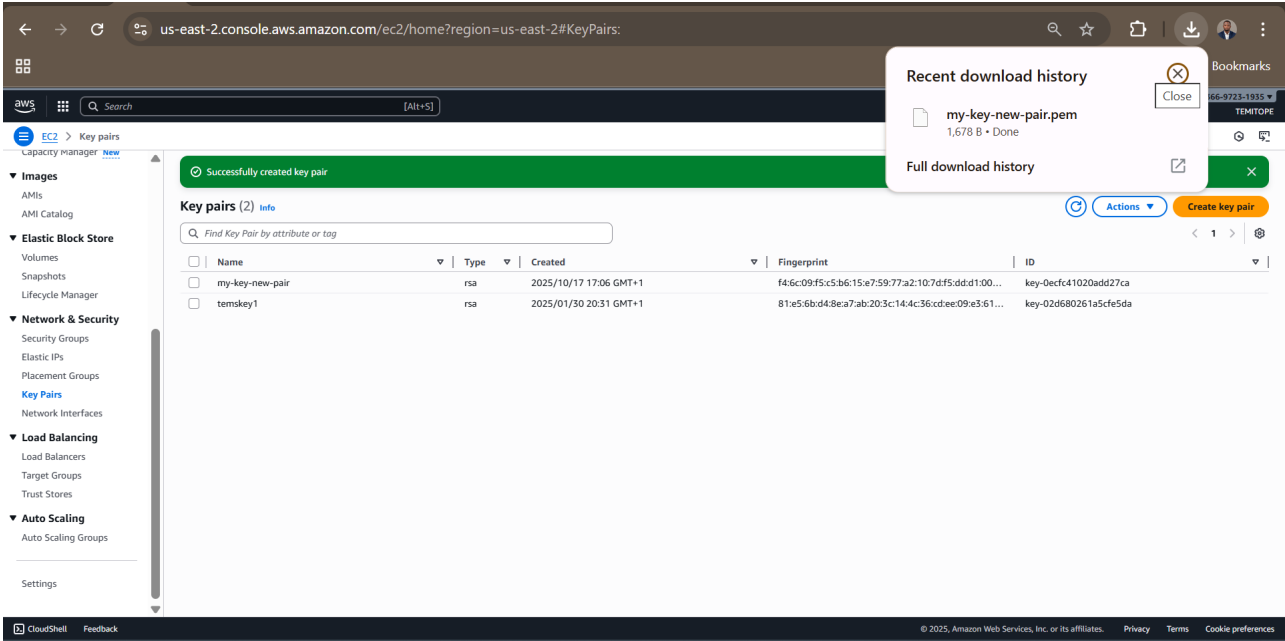
3. On the keypair page click create keypair.



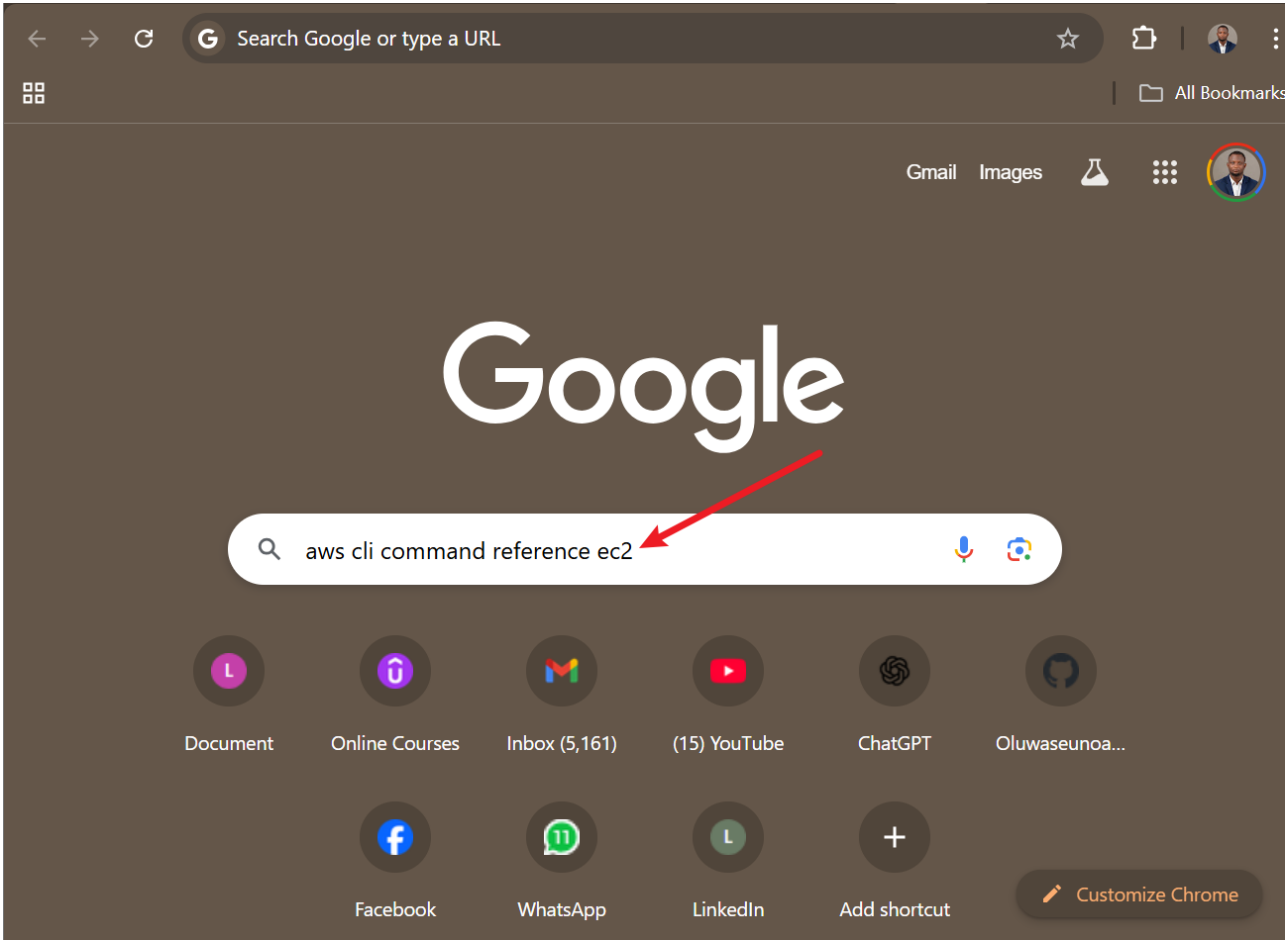
4. Name key pair select type and format and click create key pair.



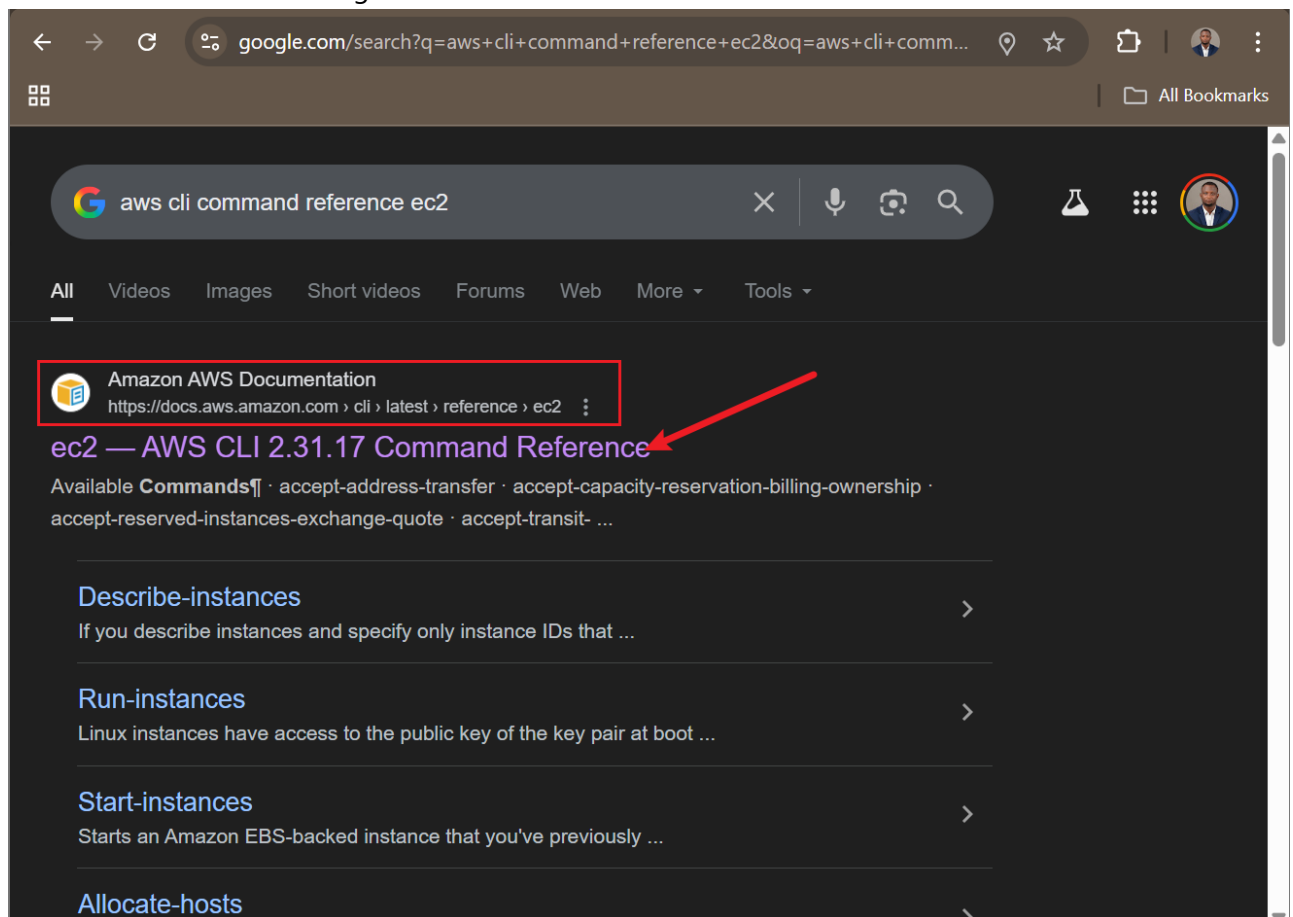
5. Keypair created and downloaded.



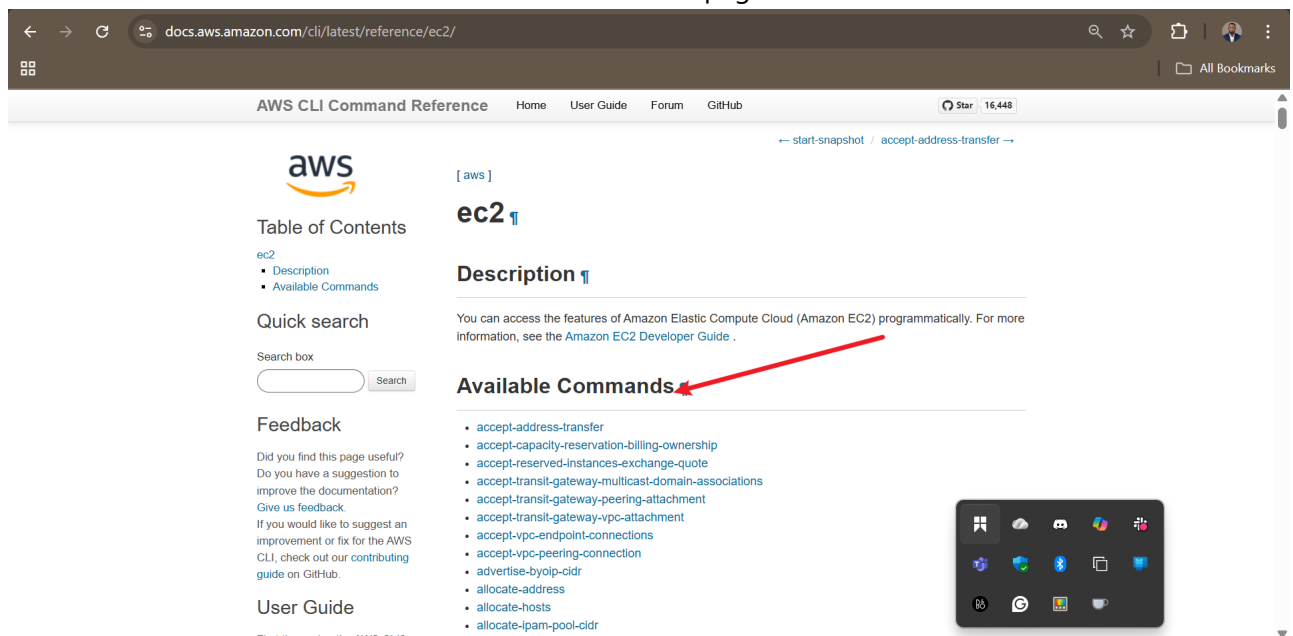
6. Search for AWS CLI command reference EC2.



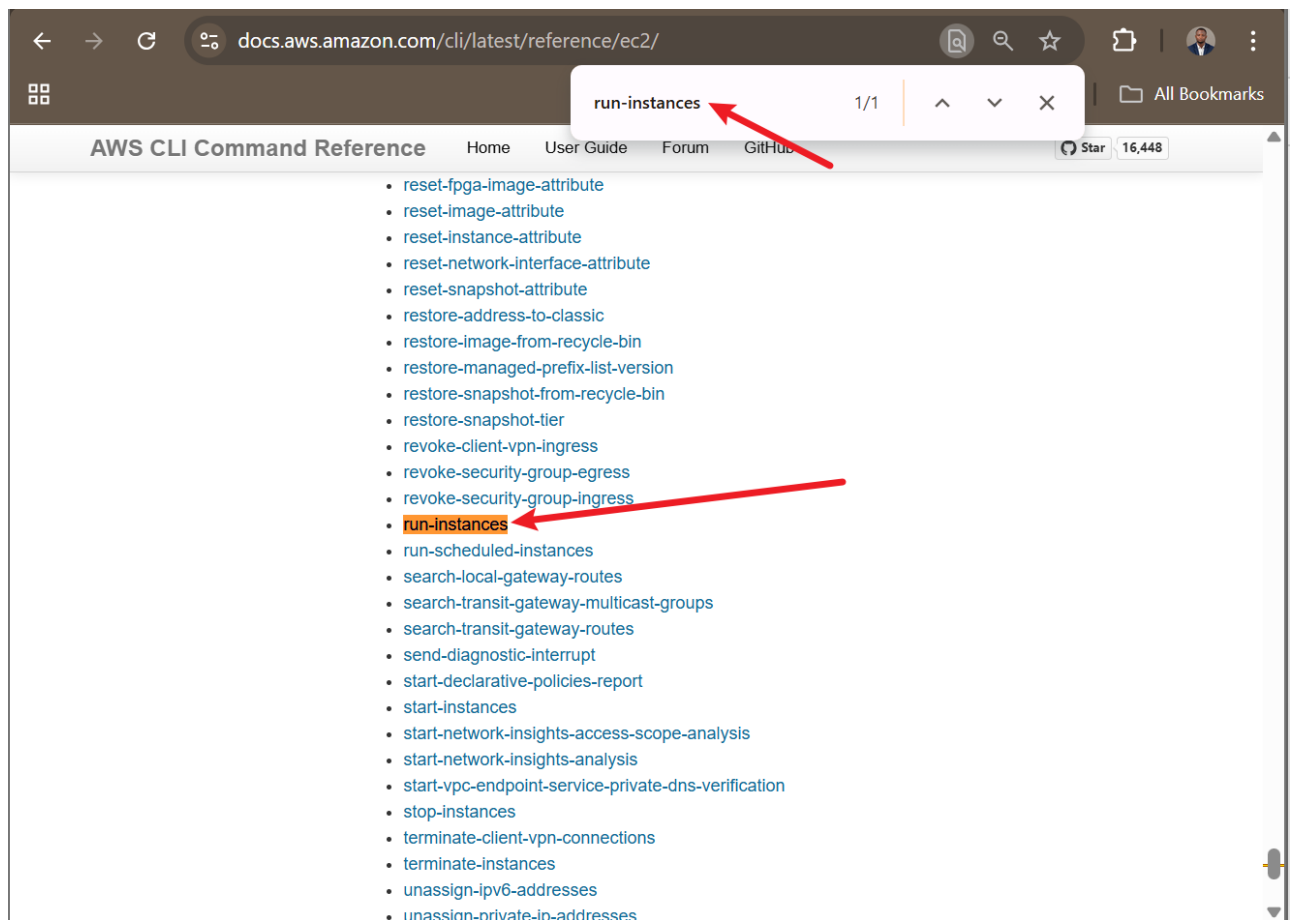
7. Click on the first link showing CLI latest reference EC2.



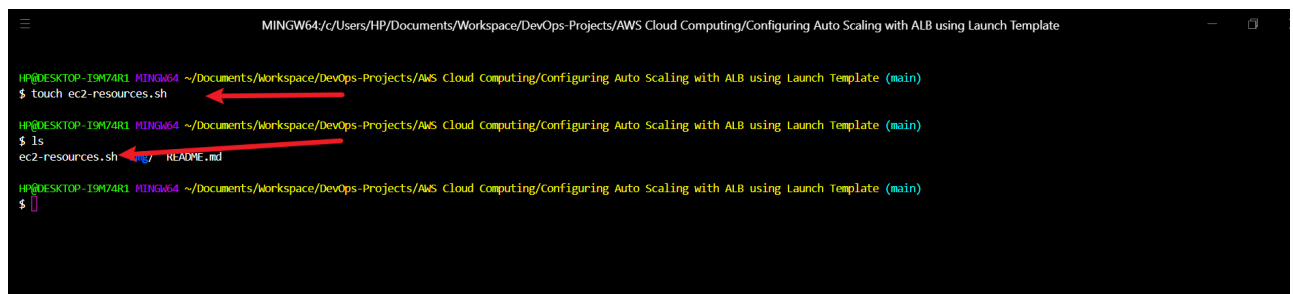
8. Now on AWS CLI commands official documentation webpage.



9. Search for run-instances with Ctrl-F and click on it when found.



10. Create `ec2-resources.sh`.



11. Add script to automate EC2 creation.

```

GNU nano 8.2 ec2-resources.sh
#!/bin/bash

# Environment variables
ENVIRONMENT=$1

check_num_of_args() {
    # Checking the number of arguments
    if [ "$#" -ne 1 ]; then
        echo "Usage: $0 <environments>"
        exit 1
    fi
}

activate_infra_environment() {
    # Acting based on the argument value
    if [ "$ENVIRONMENT" == "local" ]; then
        echo "Running script for Local Environment..."
    elif [ "$ENVIRONMENT" == "testing" ]; then
        echo "Running script for Testing Environment..."
    elif [ "$ENVIRONMENT" == "production" ]; then
        echo "Running script for Production Environment..."
    else
        echo "Invalid environment specified. Please use 'local', 'testing', or 'production'."
        exit 2
    fi
}

# Function to check if AWS CLI is installed
check_aws_cli() {
    if ! command -v aws &> /dev/null; then
        echo "AWS CLI is not installed. Please install it before proceeding."
        return 1
    fi
    return 1
}

```

12. Note that create_ec2_instances() will create two instances in a public subnet.

```

GNU nano 8.2 ec2-resources.sh
}

# Function to create EC2 Instances
create_ec2_instances() {
    # Specify the parameters for the EC2 instances
    instance_type="t2.micro"
    ami_id="resolve:ssm:/aws/service/ami-amazon-linux-latest/al2023-ami-kernel-default-x86_64"
    count=2
    region="us-east-2"
    subnet_id="subnet-06c7cfe7c9fb96a9b" # Replace with your public subnet ID
    security_group_id="sg-0012c41f2dcc7ac95" # Replace with your SG ID
    key_name="my-key-new-pair"

    echo "Launching EC2 instances in subnet: $subnet_id ..."

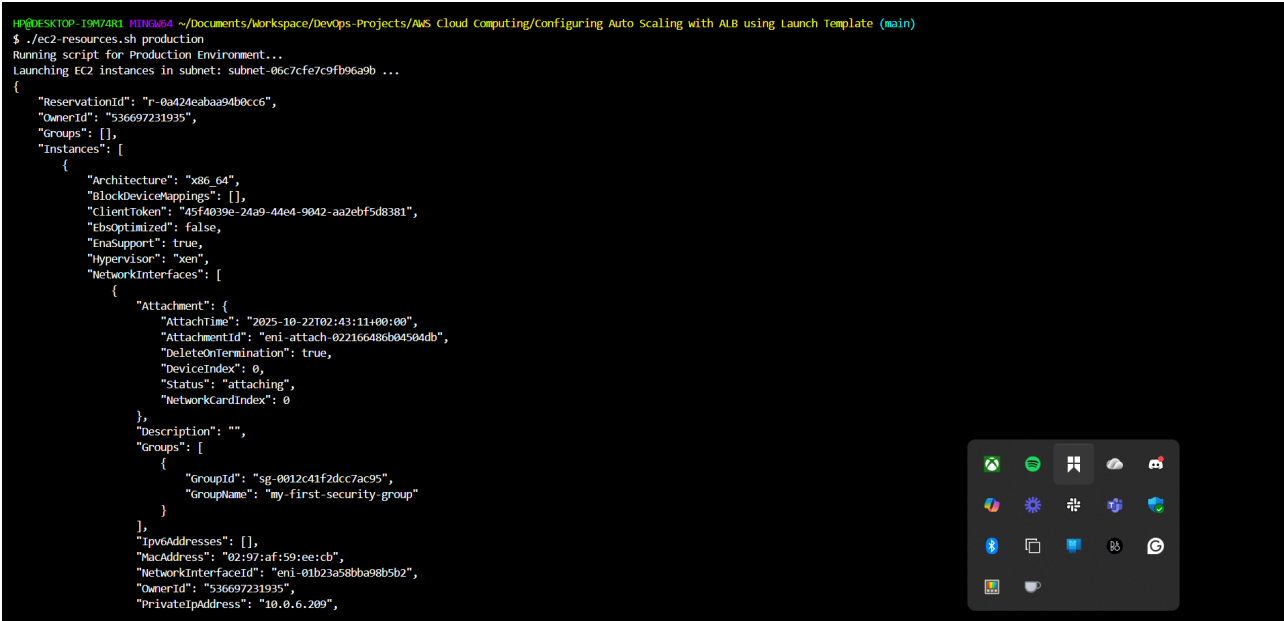
    # Launch EC2 instances with a proper network interface configuration
    aws ec2 run-instances \
        --image-id "$ami_id" \
        --instance-type "$instance_type" \
        --count $count \
        --key-name "$key_name" \
        --region "$region" \
        --network-interfaces "DeviceIndex=0,SubnetId=$subnet_id,AssociatePublicIpAddress=true,Groups=$security_group_id" \
        --tag-specifications "ResourceType=instance,Tags=[{Key=Name,Value=Custom-VPC-Instance}]"

    if [ $? -eq 0 ]; then
        echo "✅ EC2 instances created successfully in subnet $subnet_id."
    else
        echo "❌ Failed to create EC2 instances."
    fi
}

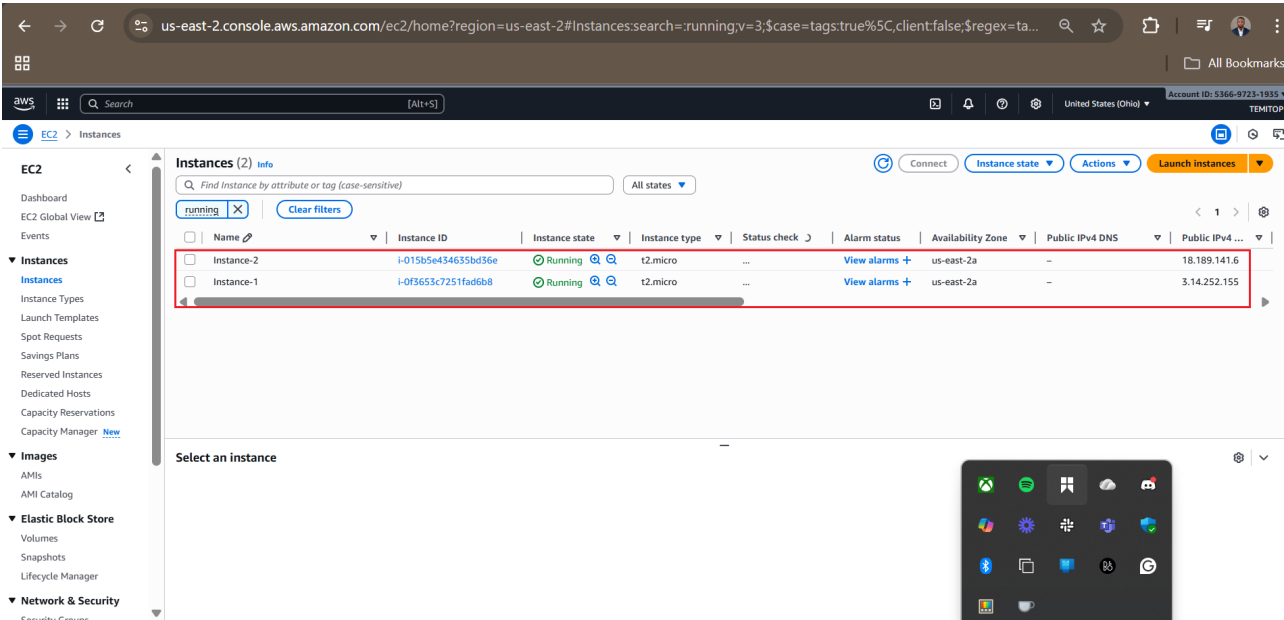
check_num_of_args "$@"
activate_infra_environment

```

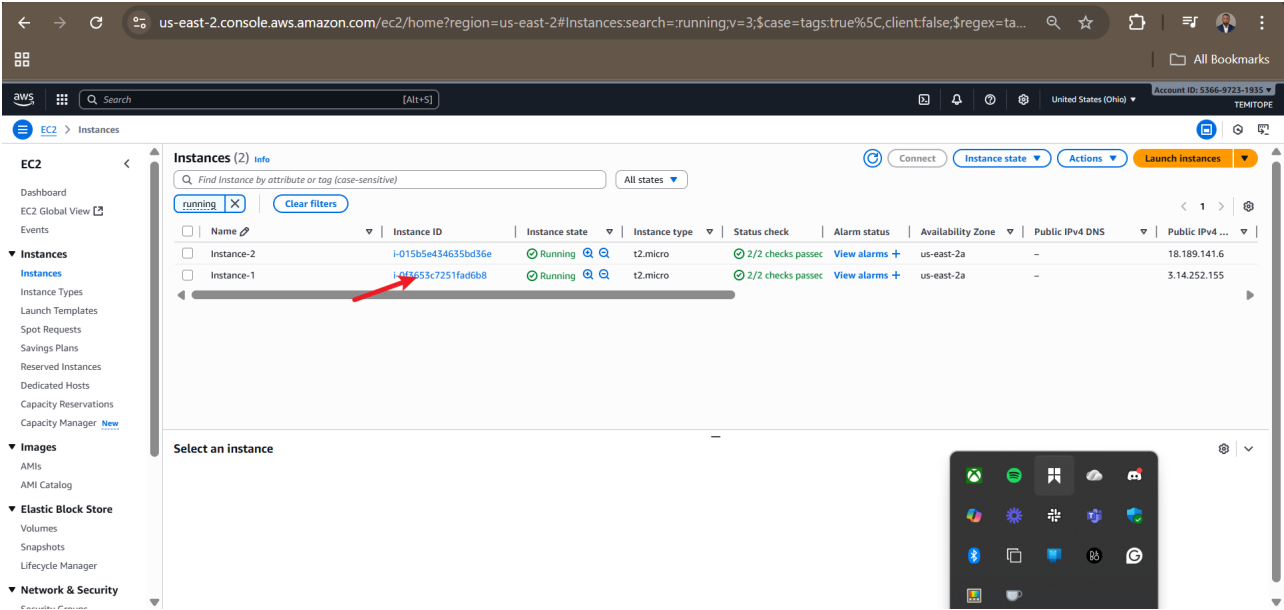
13. Execute script and EC2 resources successfully created.



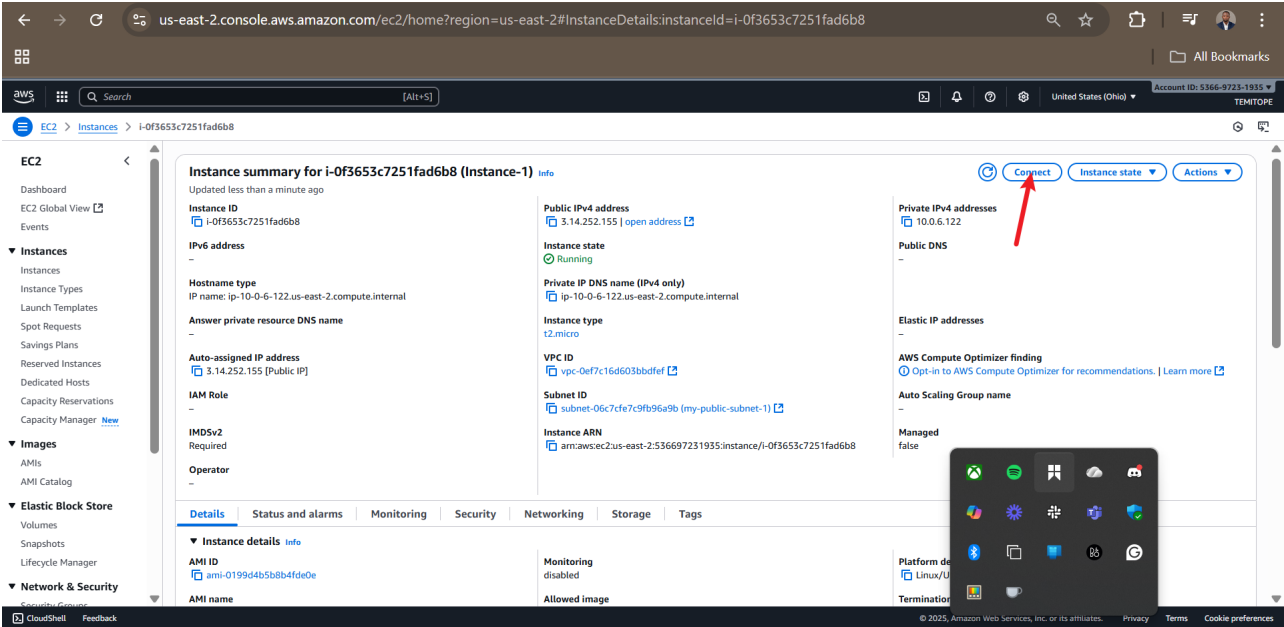
14. Two instances seen created and running on the console.



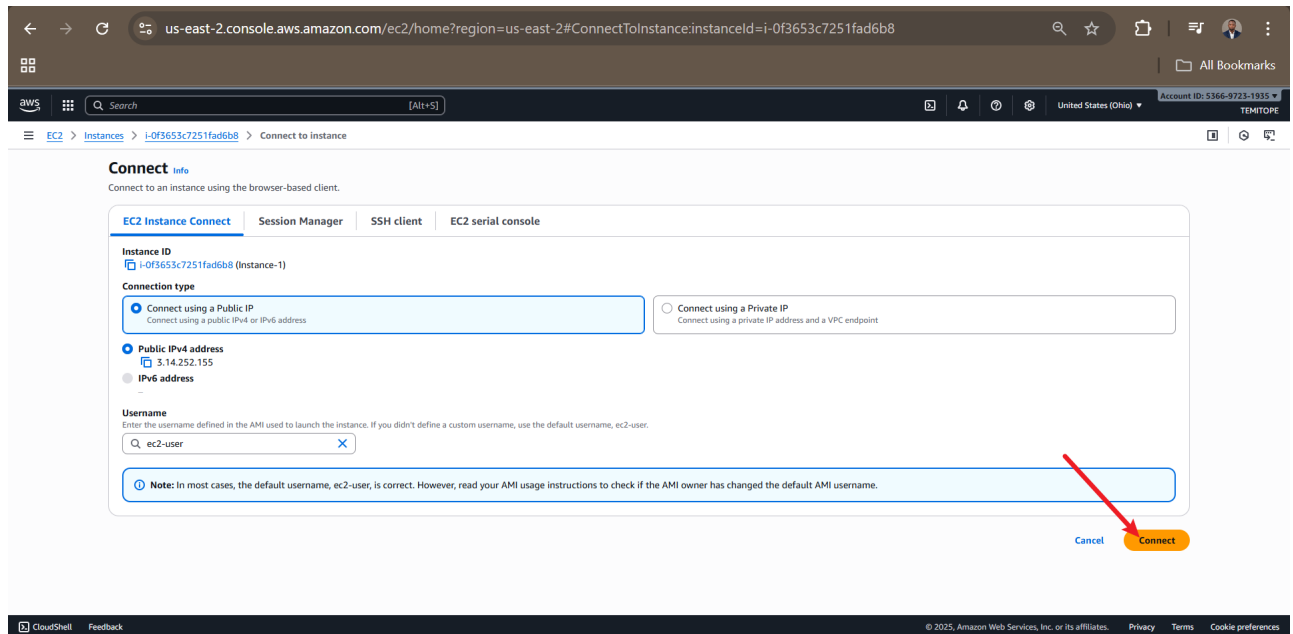
15. Click on instance-1 ID (may have another name in yours).



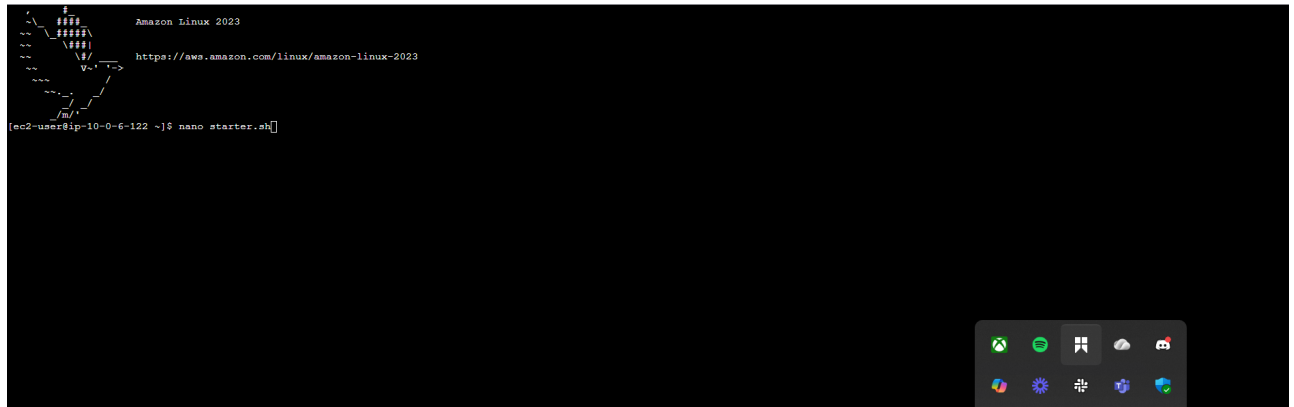
16. On instance-1 dashboard click connect.



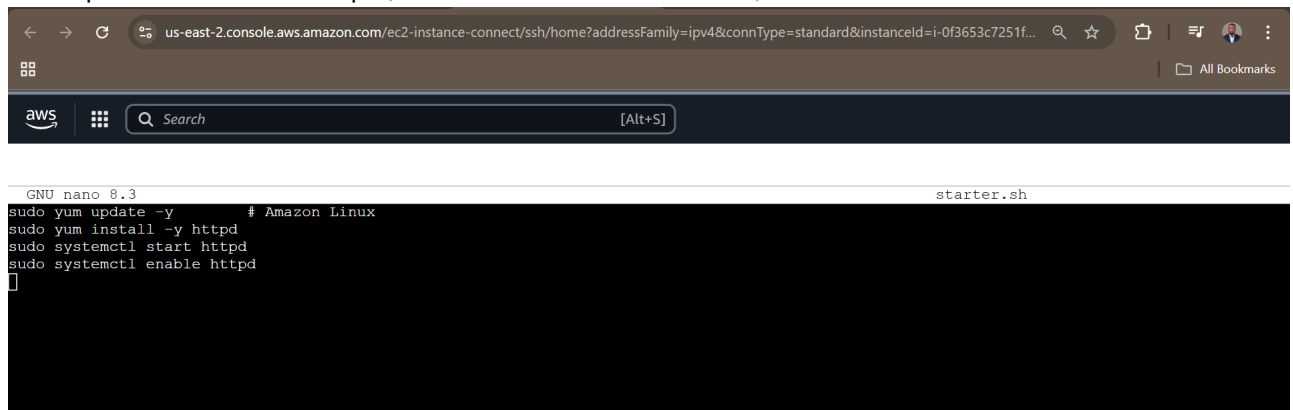
17. Under EC2 instance connect tab click connect.



18. Now connected create starter-sh with nano.



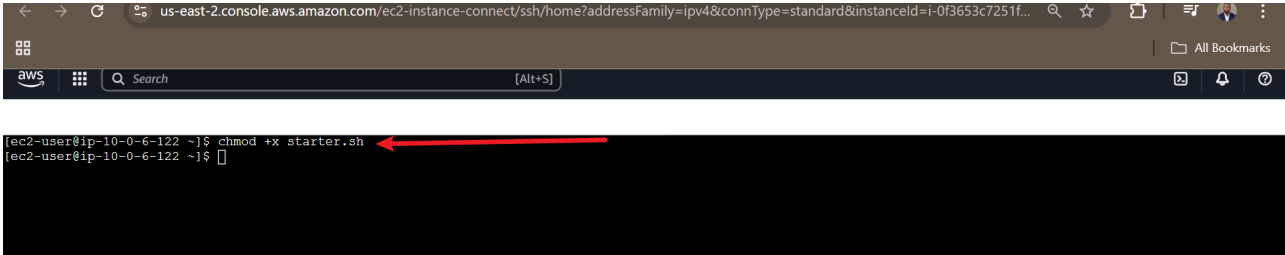
19. Add Apache installation script (I have Amazon Linux instance).



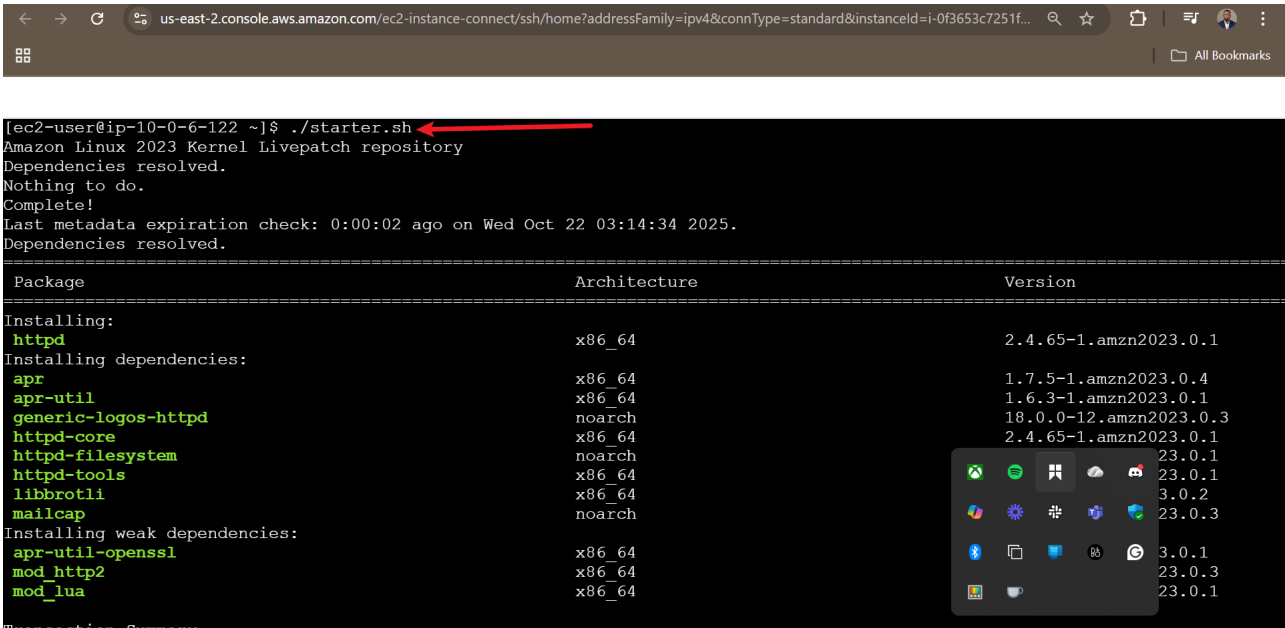
20. Save and exit (Ctrl-X and Y Enter).



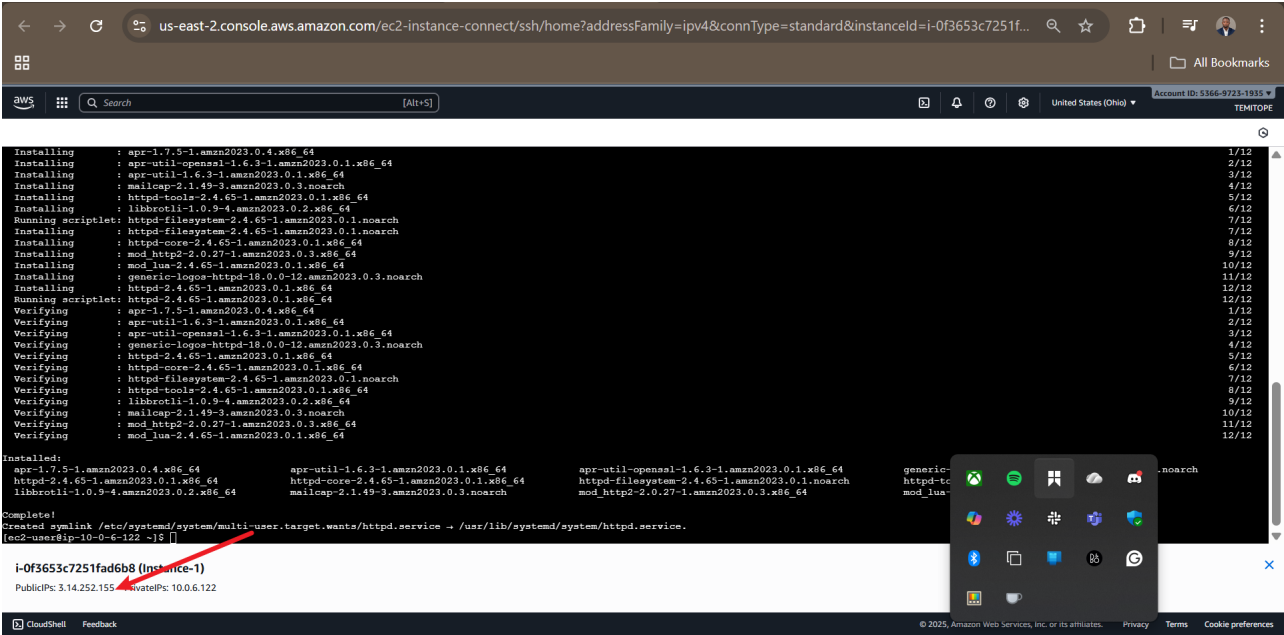
21. Turn starter-sh to executable (chmod +x).



22. Execute starter-sh to install httpd and enable it.



23. Copy the public IP.

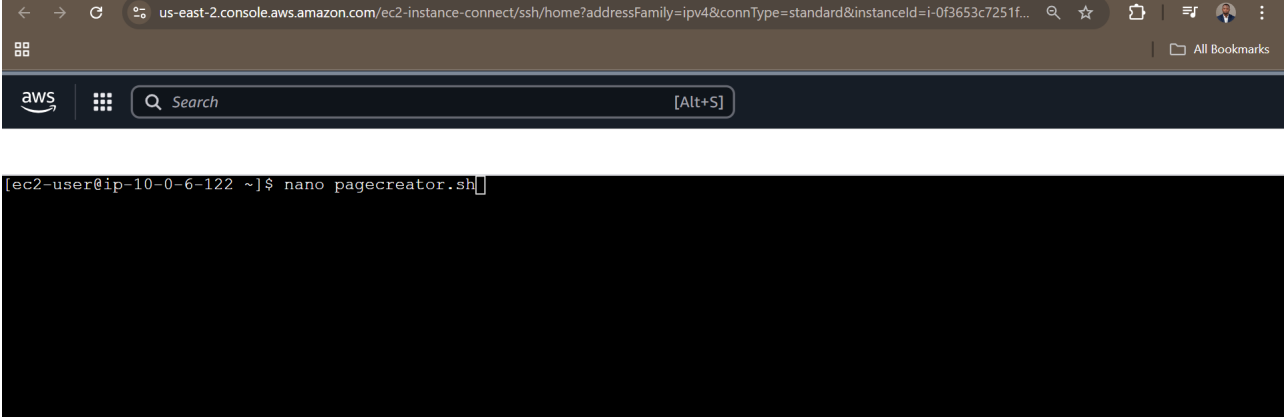


24. Paste it on a browser and visit it (via http) and the default page will be displayed.

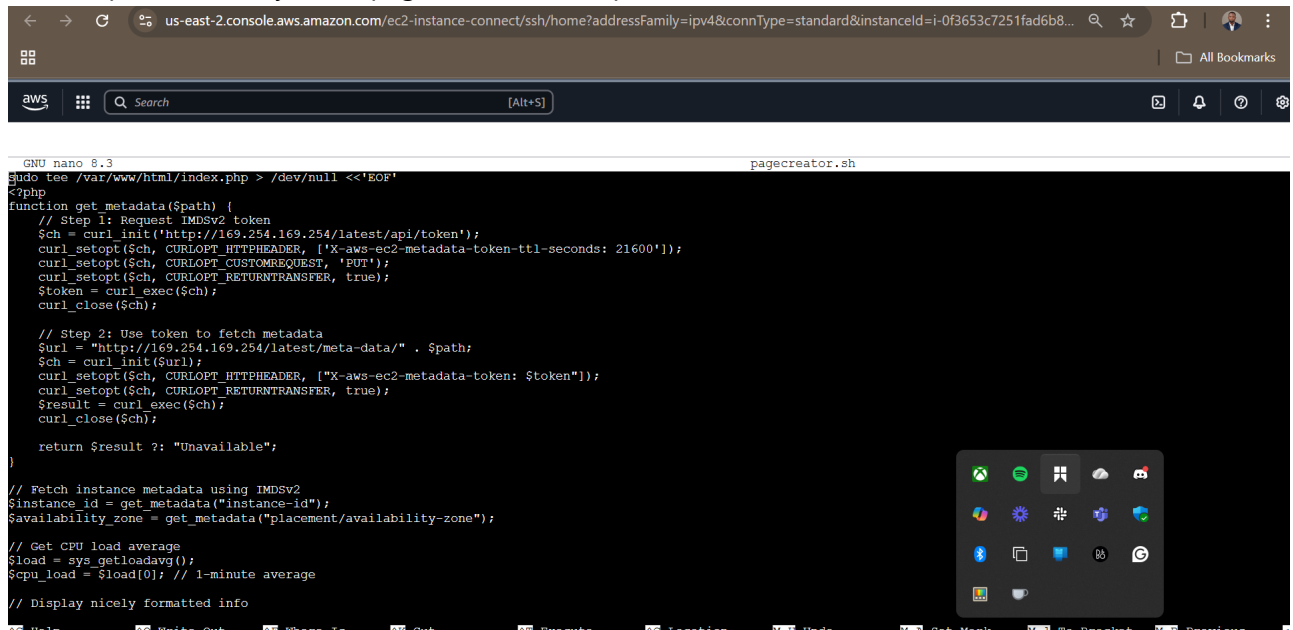


It works!

25. Back to instance connect CLI create pagecreator-sh (nano).



26. Add script to create dynamic page with PHP and provide data.



```
GNU nano 8.3 pagecreator.sh
sudo tee /var/www/html/index.php > /dev/null <<'EOF'
<?php
function get_metadata($path) {
    // Step 1: Request IMDSv2 token
    $sch = curl_init('http://169.254.169.254/latest/api/token');
    curl_setopt($sch, CURLOPT_HTTPHEADER, ['X-aws-ec2-metadata-token-ttl-seconds: 21600']);
    curl_setopt($sch, CURLOPT_CUSTOMREQUEST, 'PUT');
    curl_setopt($sch, CURLOPT_RETURNTRANSFER, true);
    $token = curl_exec($sch);
    curl_close($sch);

    // Step 2: Use token to fetch metadata
    $url = "http://169.254.169.254/latest/meta-data/" . $path;
    $sch = curl_init($url);
    curl_setopt($sch, CURLOPT_HTTPHEADER, ["X-aws-ec2-metadata-token: $token"]);
    curl_setopt($sch, CURLOPT_RETURNTRANSFER, true);
    $result = curl_exec($sch);
    curl_close($sch);

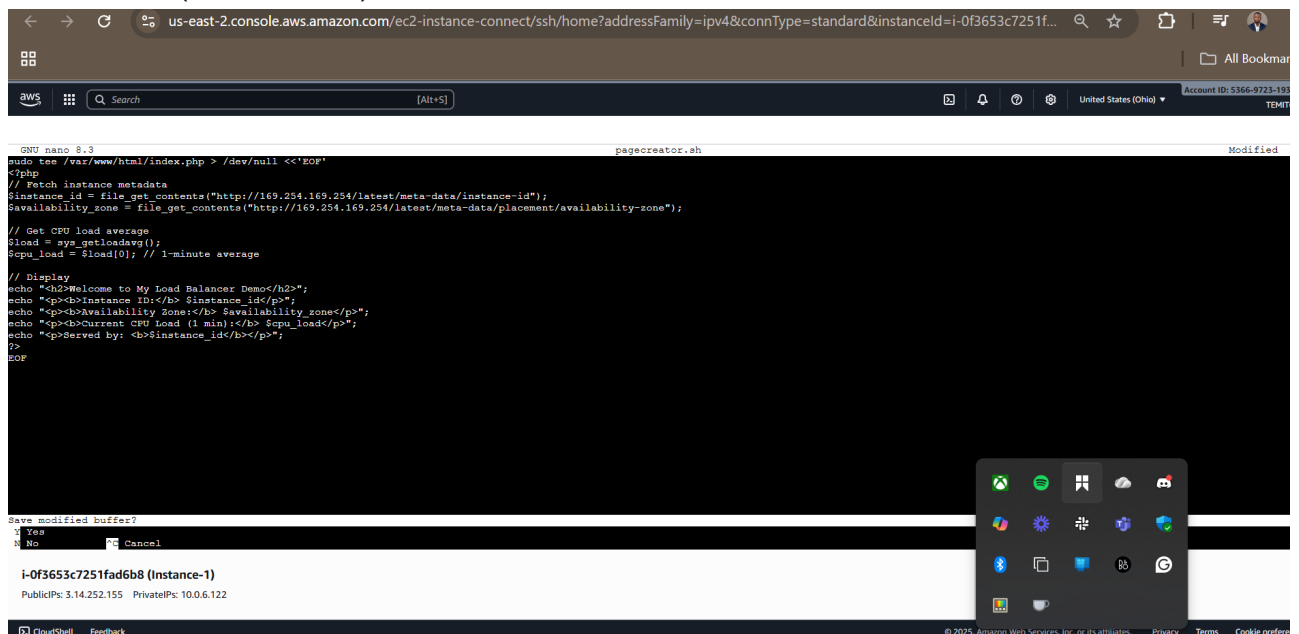
    return $result ?: "Unavailable";
}

// Fetch instance metadata using IMDSv2
$instance_id = get_metadata("instance-id");
$availability_zone = get_metadata("placement/availability-zone");

// Get CPU load average
$load = sys_getloadavg();
$cpu_load = $load[0]; // 1-minute average

// Display nicely formatted info
echo "Instance ID: $instance_id\n";
echo "Availability Zone: $availability_zone\n";
echo "CPU Load (1 min): $cpu_load\n";
EOF
```

27. Save and exit (Ctrl+X Y Enter).



```
GNU nano 8.3 pagecreator.sh Modified
sudo tee /var/www/html/index.php > /dev/null <<'EOF'
<?php
// Fetch instance metadata
$instance_id = file_get_contents("http://169.254.169.254/latest/meta-data/instance-id");
$availability_zone = file_get_contents("http://169.254.169.254/latest/meta-data/placement/availability-zone");

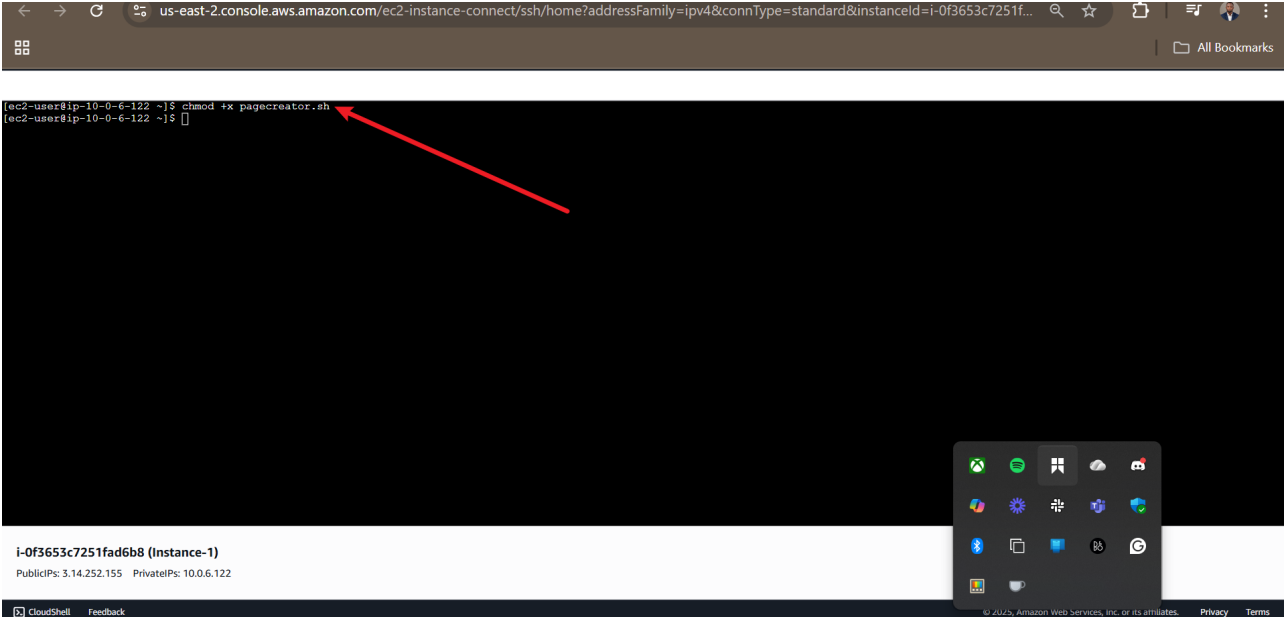
// Get CPU load average
$load = sys_getloadavg();
$cpu_load = $load[0]; // 1-minute average

// Display
echo "<h2>Welcome to My Load Balancer Demo</h2>";
echo "<p><b>Instance ID:</b> $instance_id</p>";
echo "<p><b>Availability Zone:</b> $availability_zone</p>";
echo "<p><b>Current CPU Load (1 min):</b> $cpu_load</p>";
echo "<p>Served by: <b>$instance_id</b></p>";
EOF

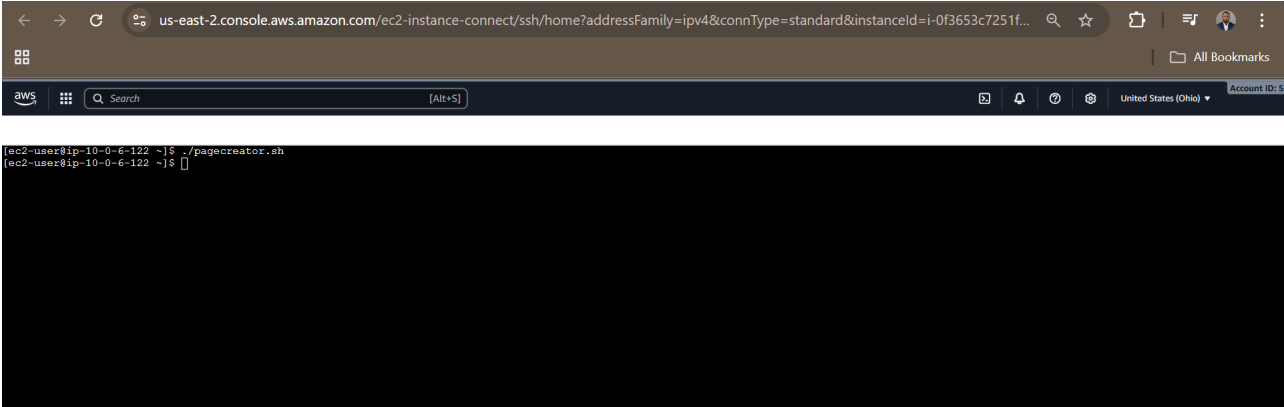
Save modified buffer?
Y Yes
N No Cancel

i-0f3653c7251fad6b8 (Instance-1)
PublicIPs: 3.14.252.155 PrivateIPs: 10.0.6.122
```

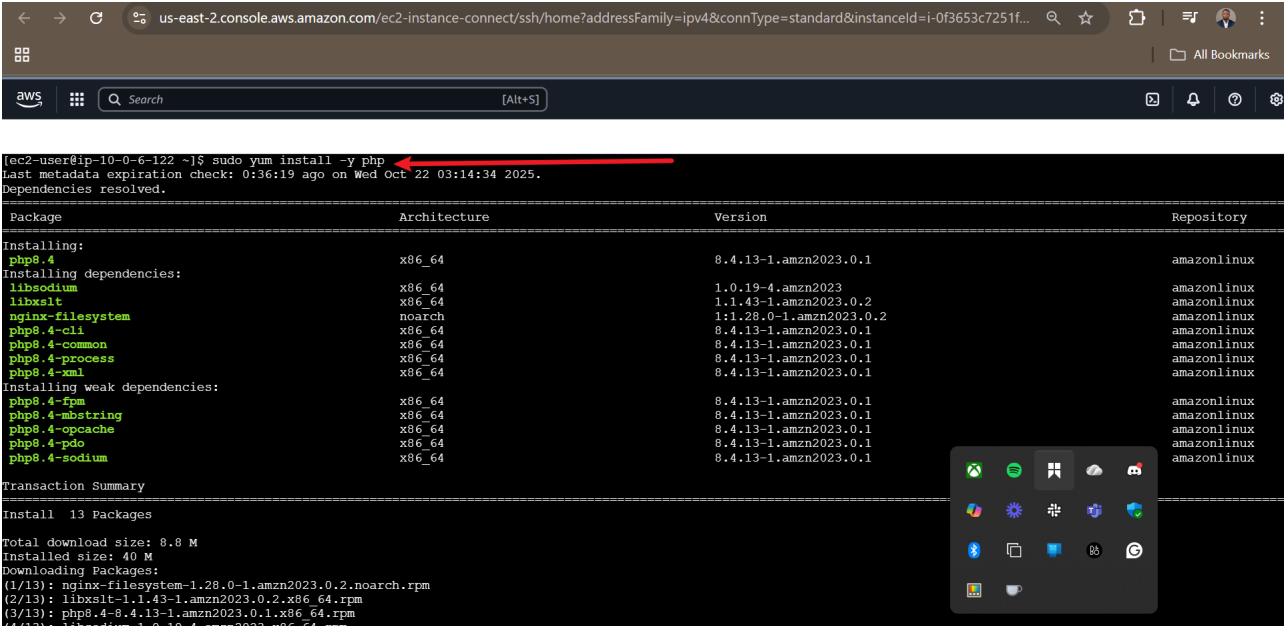

28. Turn pagecreator-sh to executable (chmod +x).



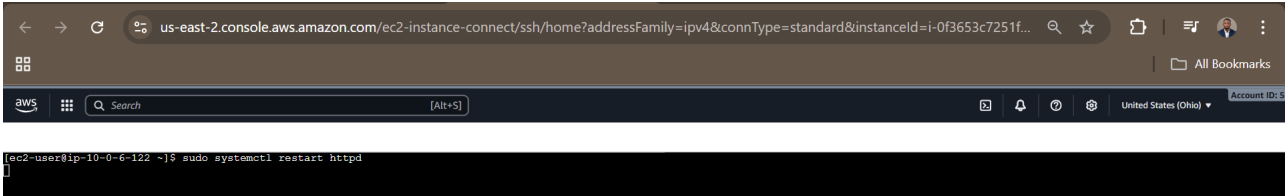
29. Execute page creator-sh to create page.



30. Yum install PHP.



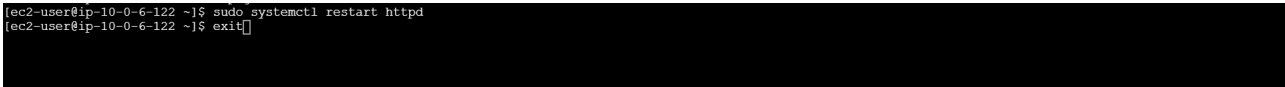
31. Restart Apache with systemctl.



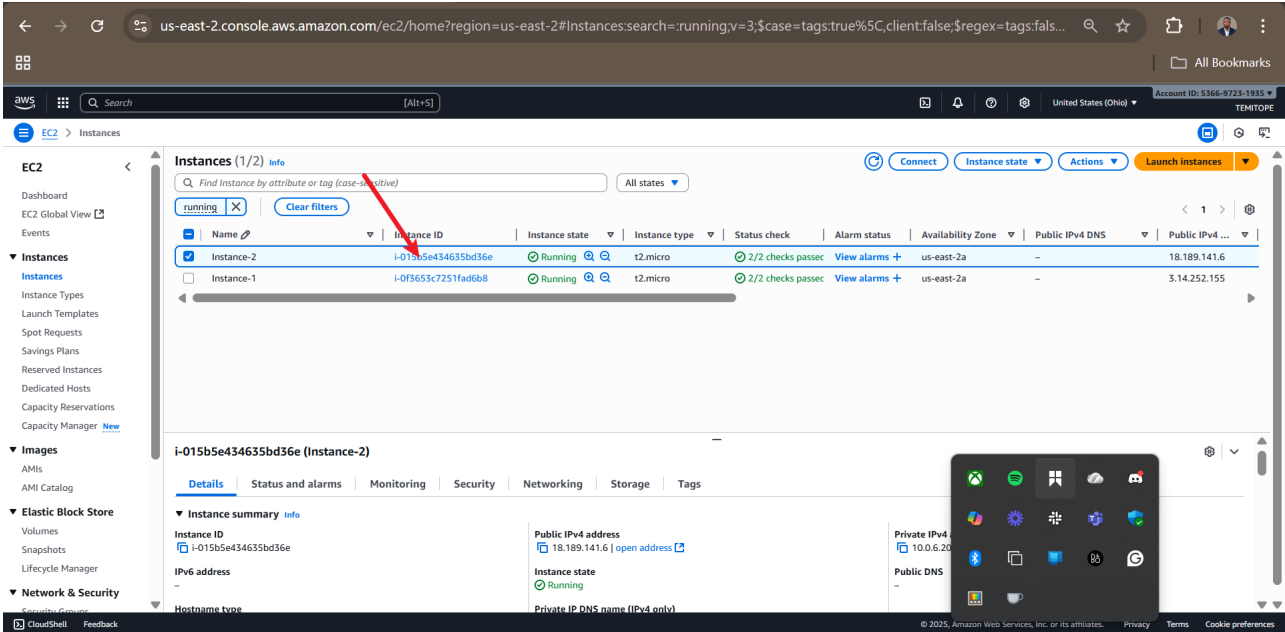
32. Visit the page again website now display instance-1 details.



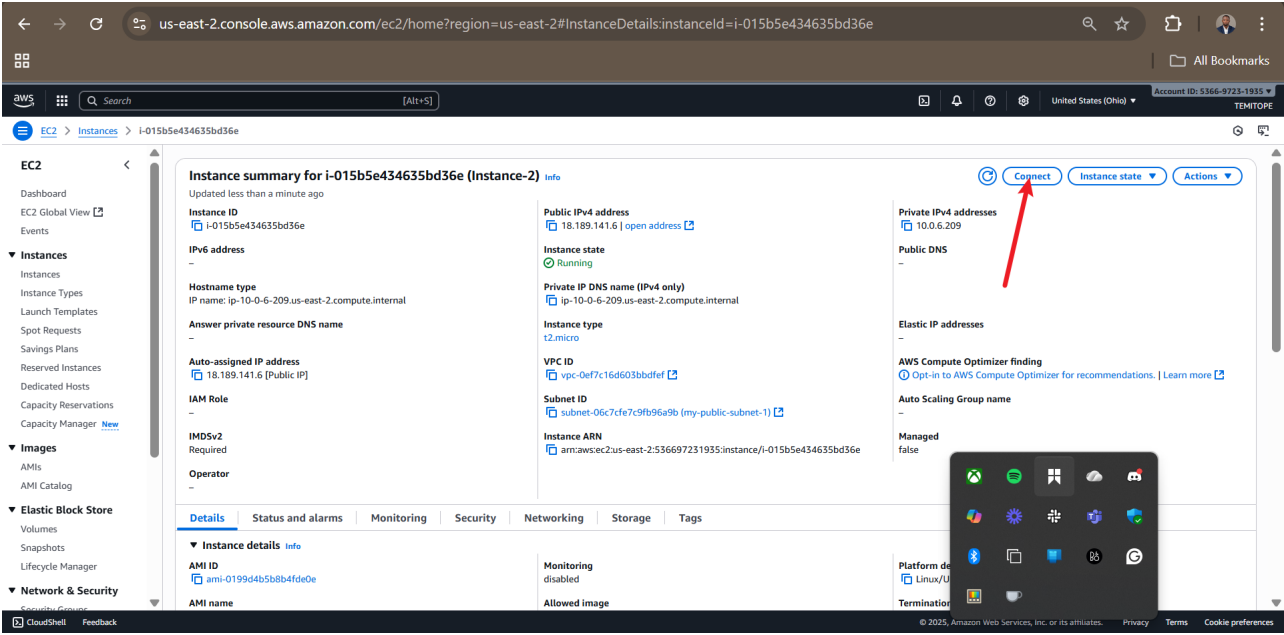
33. Exit CLI.



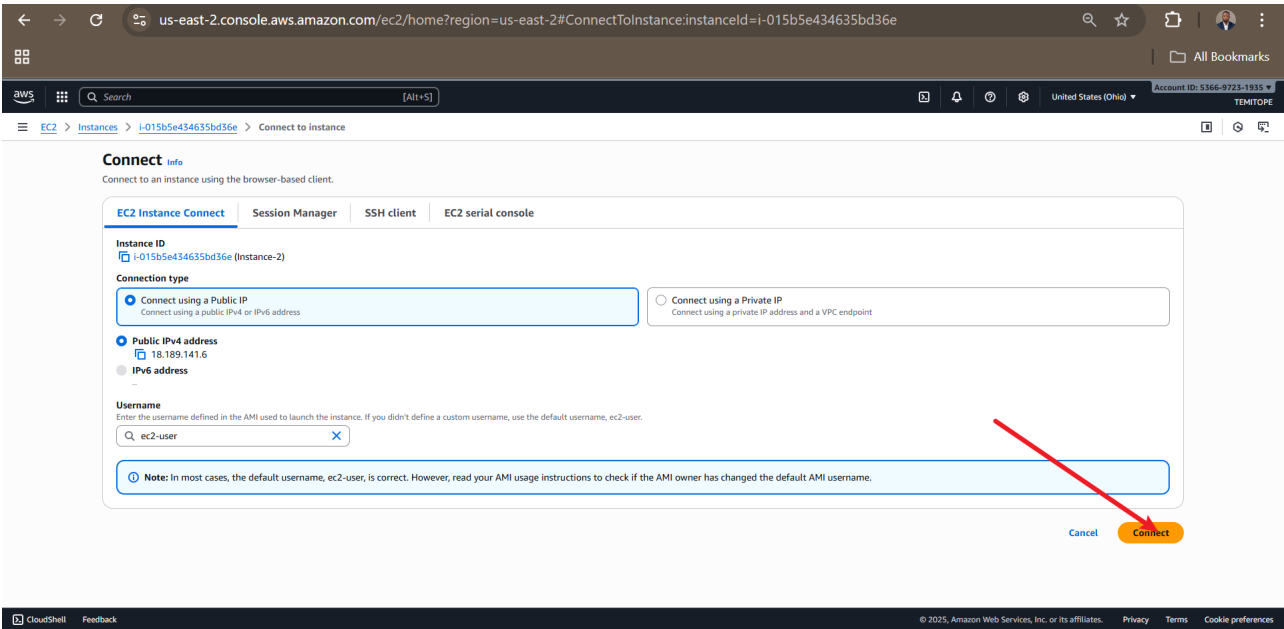
34. Navigate back to instances dashboard and click on instance-2 ID.



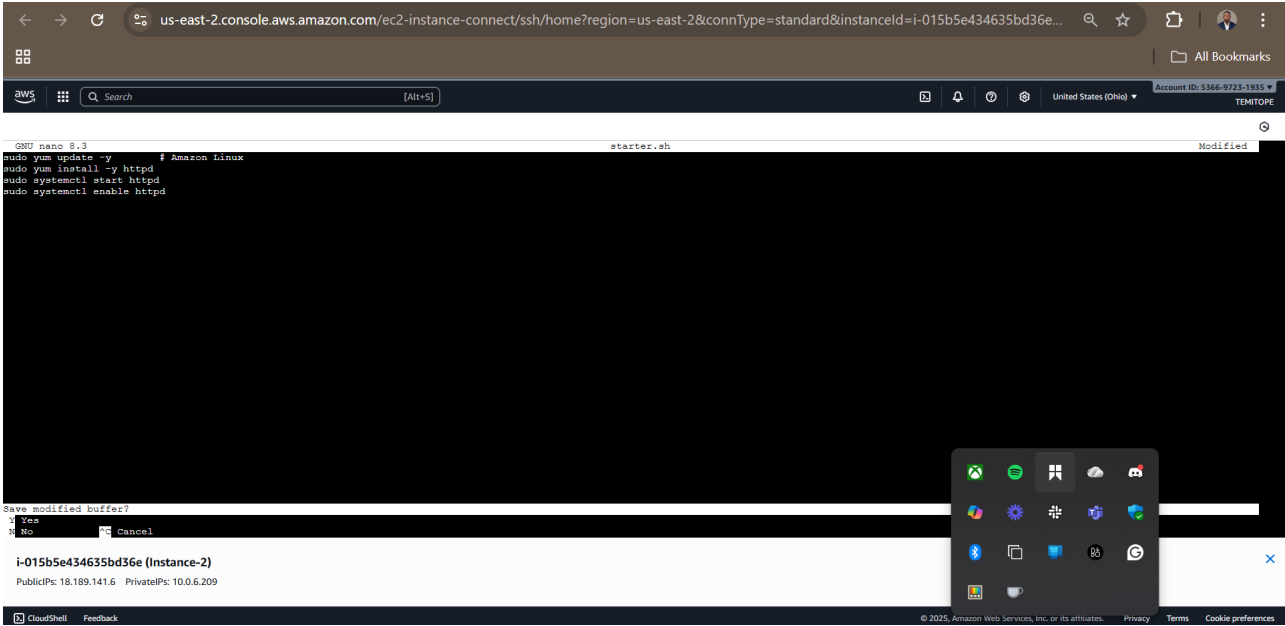
35. On instance-2 dashboard click connect.



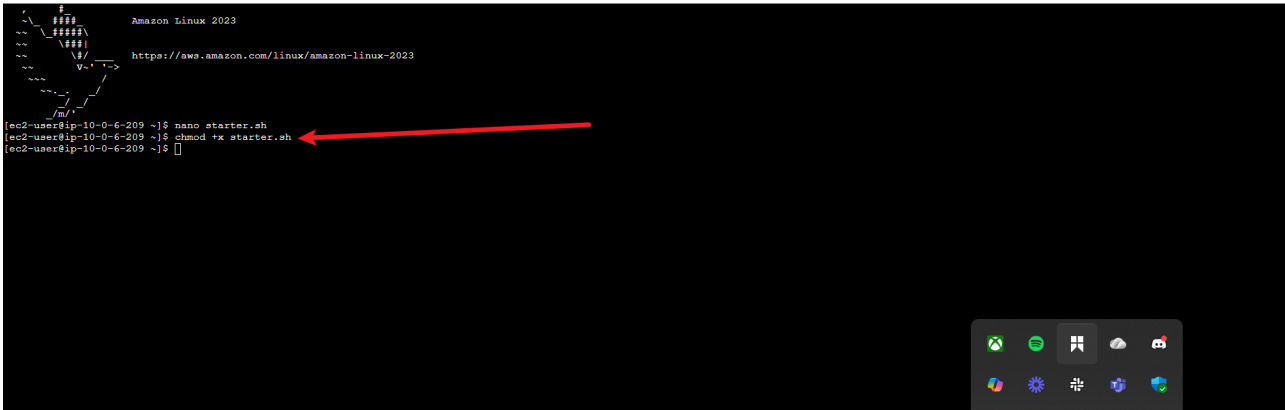
36. Under EC2 instance connect tab click connect.



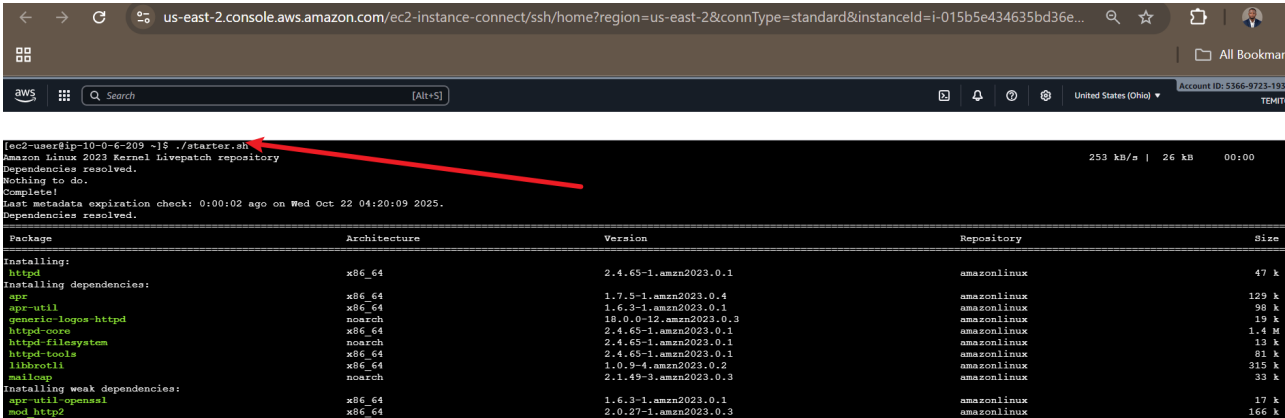
37. Nano starter-sh add Apache installation script save and exit.



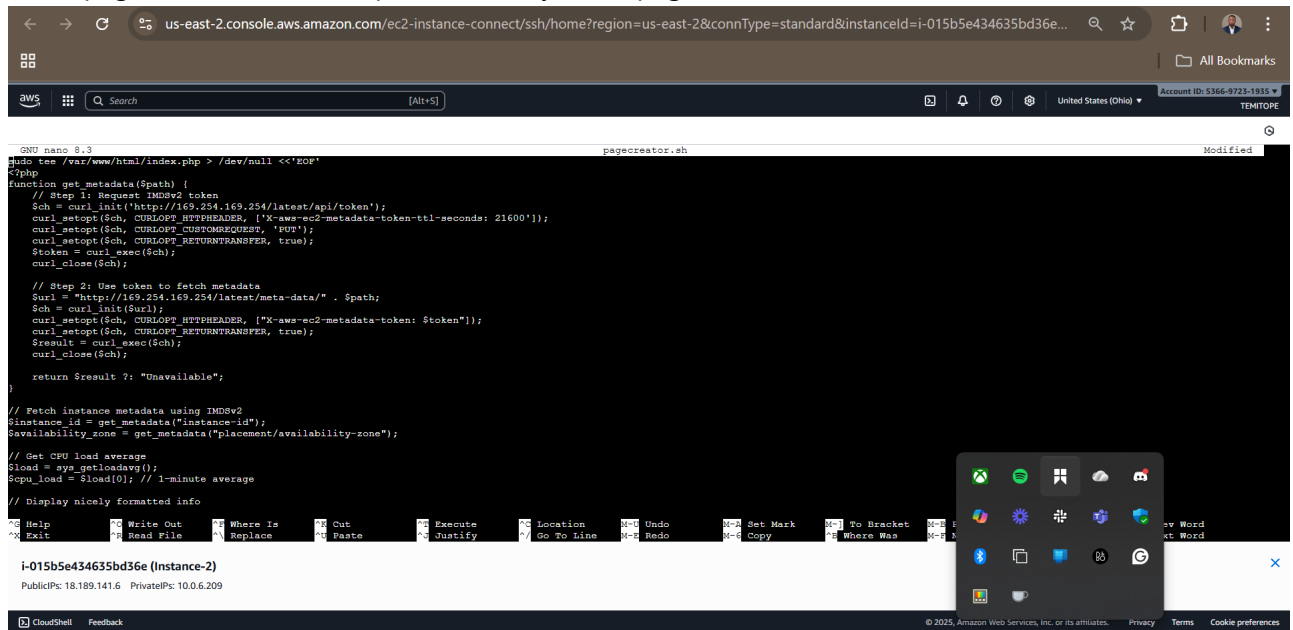
38. Turn starter-sh to executable (chmod +x).



39. Execute starter-sh to install httpd and enable it.



40. Nano pagecreator-sh add script to create dynamic page.



```
GNU nano 8.3 pagecreator.sh
sudo tee /var/www/html/index.php > /dev/null <<'EOF'
<:php
function get_metadata($path) {
    // Step 1: Request IMDSv2 token
    $ch = curl_init('http://169.254.169.254/latest/api/token');
    curl_setopt($ch, CURLOPT_HTTPHEADER, ['X-aws-ec2-metadata-token-ttl-seconds: 21600']);
    curl_setopt($ch, CURLOPT_CUSTOMREQUEST, 'PUT');
    curl_setopt($ch, CURLOPT_RETURNTRANSFER, true);
    $token = curl_exec($ch);
    curl_close($ch);

    // Step 2: Use token to fetch metadata
    $url = "http://169.254.169.254/latest/meta-data/" . $path;
    $ch = curl_init($url);
    curl_setopt($ch, CURLOPT_HTTPHEADER, ["X-aws-ec2-metadata-token: $token"]);
    curl_setopt($ch, CURLOPT_RETURNTRANSFER, true);
    $result = curl_exec($ch);
    curl_close($ch);

    return $result ?: "Unavailable";
}

// Fetch instance metadata using IMDSv2
$instance_id = get_metadata("instance-id");
$availability_zone = get_metadata("placement/availability-zone");

// Get CPU load average
$load = sys_getloadavg();
$cpu_load = $load[0]; // 1-minute average

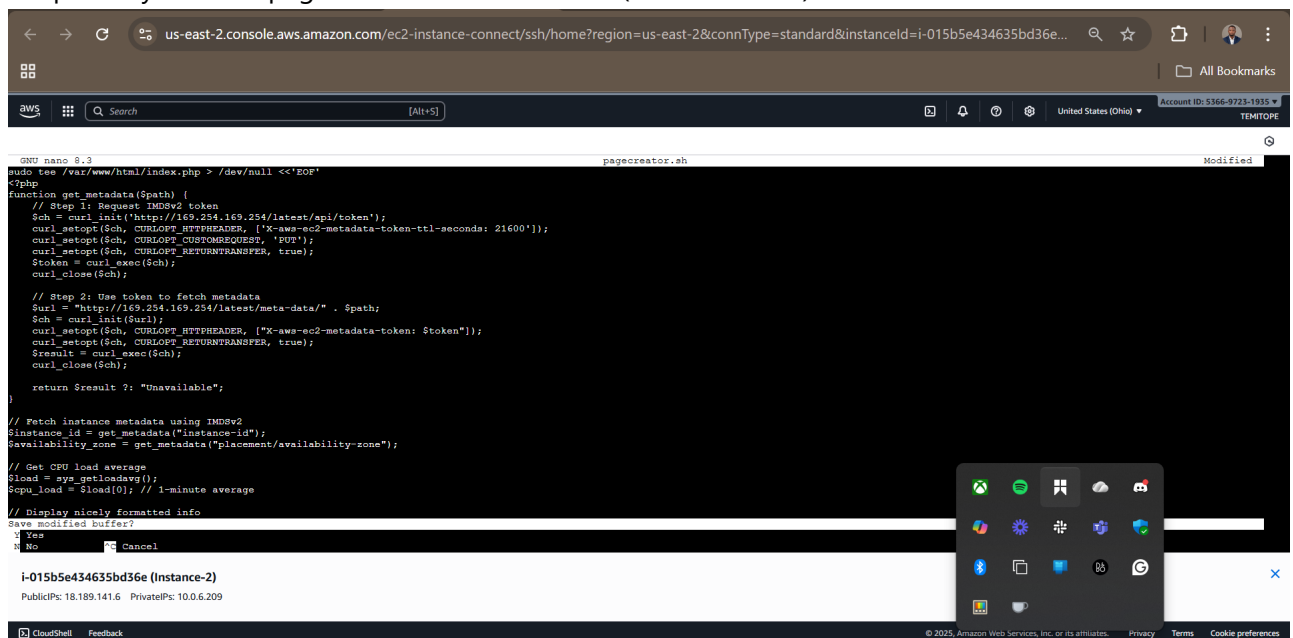
// Display nicely formatted info
}

Help Write Out Where Is Out Execute Location Undo Get Mark To Bracket
Exit Read File Replace Paste Justify Go To Line Redo Copy Where Was

i-015b5e434635bd36e (Instance-2)
PublicIPs: 18.189.141.6 PrivateIPs: 10.0.6.209

CloudShell Feedback
```

41. Script ready to serve page and data save and exit (Ctrl+X Y Enter).



```
GNU nano 8.3 pagecreator.sh
sudo tee /var/www/html/index.php > /dev/null <<'EOF'
<:php
function get_metadata($path) {
    // Step 1: Request IMDSv2 token
    $ch = curl_init('http://169.254.169.254/latest/api/token');
    curl_setopt($ch, CURLOPT_HTTPHEADER, ['X-aws-ec2-metadata-token-ttl-seconds: 21600']);
    curl_setopt($ch, CURLOPT_CUSTOMREQUEST, 'PUT');
    curl_setopt($ch, CURLOPT_RETURNTRANSFER, true);
    $token = curl_exec($ch);
    curl_close($ch);

    // Step 2: Use token to fetch metadata
    $url = "http://169.254.169.254/latest/meta-data/" . $path;
    $ch = curl_init($url);
    curl_setopt($ch, CURLOPT_HTTPHEADER, ["X-aws-ec2-metadata-token: $token"]);
    curl_setopt($ch, CURLOPT_RETURNTRANSFER, true);
    $result = curl_exec($ch);
    curl_close($ch);

    return $result ?: "Unavailable";
}

// Fetch instance metadata using IMDSv2
$instance_id = get_metadata("instance-id");
$availability_zone = get_metadata("placement/availability-zone");

// Get CPU load average
$load = sys_getloadavg();
$cpu_load = $load[0]; // 1-minute average

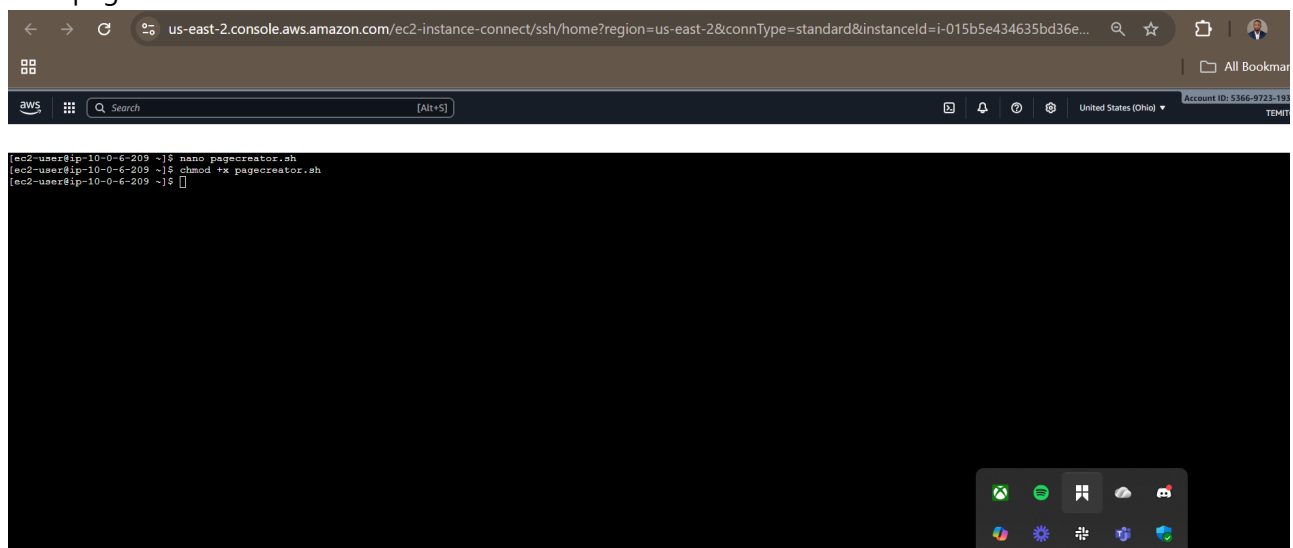
// Display nicely formatted info
}

Save modified buffer
^Y Yes
^N No
^C Cancel

i-015b5e434635bd36e (Instance-2)
PublicIPs: 18.189.141.6 PrivateIPs: 10.0.6.209

CloudShell Feedback
```

42. Turn page creator-sh to executable and execute it.



```
[ec2-user@ip-10-0-6-209 ~]$ nano pagecreator.sh
[ec2-user@ip-10-0-6-209 ~]$ chmod +x pagecreator.sh
[ec2-user@ip-10-0-6-209 ~]$
```

```

[ec2-user@ip-10-0-6-209 ~]$ sudo yum install -y php
Last metadata expiration check: 0:18:55 ago on Wed Oct 22 04:20:09 2025.
Dependencies resolved.

```

Package	Architecture	Version	Repository	Size
Installing:				
php8.4	x86_64	8.4.13-1.amzn2023.0.1	amazonlinux	16 K
Installing dependencies:				
libsodium	x86_64	1.0.19-4.amzn2023	amazonlinux	176 K
libxslt	x86_64	1.1.43-1.amzn2023.0.2	amazonlinux	183 K
nginxfilesystem	noarch	1:1.26.0-1.amzn2023.0.2	amazonlinux	9.6 K
php8.4-cgi	x86_64	8.4.13-1.amzn2023.0.1	amazonlinux	3.8 K
php8.4-common	x86_64	8.4.13-1.amzn2023.0.1	amazonlinux	800 K
php8.4-process	x86_64	8.4.13-1.amzn2023.0.1	amazonlinux	51 K
php8.4-xml	x86_64	8.4.13-1.amzn2023.0.1	amazonlinux	675 K
Installing weak dependencies:				
php8.4-fpm	x86_64	8.4.13-1.amzn2023.0.1	amazonlinux	2.0 M
php8.4-mcrypt	x86_64	8.4.13-1.amzn2023.0.1	amazonlinux	540 K
php8.4-opcache	x86_64	8.4.13-1.amzn2023.0.1	amazonlinux	490 K
php8.4-pdo	x86_64	8.4.13-1.amzn2023.0.1	amazonlinux	99 K
php8.4-sodium	x86_64	8.4.13-1.amzn2023.0.1	amazonlinux	47 K

```

Transaction Summary
Install 13 Packages

Total download size: 8.8 M
Installed size: 40 M
Downloading Packages:
(1/13): nginxfilesystem-1.28.0-1.amzn2023.0.2.noarch.rpm
(2/13): libxslt-1.1.43-1.amzn2023.0.2.x86_64.rpm
(3/13): php8.4-8.4.13-1.amzn2023.0.1.x86_64.rpm
(4/13): libsodium-1.0.19-4.amzn2023.x86_64.rpm
(5/13): php8.4-cgi-8.4.13-1.amzn2023.0.1.x86_64.rpm

```

i-015b5e434635bd36e (Instance-2)
PublicIPs: 18.189.141.6 PrivateIPs: 10.0.6.209

The screenshot shows the AWS CloudShell interface. The terminal window displays the following commands and output:

```
[ec2-user@ip-10-0-6-209 ~]$ sudo systemctl restart httpd
[ec2-user@ip-10-0-6-209 ~]$ []
```

At the bottom of the terminal window, the instance ID **i-015b5e434635bd36c (instance-2)** is displayed, with a red arrow pointing to it. The public IP address is 18.189.141.6 and the private IP address is 10.0.6.209.

All Bookmarks

Welcome to Instance-2

Instance ID:

i-015b5e434635bd36e

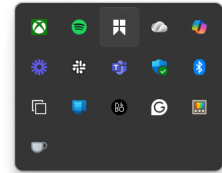
Availability Zone:

us-east-2a

Current CPU Load (1 min):

0%

Served by: i-015b5e434635bd36e



us-east-2.console.aws.amazon.com/ec2/home?region=us-east-2#Instances:

aws [Alt+S] United States (Ohio) Account ID: 5366-9723-1935

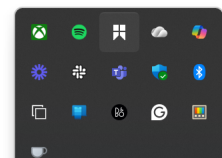
EC2 > Instances

Instances (2) info

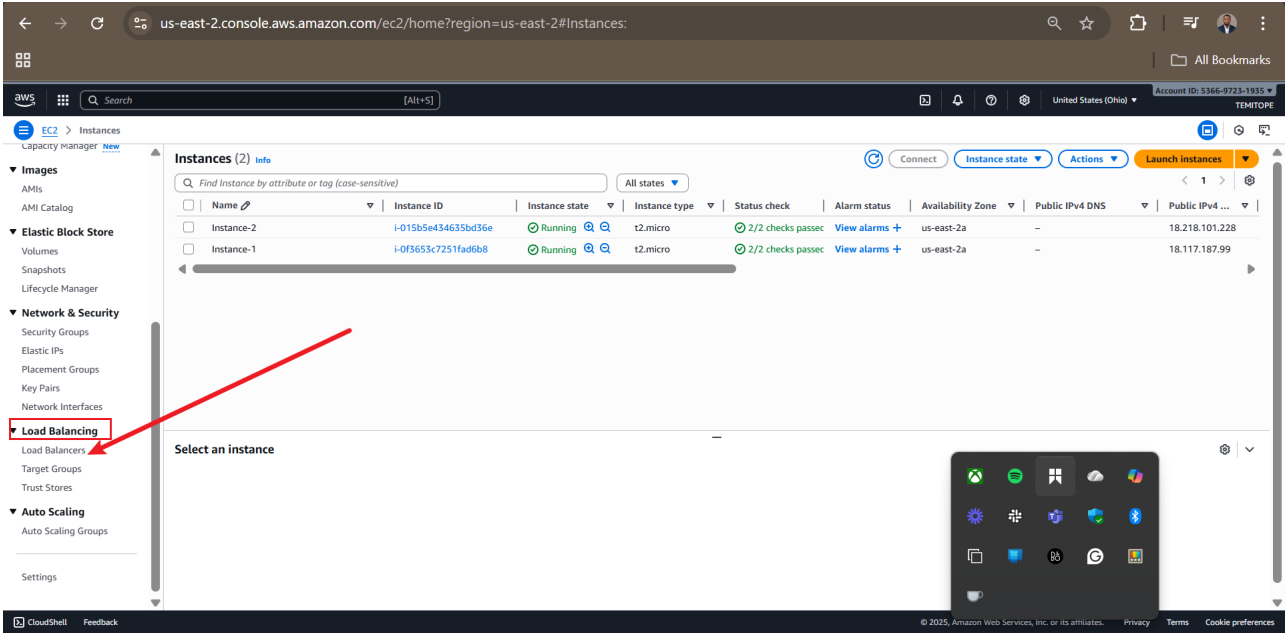
All states

☐	Name	Instance ID	Instance state	Instance type	Status check	Alarm status	Availability Zone	Public IPv4 DNS	Public IPv4 ...
☐	Instance-2	i-015b5e434635bd36e	Running	t2.micro	2/2 checks passed	View alarms +	us-east-2a	-	18.218.101.228
☐	Instance-1	i-0f3653c7251fad6b8	Running	t2.micro	2/2 checks passed	View alarms +	us-east-2a	-	18.117.187.99

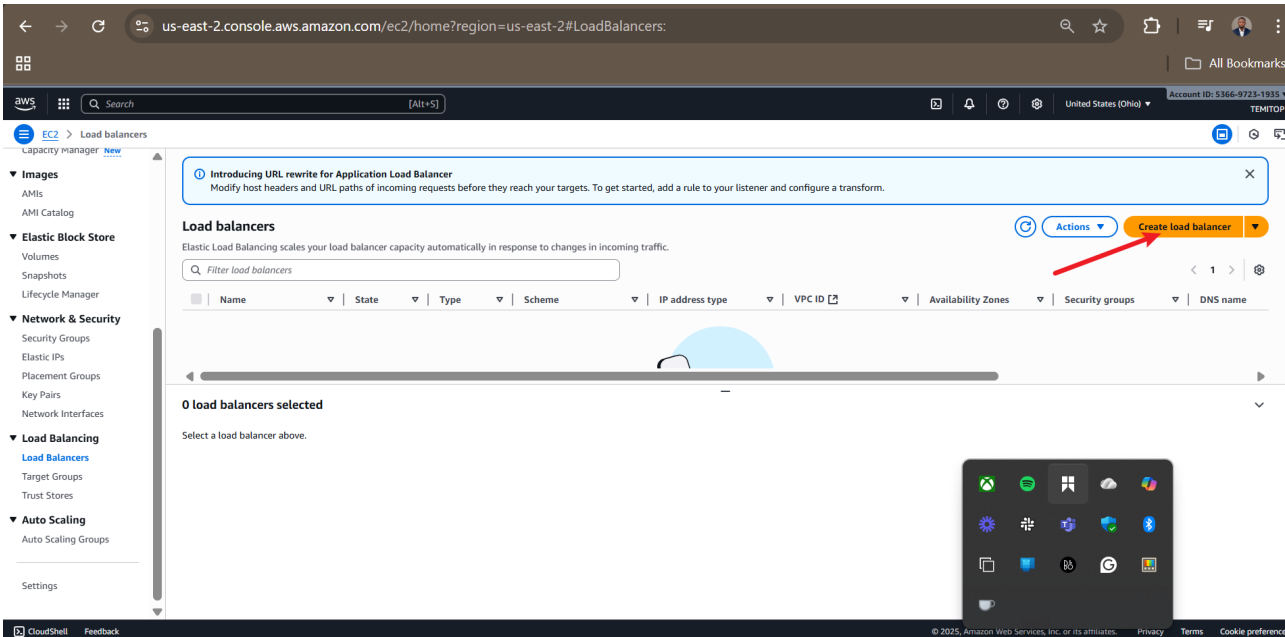
Select an instance



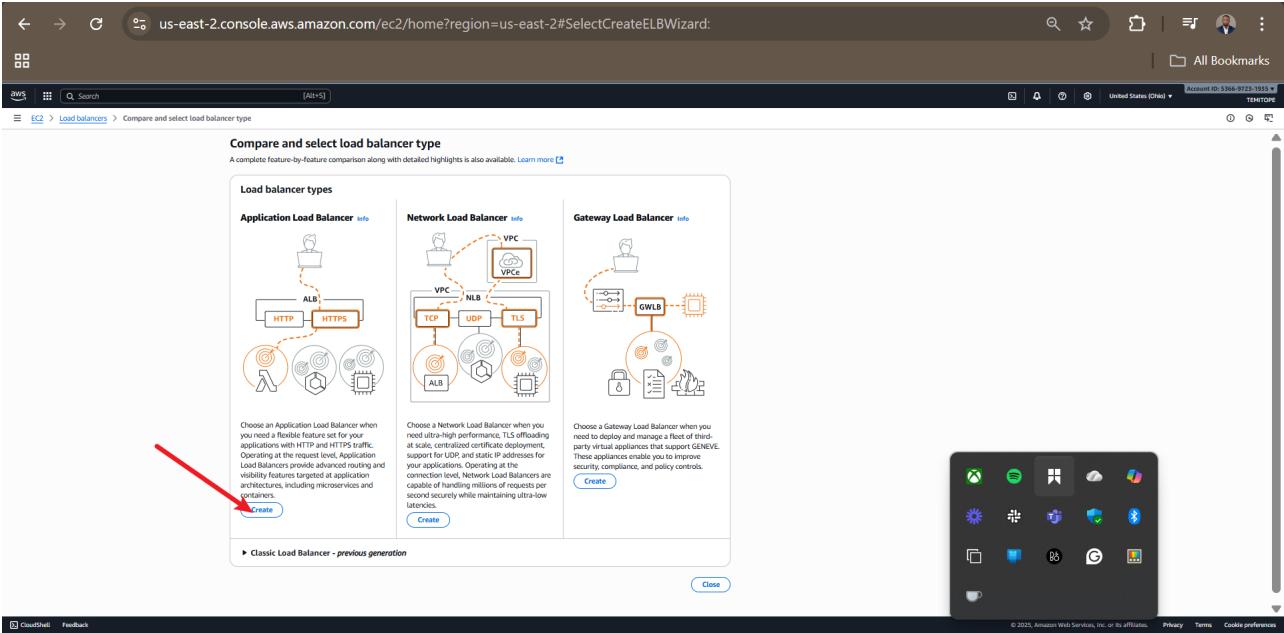
48. On the side menu scroll down to load balancing menu and click on load balancer.



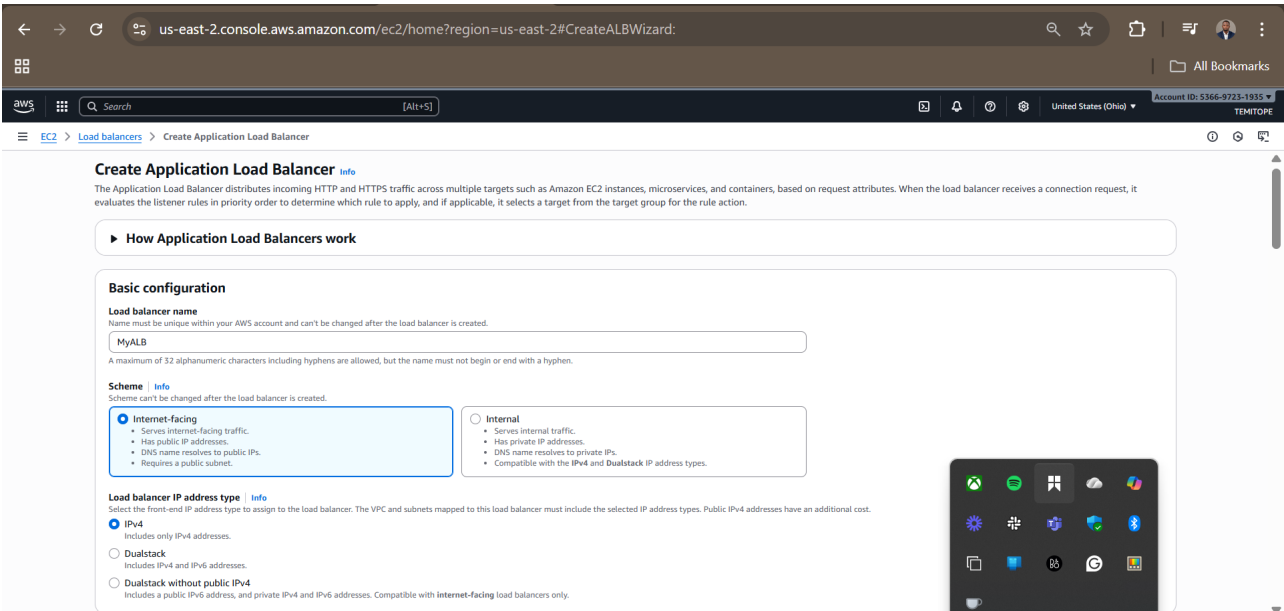
49. On load balancers dashboard click on create load balancer.



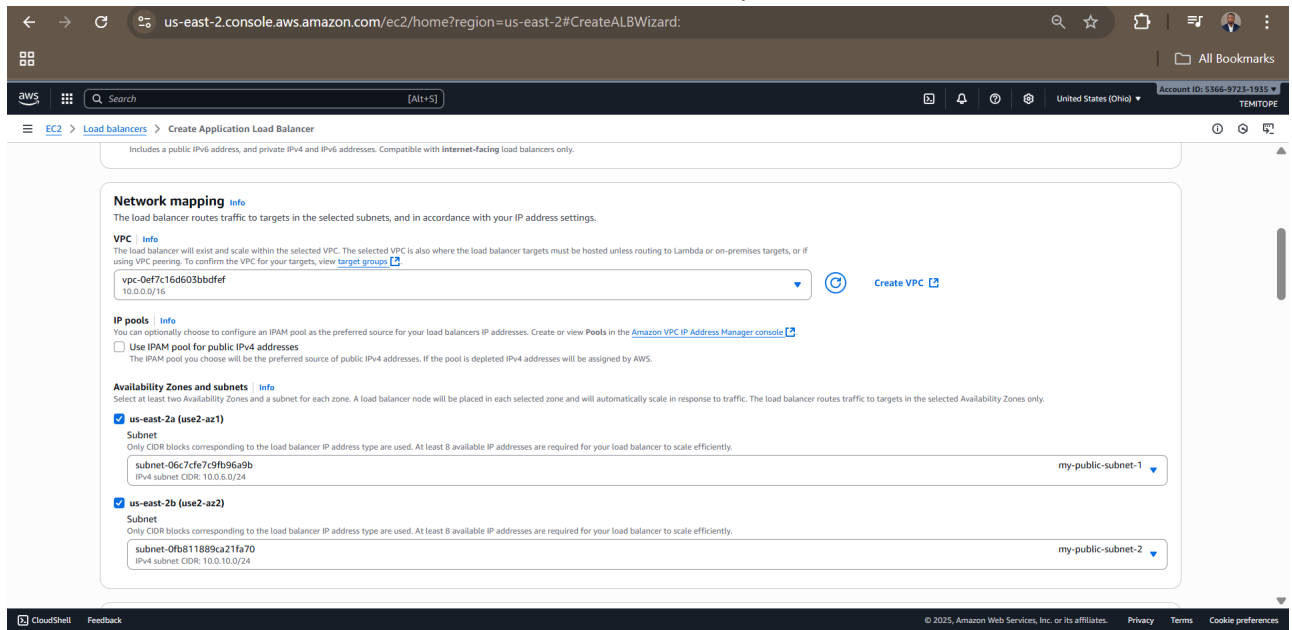
50. Click to create application load balancer.



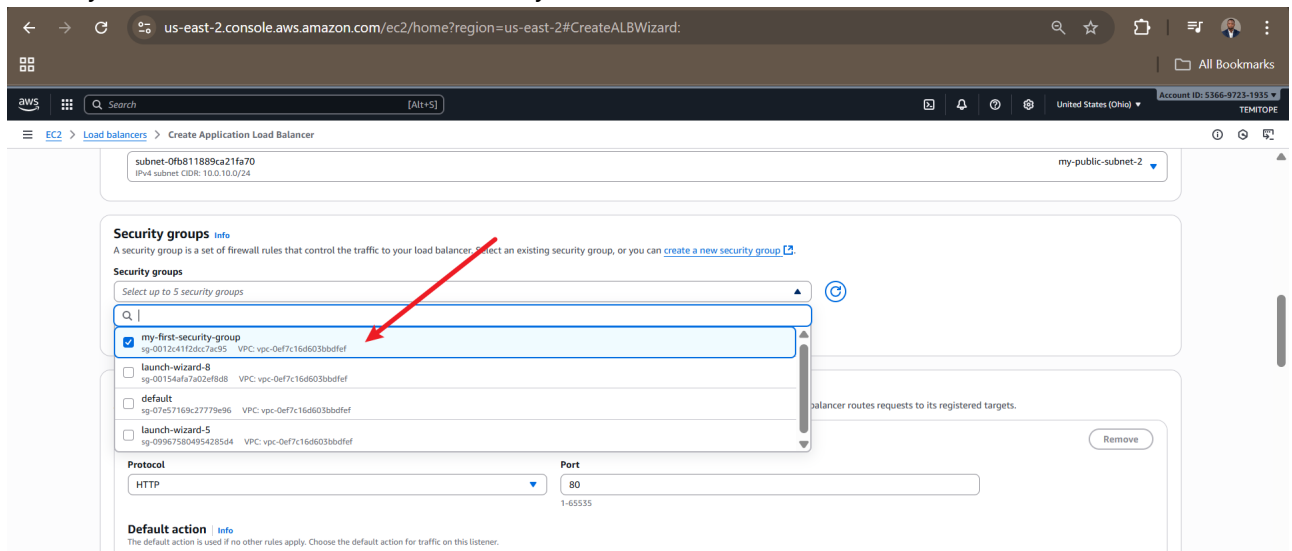
51. Name load balancer select internet facing and IPv4 then scroll down.



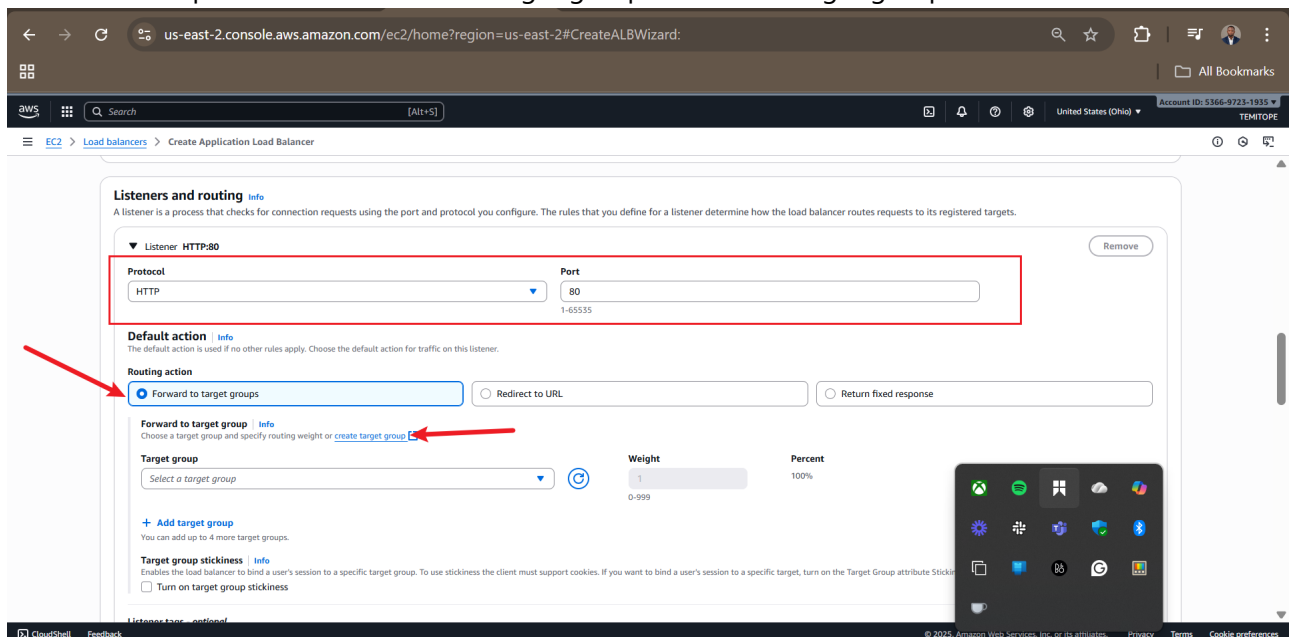
52. Select VPC the servers are check at least 2 AZ and subnets (public subnet) and scroll down.



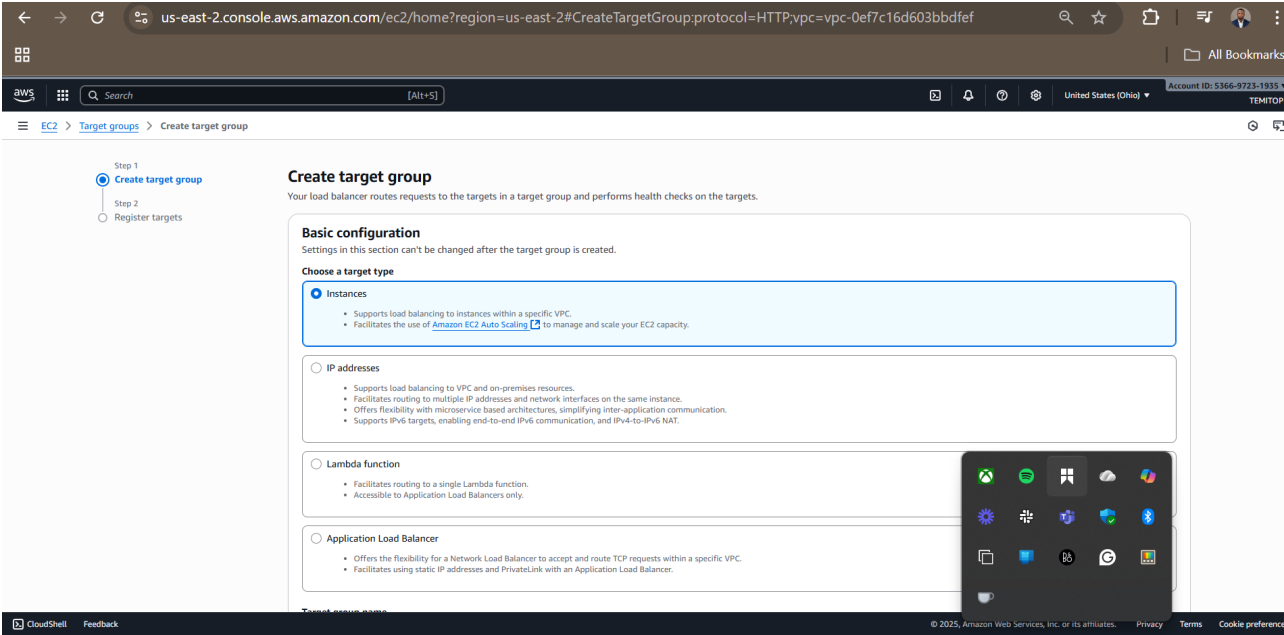
53. Select your SG or create one that allows any HTTP and SSH traffic and scroll down.



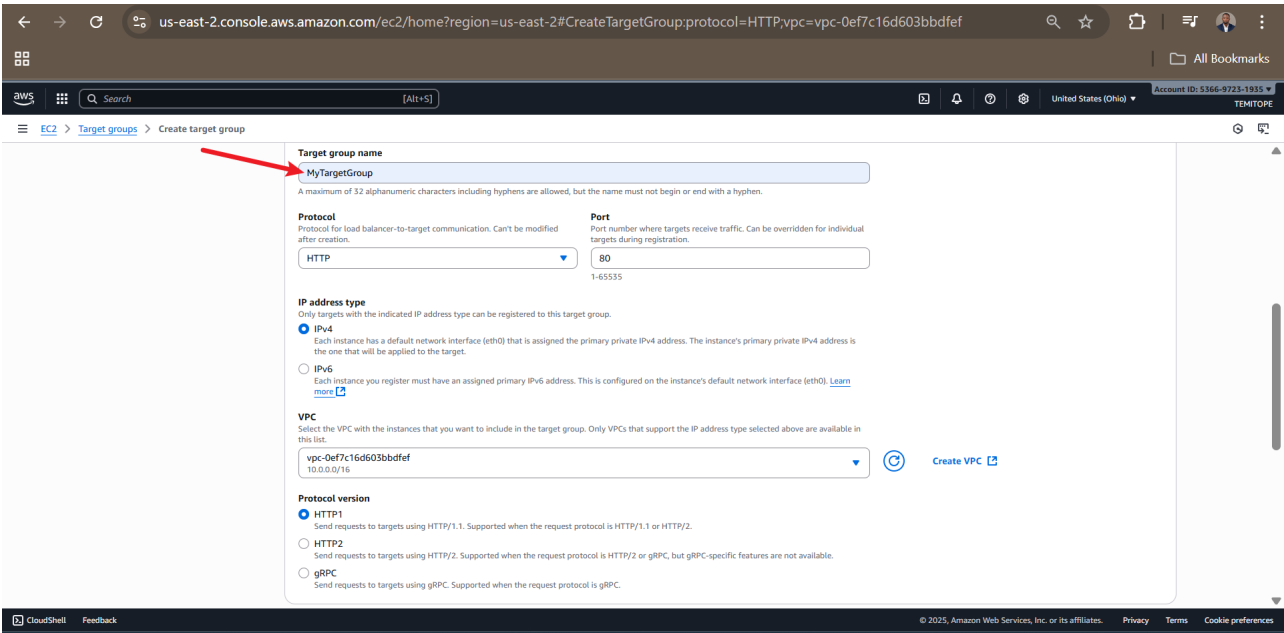
54. Protocol HTTP port 80 click forward to target group and create target group.



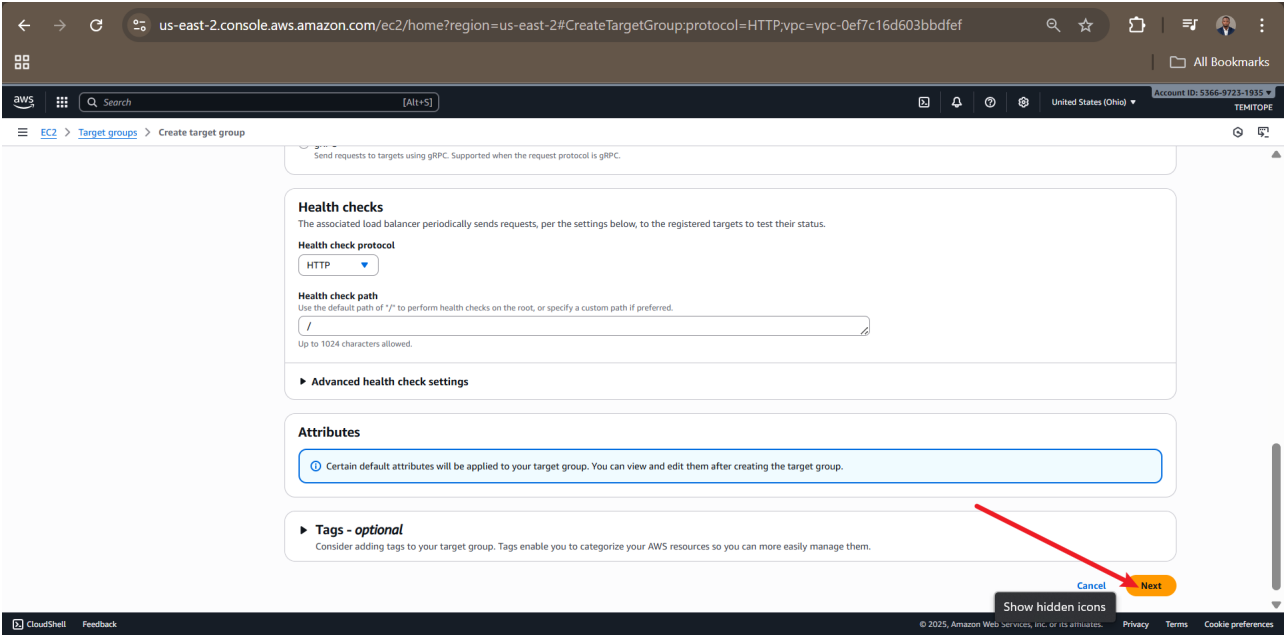
55. Create target group page open select instances and scroll down.



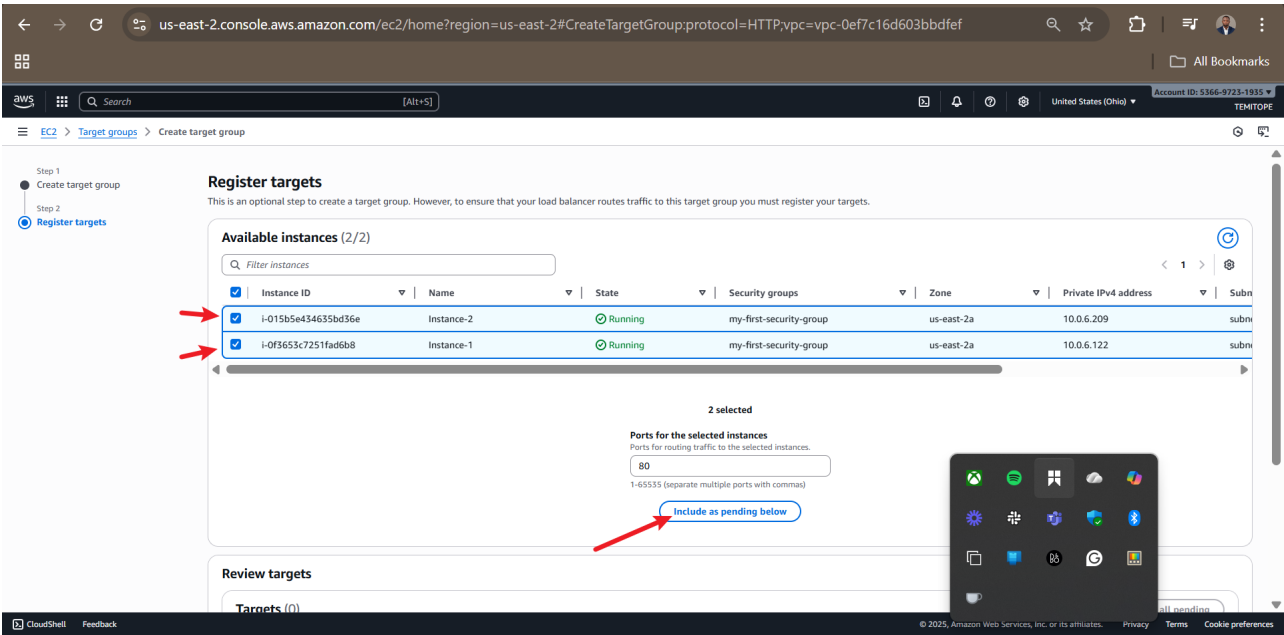
56. Name target group select HTTP port 80 choose the VPC and scroll down.



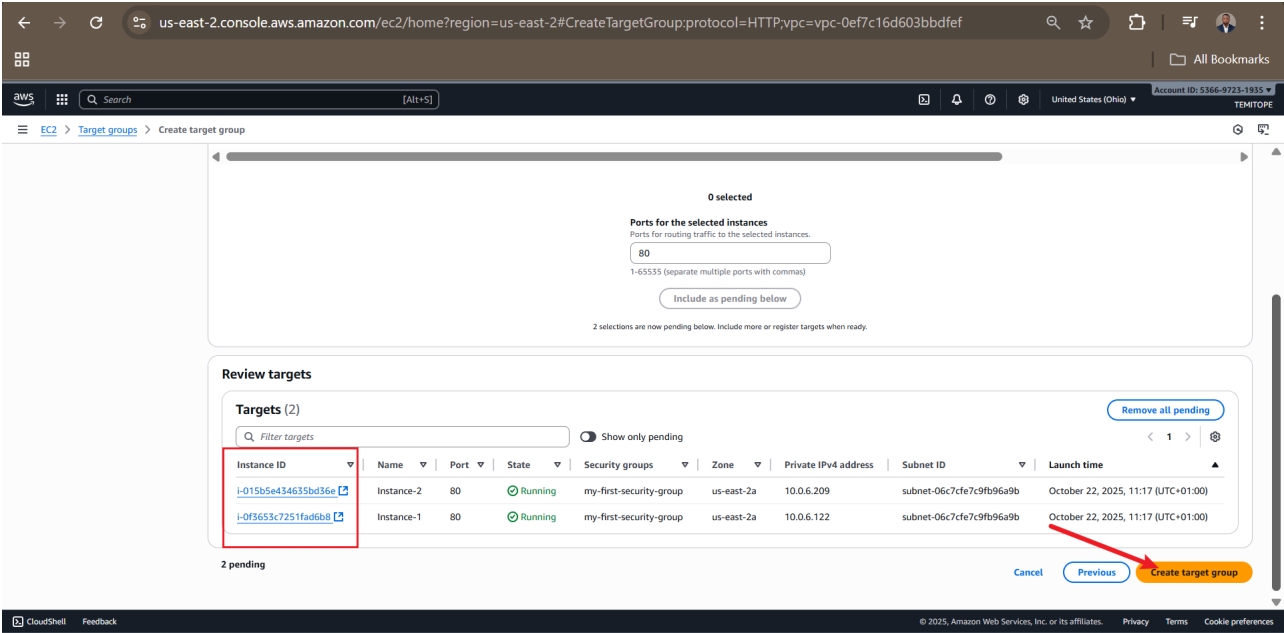
57. Leave everything at default and click next.



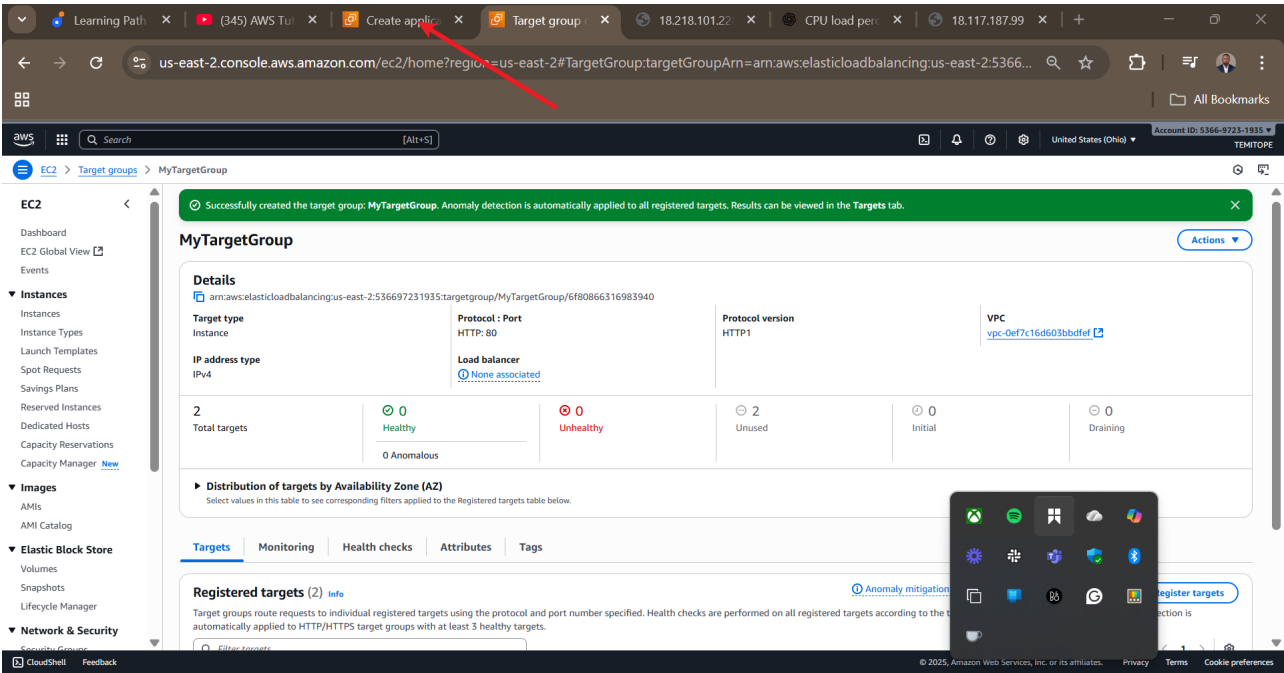
58. Select the servers and click include as pending below.



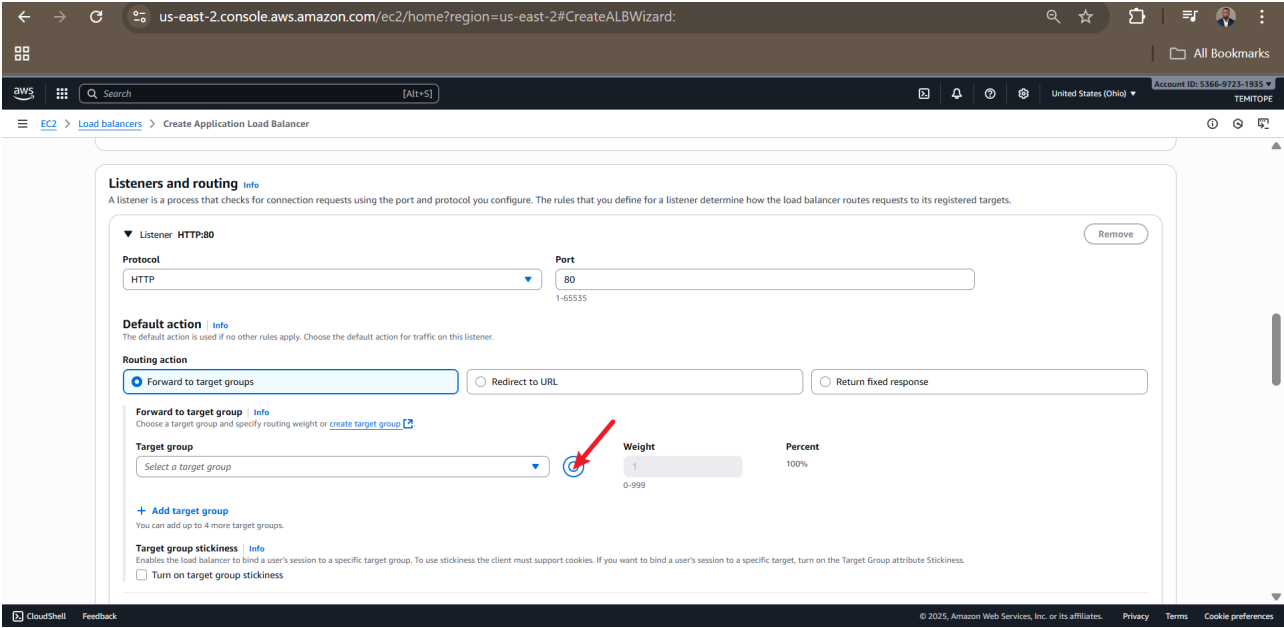
59. Servers are now included click on create target group.



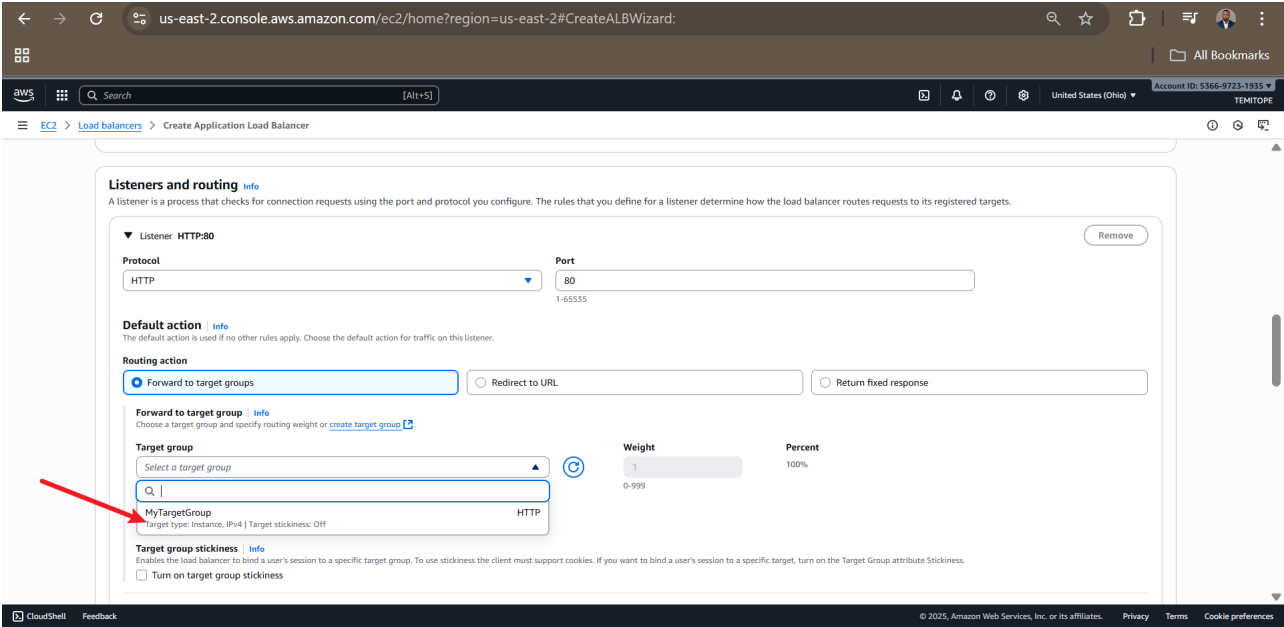
60. Target group successfully created now go back to LB tab to continue.



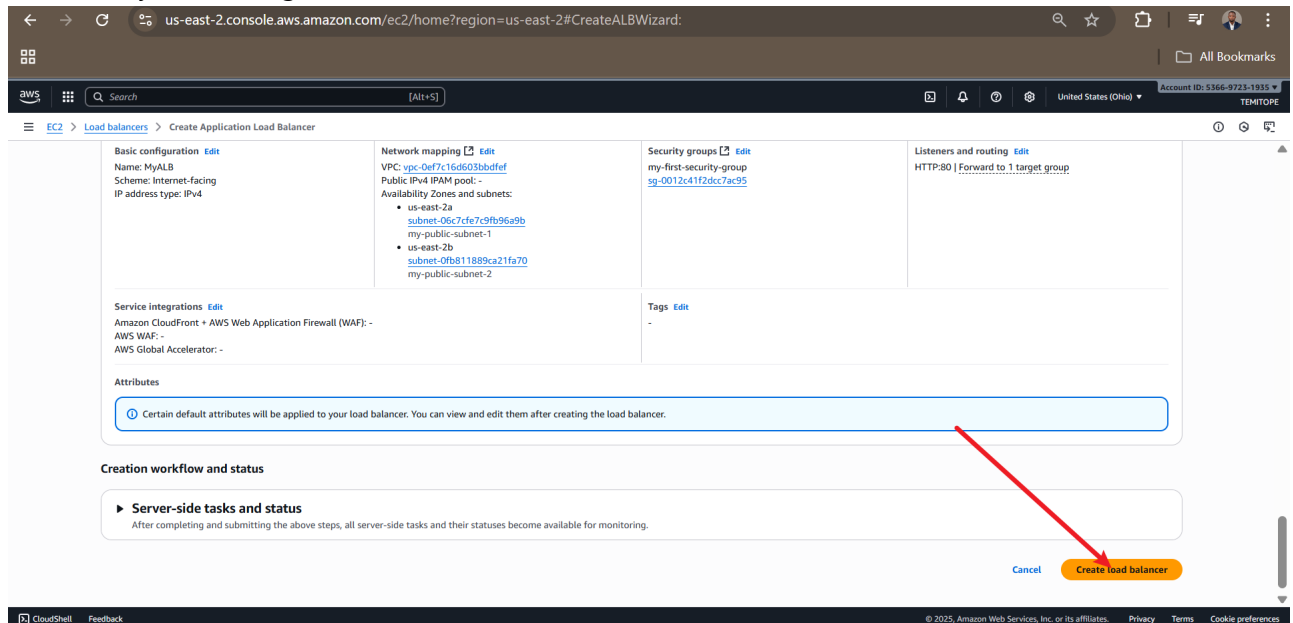
61. Click the refresh button to load the created TG.



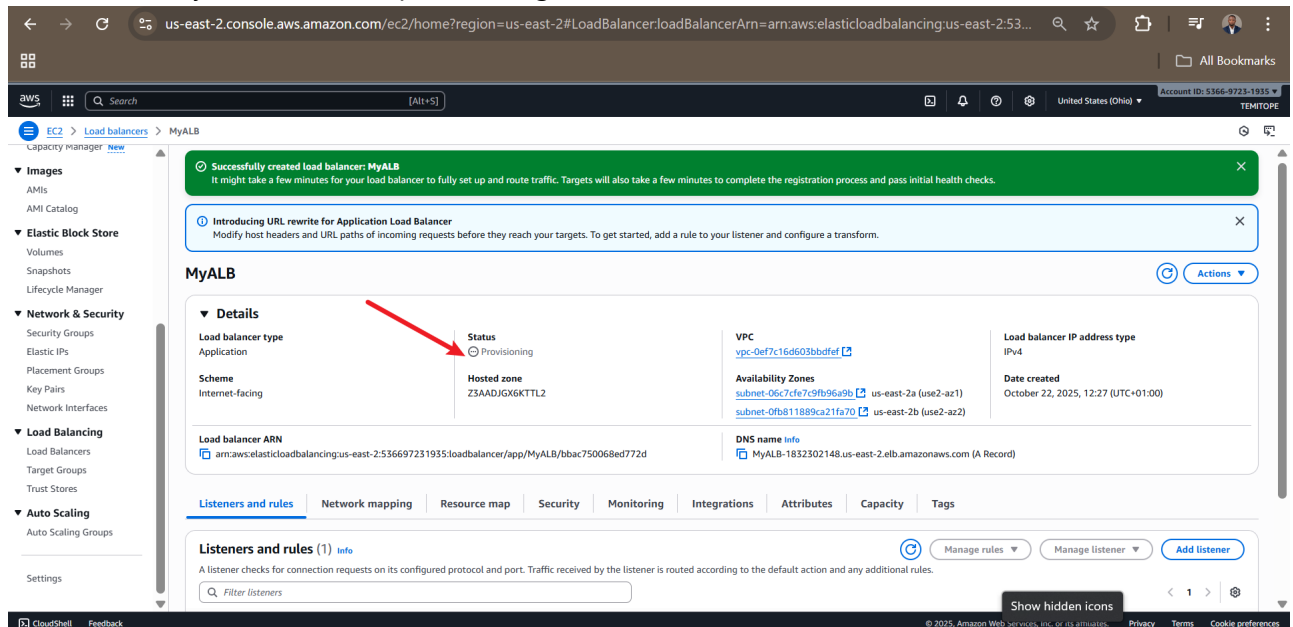
62. Select the newly created target group scroll down.



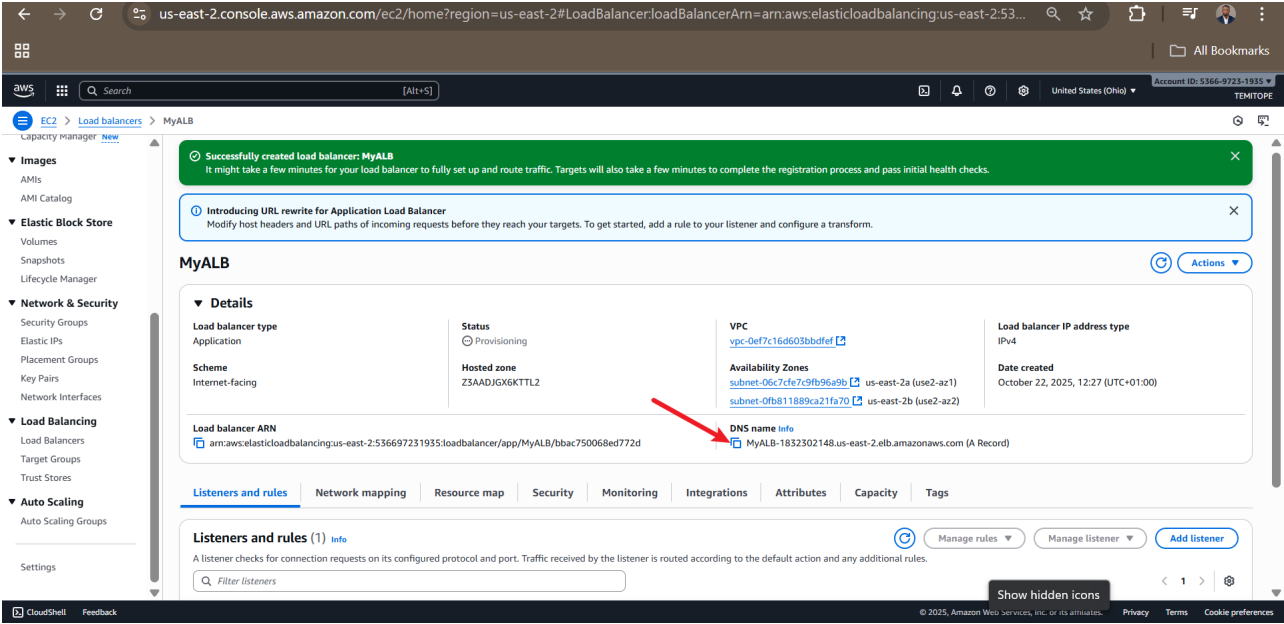
63. Leave every other thing at default and click create load balancer.



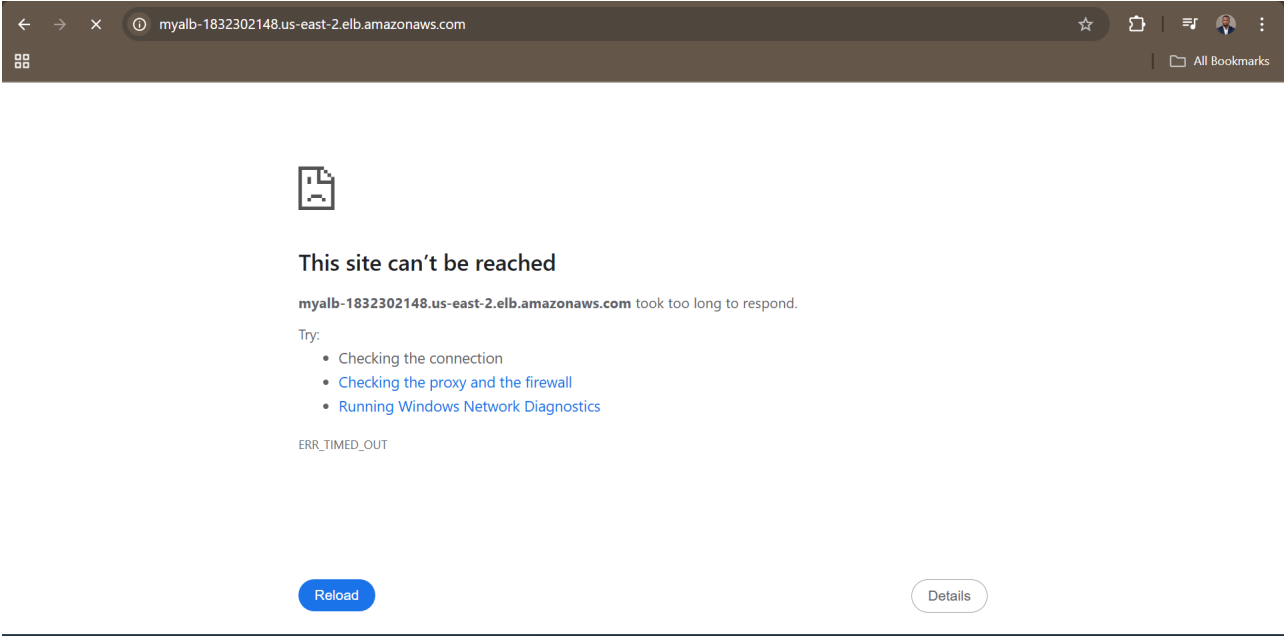
64. ALB successfully created and in provisioning state.



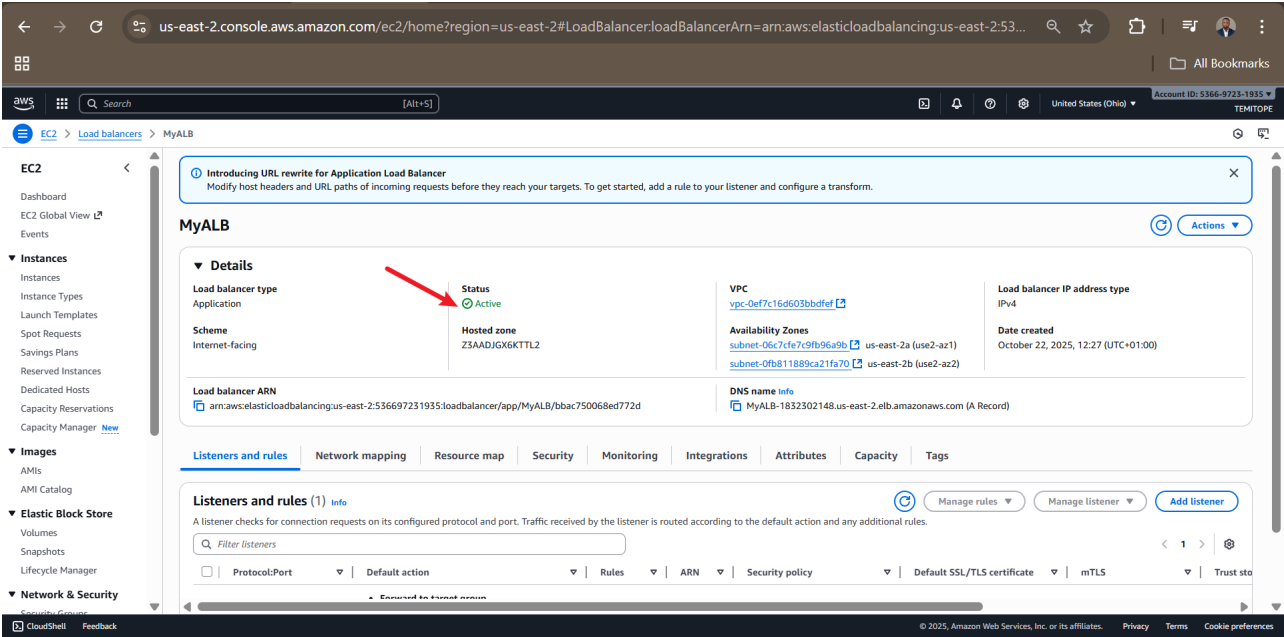
65. Copy the DNS name to paste it on a browser.



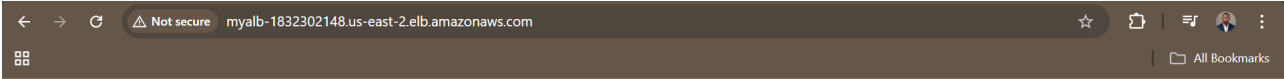
66. Site can not be reached because ALB is still in provisioning state.



67. Refresh after a while and ALB now active.



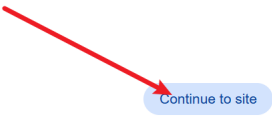
68. DNS now work click continue to site.



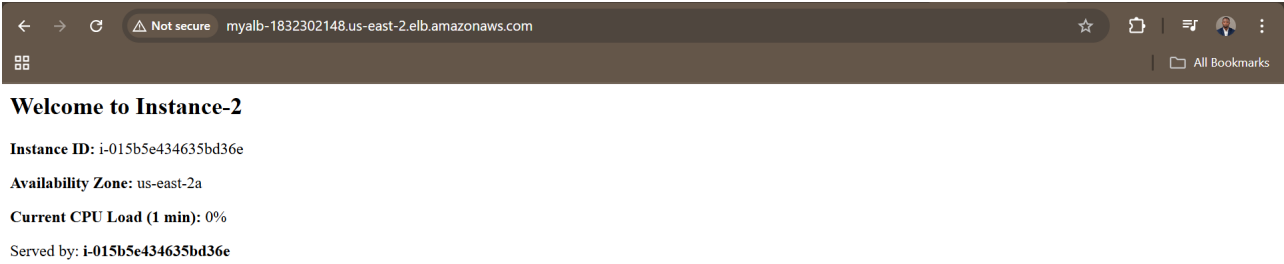
myalb-1832302148.us-east-2.elb.amazonaws.com doesn't support a secure connection with HTTPS

- **Attackers can see and change** information you send or receive from the site.
- **It's safest to visit this site later** if you're using a public network. There is less risk from a trusted network, like your home or work Wi-Fi.

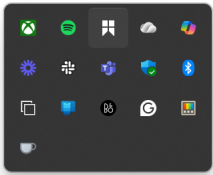
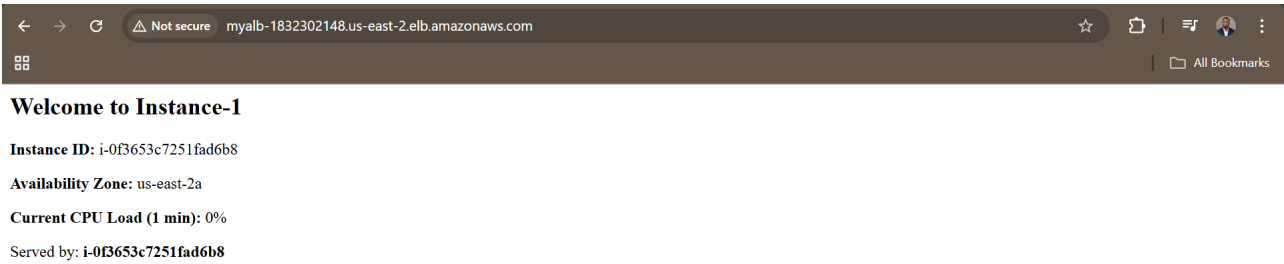
You might also contact the site owner and suggest they upgrade to HTTPS. [Learn more about this warning](#)



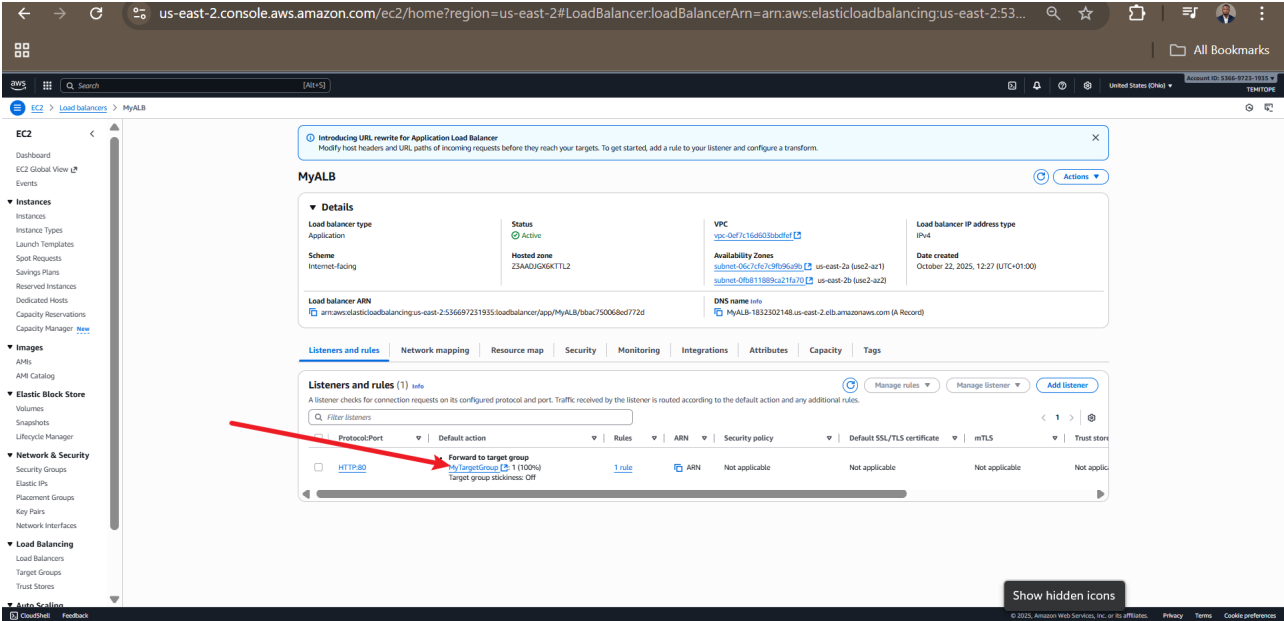
69. Instance-2 webpage loaded using the ALB DNS link.



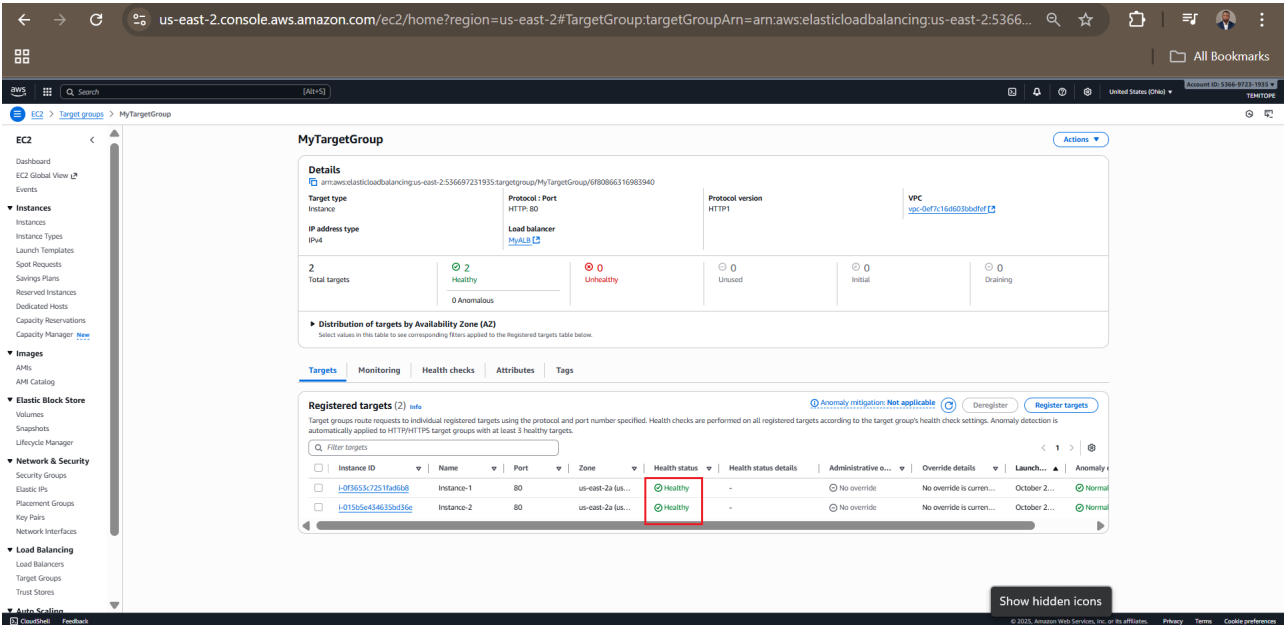
70. After reload instance-1 webpage loaded using the ALB DNS link (round robin).



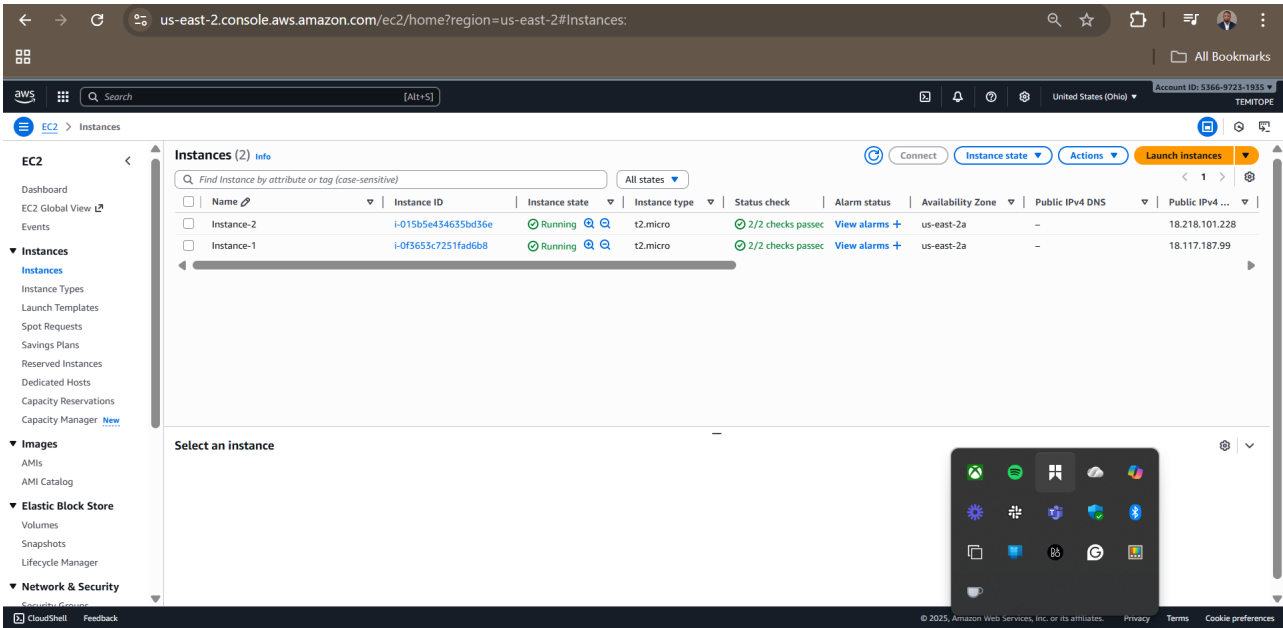
71. In case of problem click target group to inspect the target health.



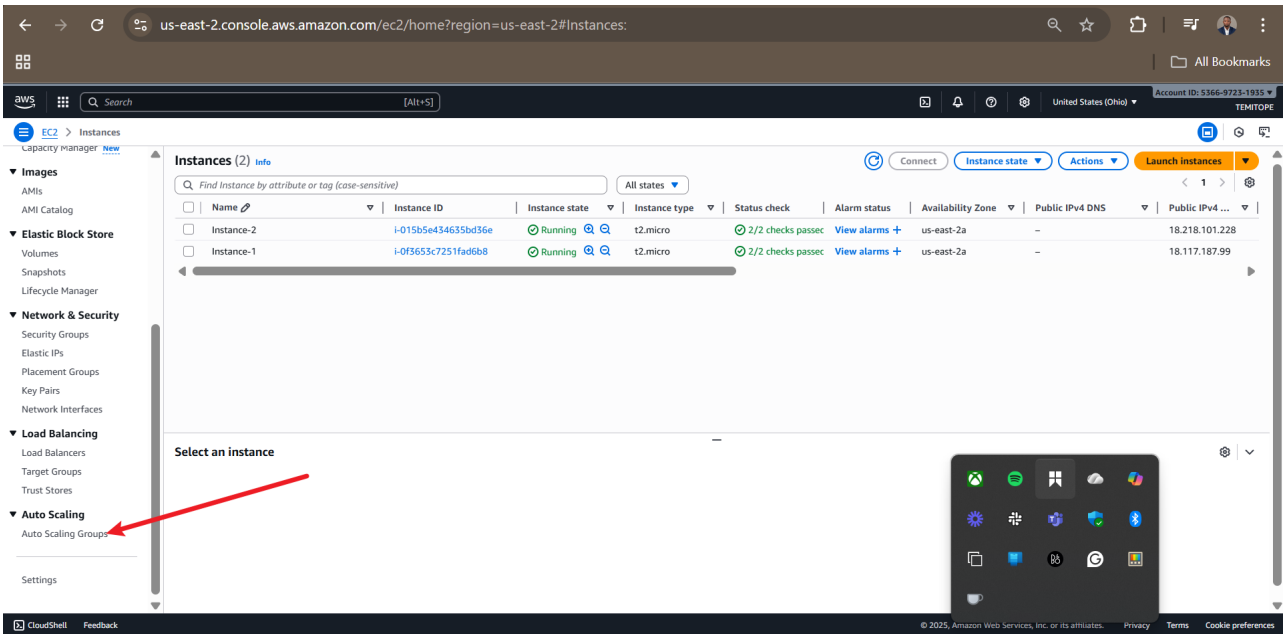
72. Both registered target are healthy for now.



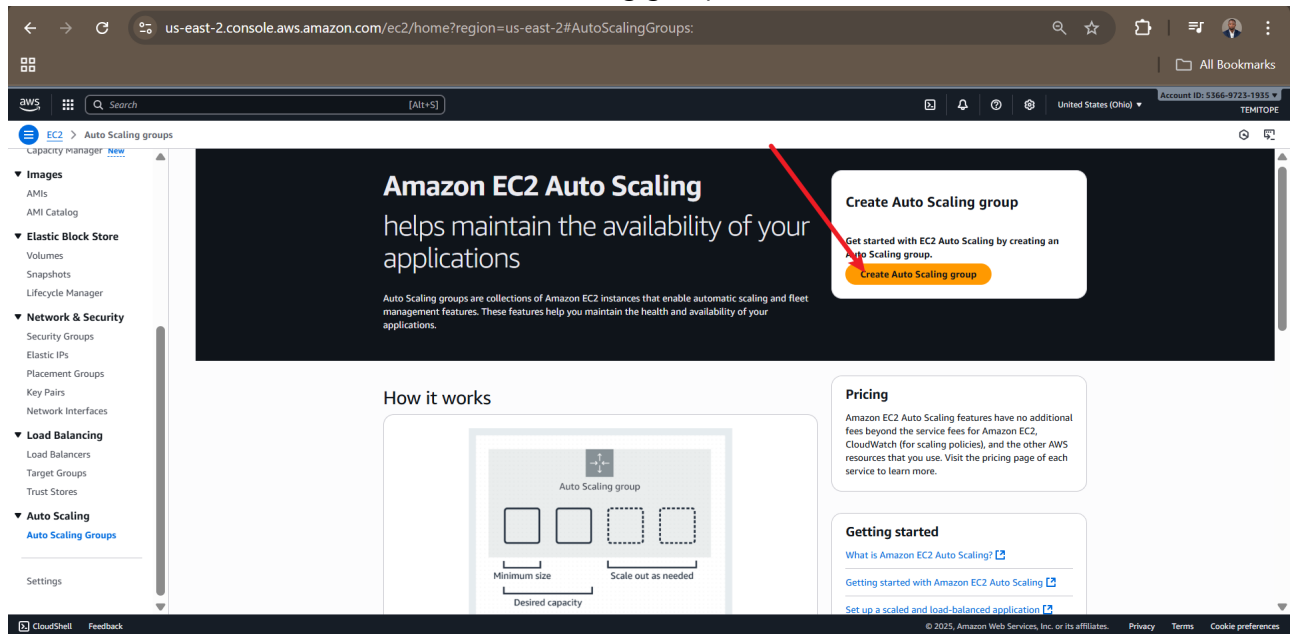
73. To auto scale the servers navigate to instances dashboard.



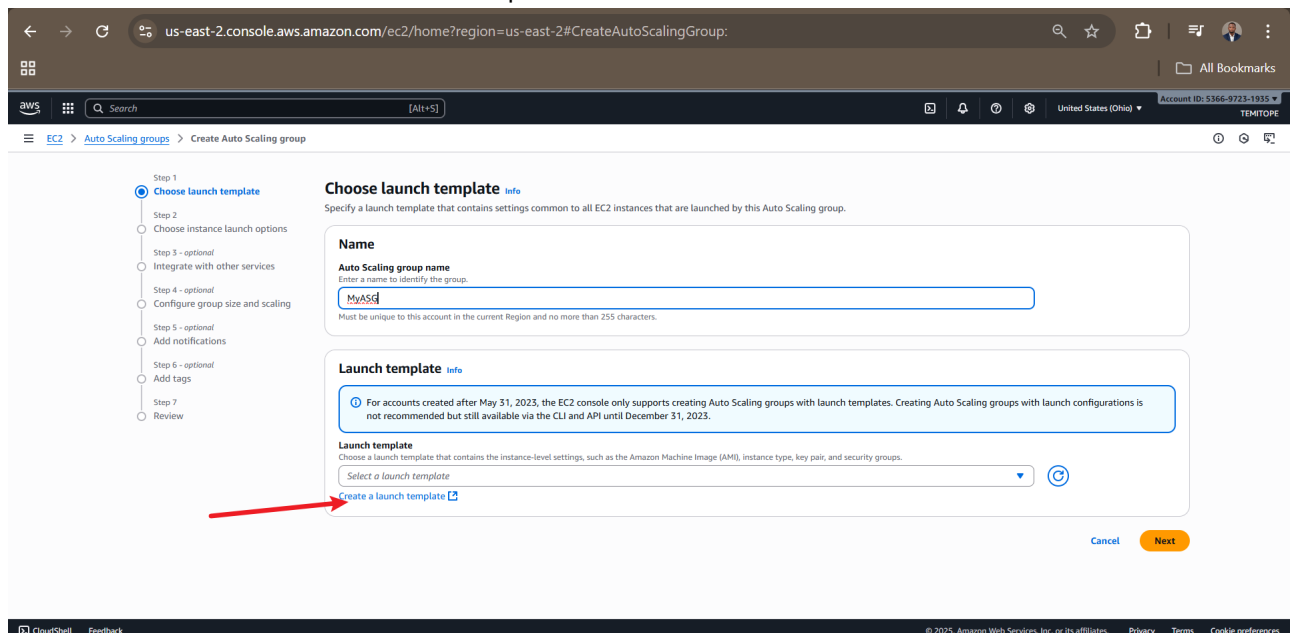
74. On the side menu scroll down to auto scaling menu and click on auto scaling groups.



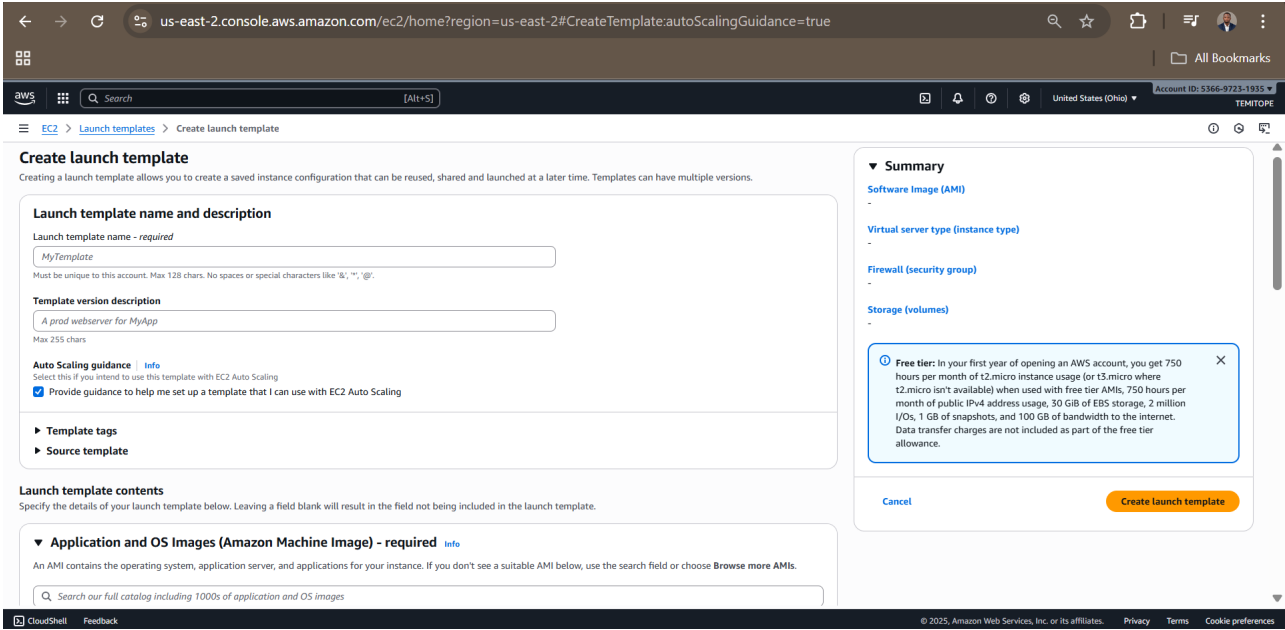
75. On EC2 ASG dashboard click on create auto scaling group.



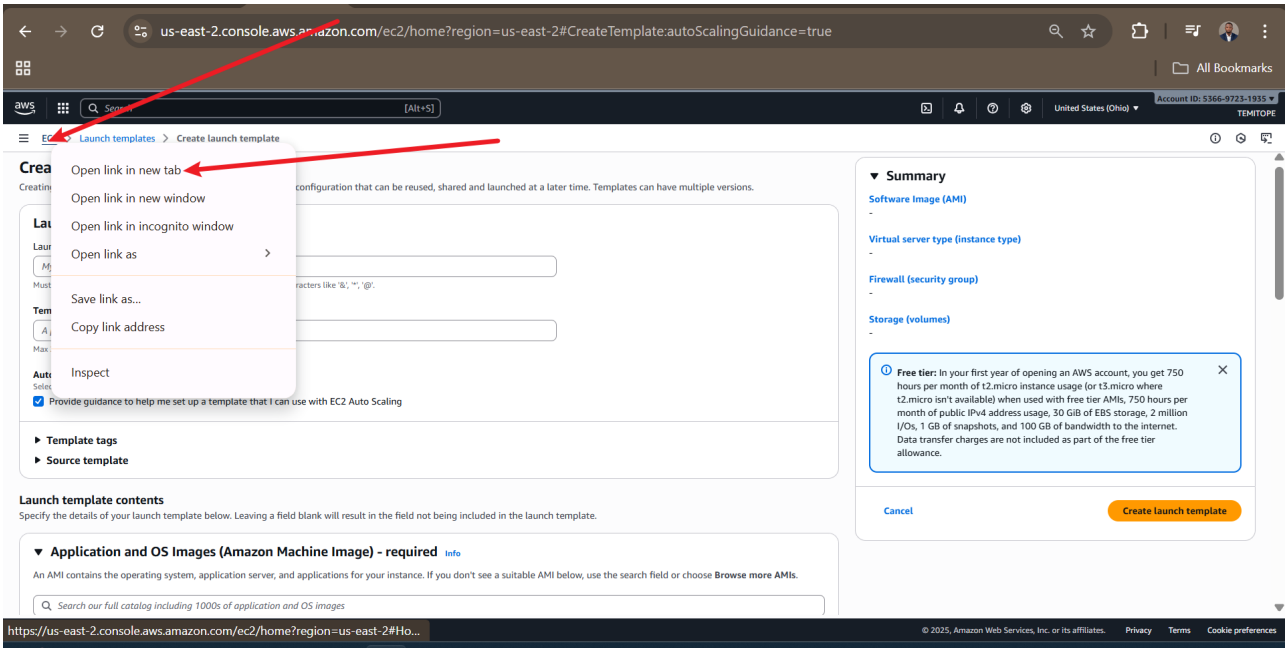
76. Name ASG and click create a launch template.



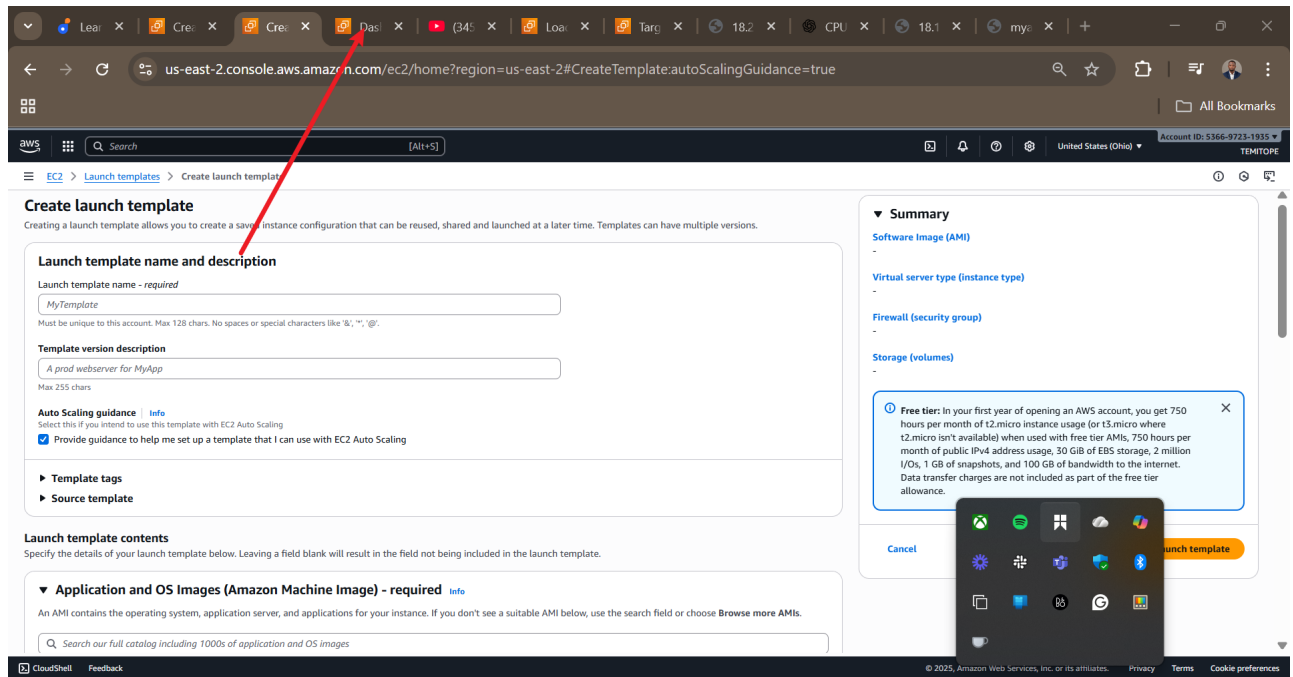
77. Create launch template page open let us create AMI image of the server.



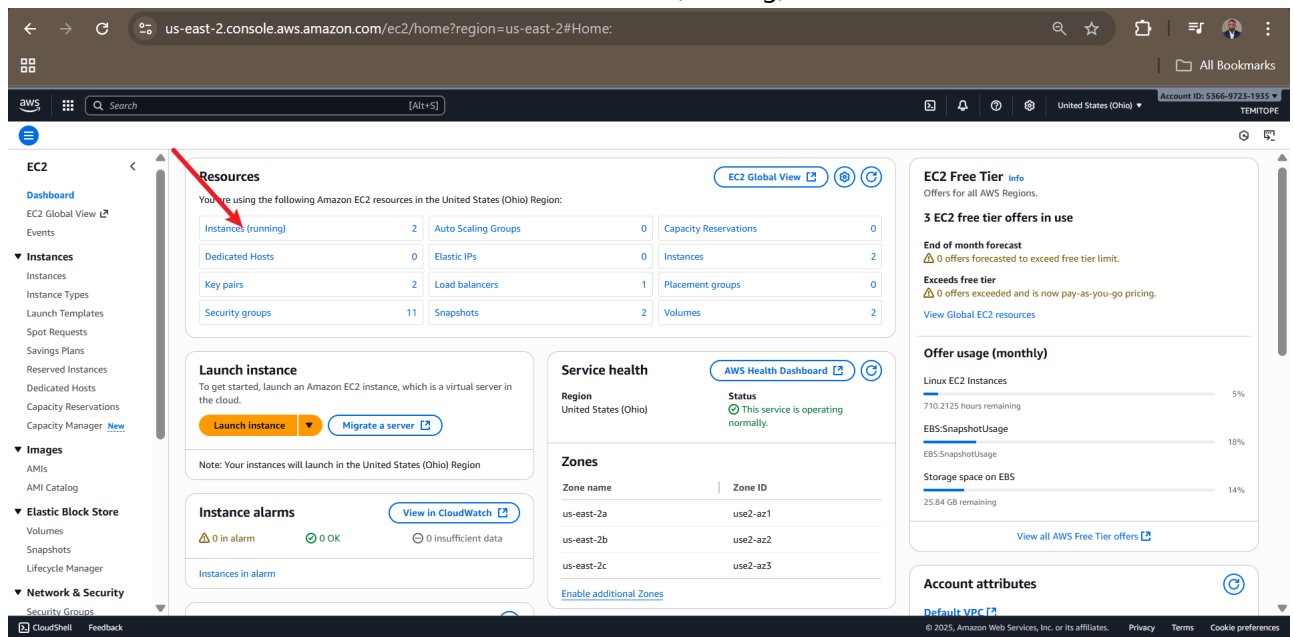
78. Right-click on EC2 from navigation menu and open link in a new tab.



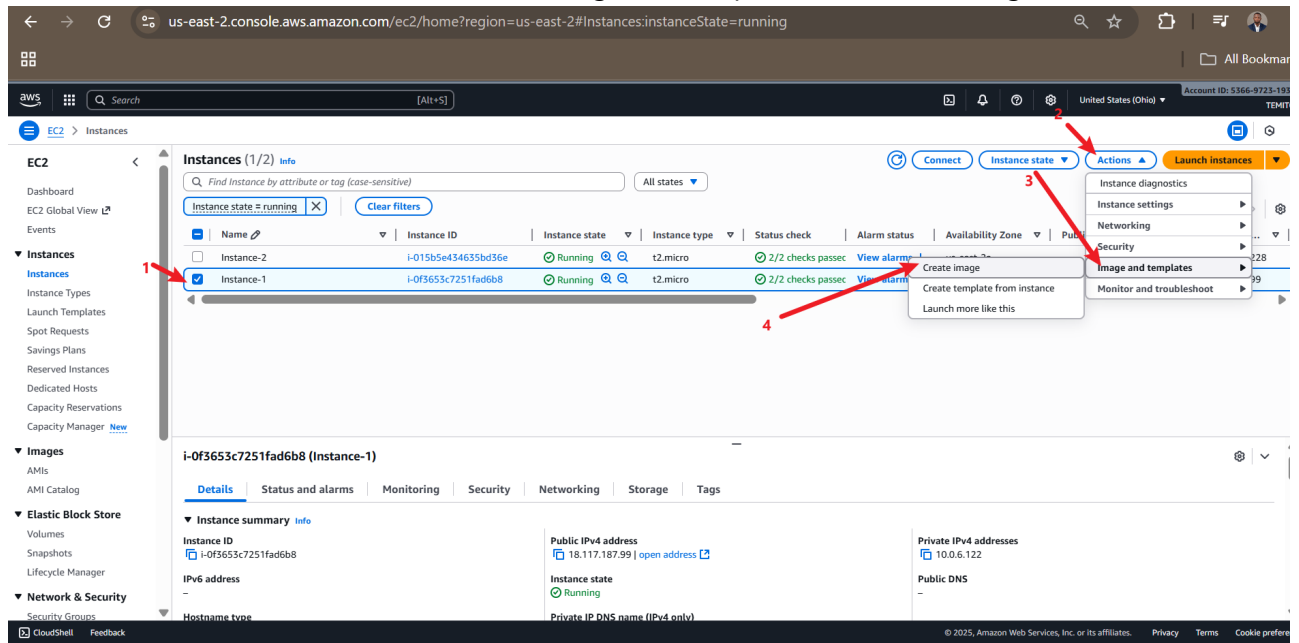
79. Click on the new tab to switch to it.



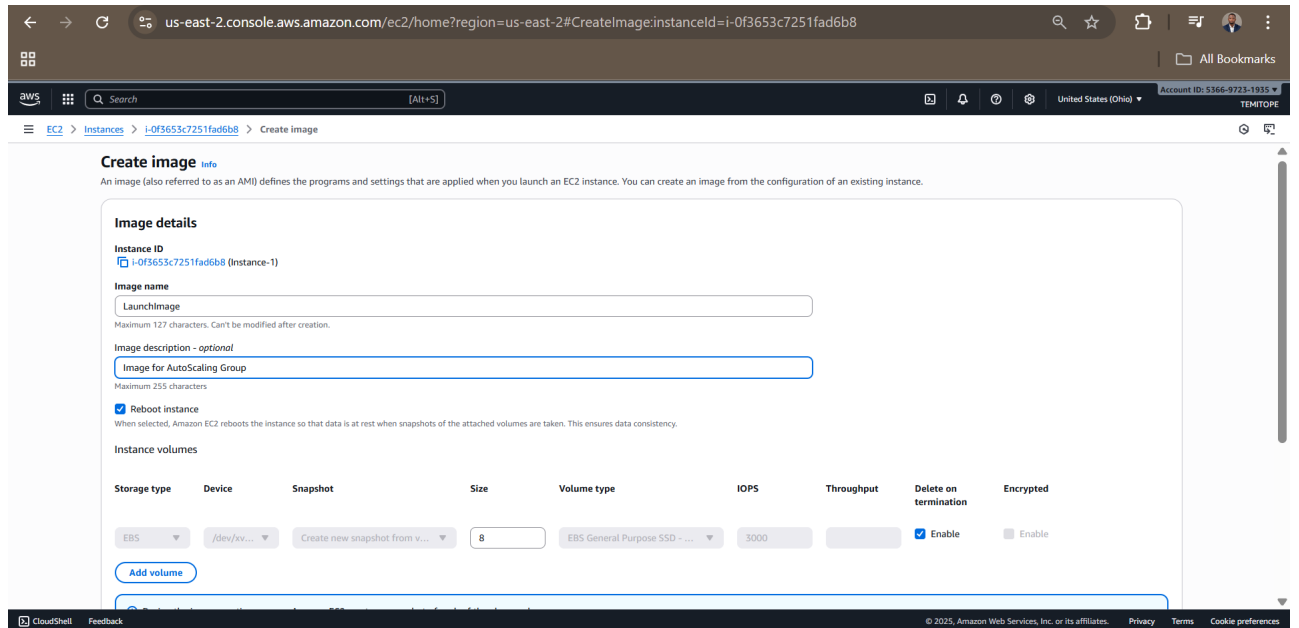
80. On EC2 dashboard under resources click on instances (running).



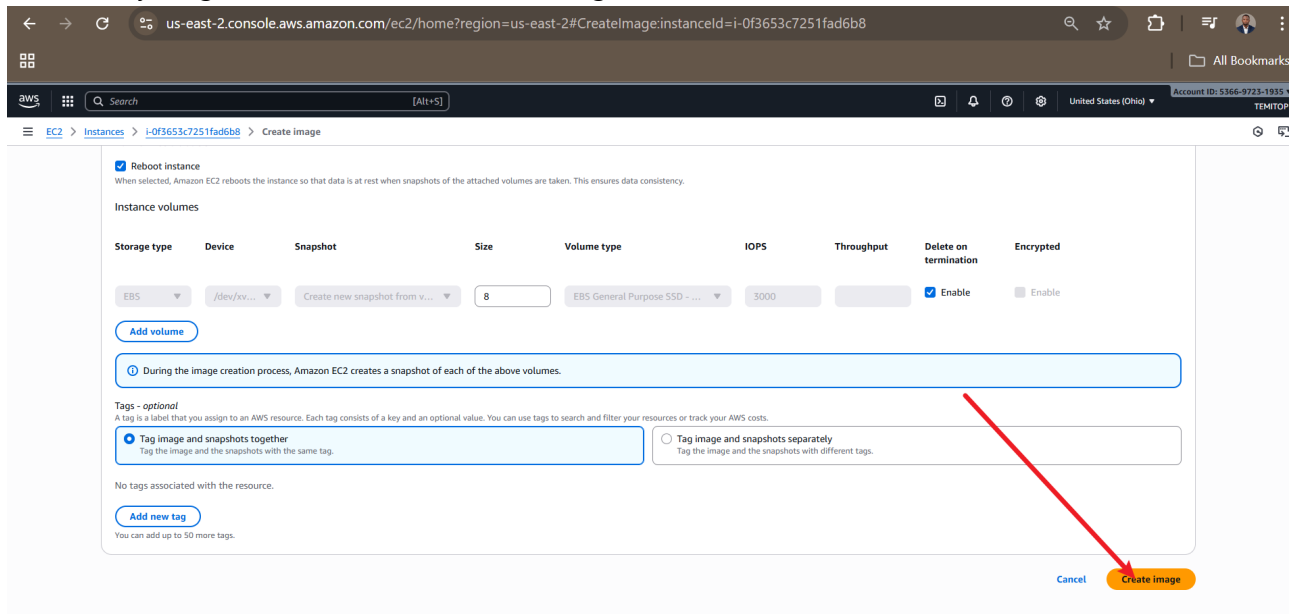
81. Select instance-1 then click on actions then image and templates then create image.



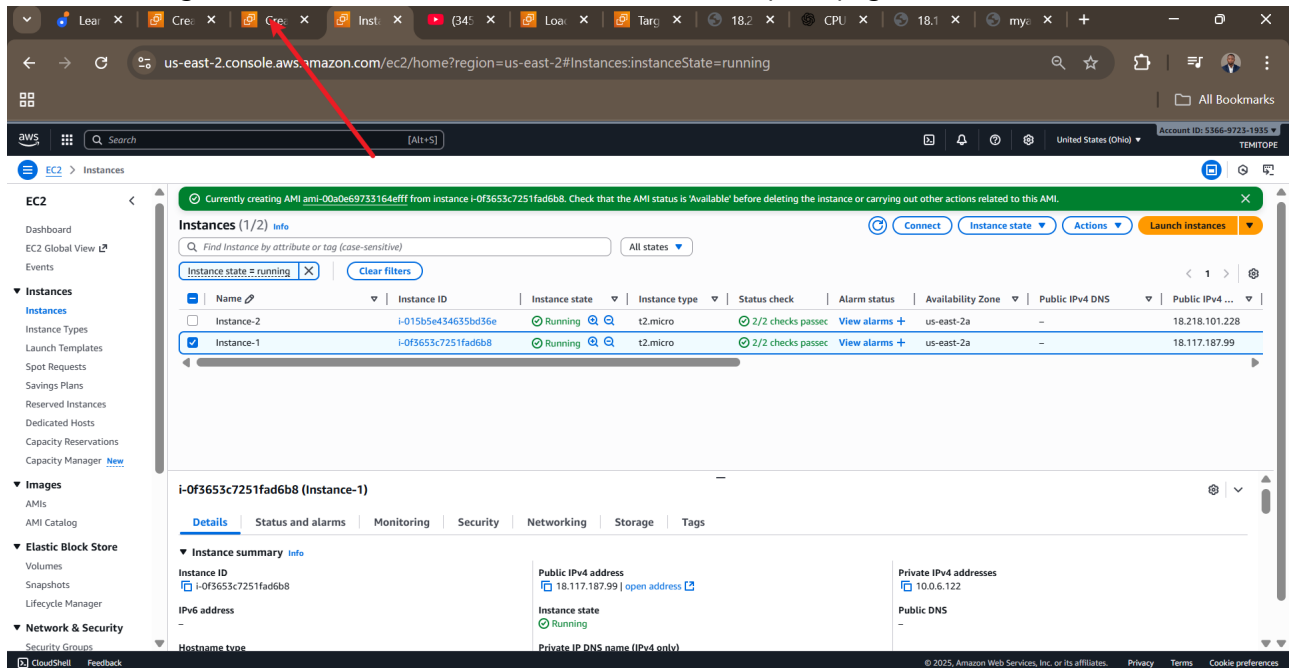
82. Name image add description and scroll down.



83. Leave everything at default and click create image.



84. AMI now being created now switch back to create launch template page to continue.



us-east-2.console.aws.amazon.com/ec2/home?region=us-east-2#CreateTemplate:autoScalingGuidance=true

All Bookmarks

Search [Alt+S]

United States (Ohio) Account ID: 5366-9723-1935 TEMITOPE

EC2 > Launch templates > Create launch template

Create launch template

Creating a launch template allows you to create a saved instance configuration that can be reused, shared and launched at a later time. Templates can have multiple versions.

Launch template name and description

Launch template name - required

MyTemplate

Must be unique to this account. Max 128 chars. No spaces or special characters like '&', '*', '/', '\', '~'.

Template version description

Template for AutoScaling Group

Max 255 chars

Auto Scaling guidance info

Select this if you intend to use this template with EC2 Auto Scaling

☒ Provide guidance to help me set up a template that I can use with EC2 Auto Scaling

► Template tags

► Source template

Launch template contents

Specify the details of your launch template below. Leaving a field blank will result in the field not being included in the launch template.

▼ Application and OS Images (Amazon Machine Image) - required info

An AMI contains the operating system, application server, and applications for your instance. If you don't see a suitable AMI below, use the search field or choose Browse more AMIs.

Search our full catalog including 1000s of application and OS images

Summary

Software Image (AMI)

-

Virtual server type (instance type)

-

Firewall (security group)

-

Storage (volumes)

-

Free tier: In your first year of opening an AWS account, you get 750 hours per month of t2.micro instance usage (or t3.micro where t2.micro isn't available) when used with free tier AMIs, 750 hours per month of public IPv4 address usage, 30 GiB of EBS storage, 2 million I/Os, 1 GB of snapshots, and 100 GB of bandwidth to the internet. Data transfer charges are not included as part of the free tier allowance.

Cancel

Create launch template

The screenshot shows the AWS Management Console interface. At the top, the browser address bar displays the URL: `us-east-2.console.aws.amazon.com/ec2/home?region=us-east-2#CreateTemplate:autoScalingGuide=true`. The console header includes the AWS logo, a search bar, and account information for 'United States (Ohio)' with Account ID: 5366-9723-1935.

The main content area is titled 'Launch template contents' and includes a sub-header: 'Specify the details of your launch template below. Leaving a field blank will result in the field not being included in the launch template.'

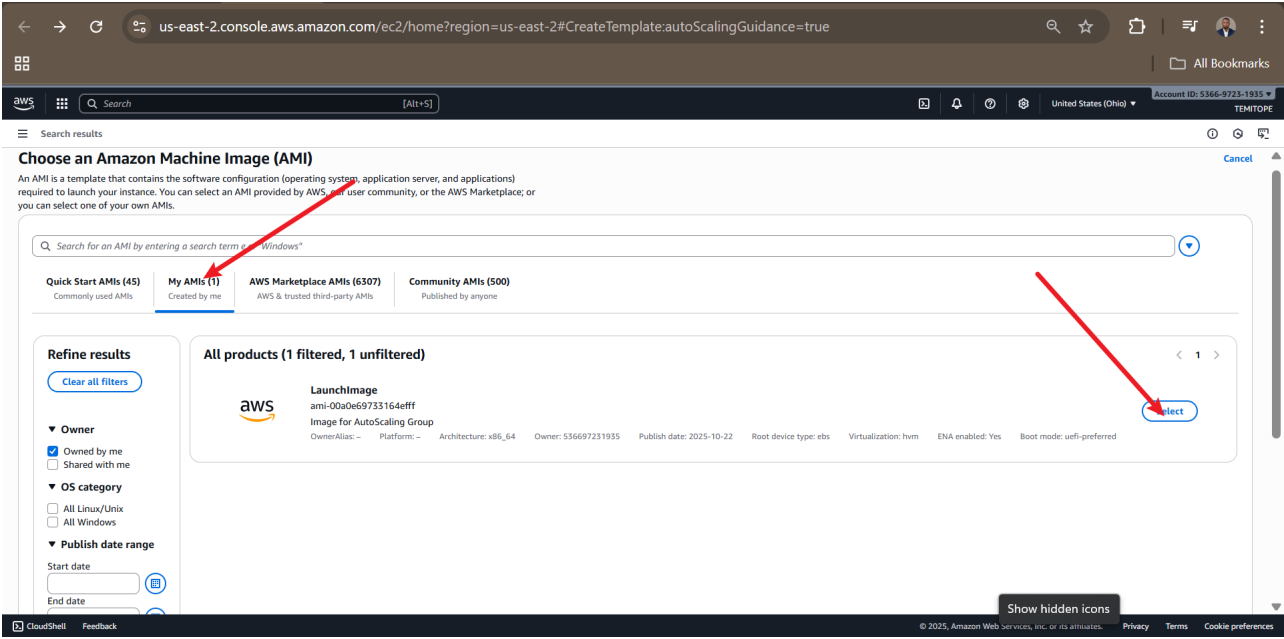
The primary section is 'Application and OS Images (Amazon Machine Image) - required', which includes an 'Info' link. Below this, a text block states: 'An AMI contains the operating system, application server, and applications for your instance. If you don't see a suitable AMI below, use the search field or choose [Browse more AMIs](#).'

A search bar is provided with the placeholder text: 'Search our full catalog including 1000s of application and OS images'. Below the search bar are two tabs: 'Recents' and 'Quick Start'. Under 'Quick Start', there are two filterable lists: 'Recently launched' and 'Currently in use'. A red arrow points to the 'Browse more AMIs' link, which is located to the right of the 'Currently in use' list. The link text is 'Browse more AMIs' and includes a magnifying glass icon. Below the link, it says 'Including AMIs from AWS, Marketplace and the Community'.

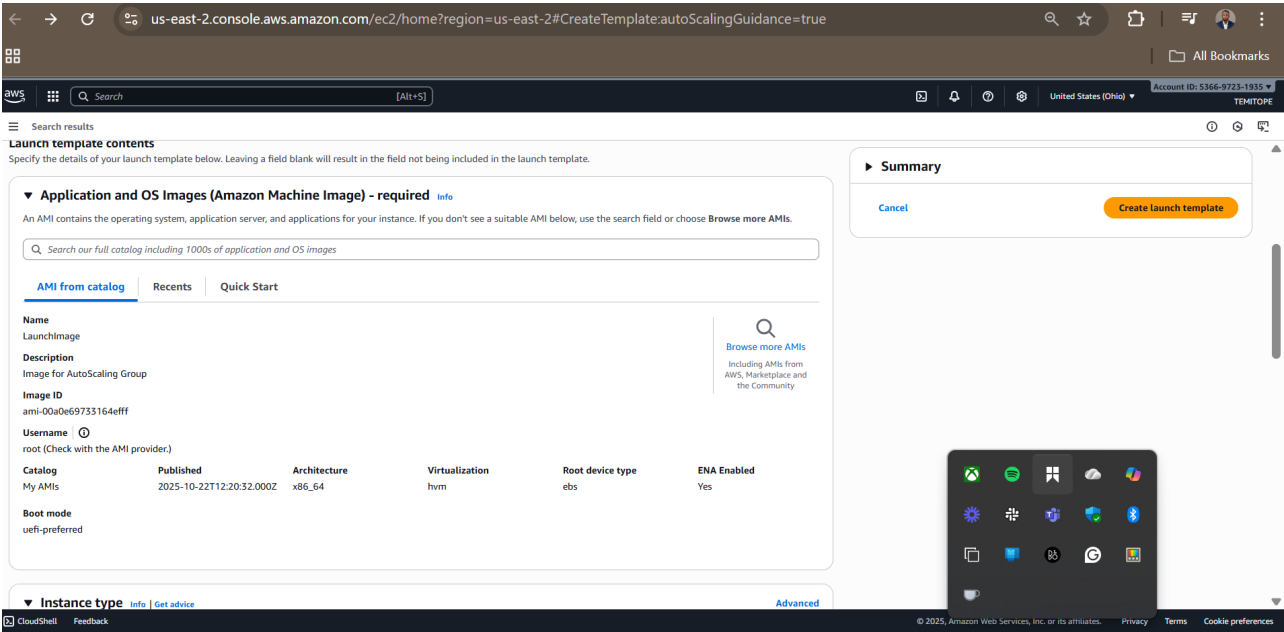
The 'Amazon Machine Image (AMI)' section displays a table of AMIs. The first AMI is 'al2023-ami-2023.9.20251014.0-kernel-6.1-x86_64' with ID 'ami-0199d4b5b8b4fde0e'. It is described as 'Amazon Linux 2023 AMI 2023.9.20251014.0 x86_64 HVM kernel-6.1'. The architecture is 'x86_64' and it is marked as a 'Verified provider'.

The bottom of the console shows a footer with 'CloudShell', 'Feedback', and 'Show hidden icons' links, along with copyright information for Amazon Web Services, Inc. or its affiliates.

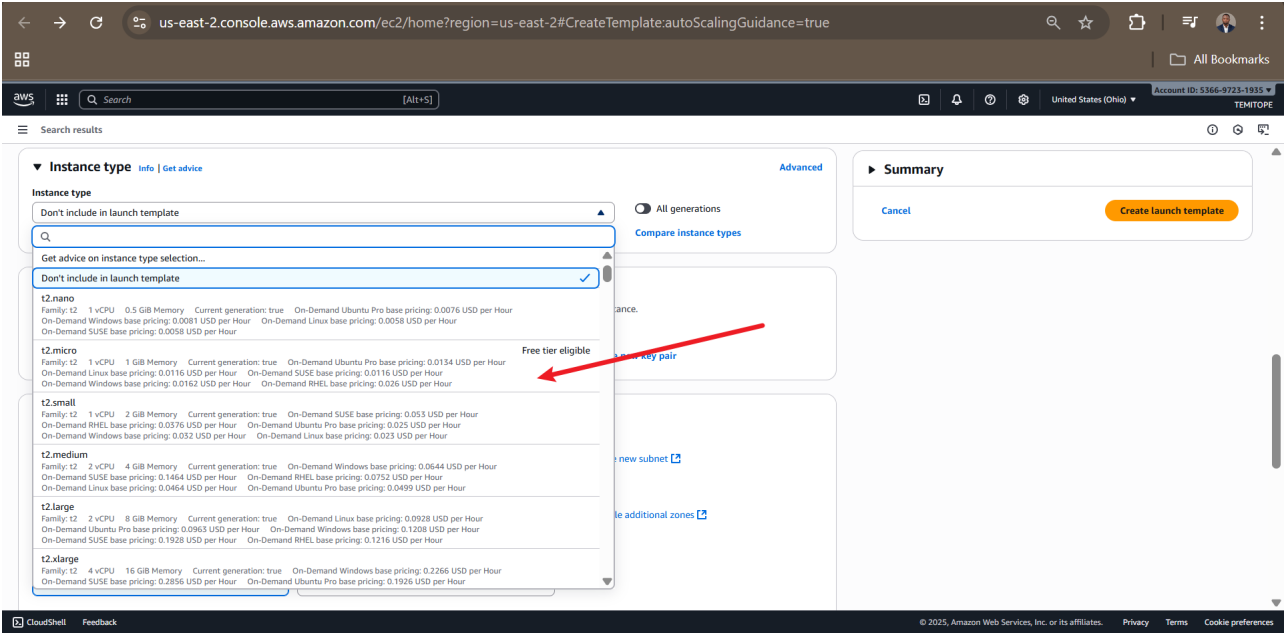
87. Click My AMI and select the just created AMI.



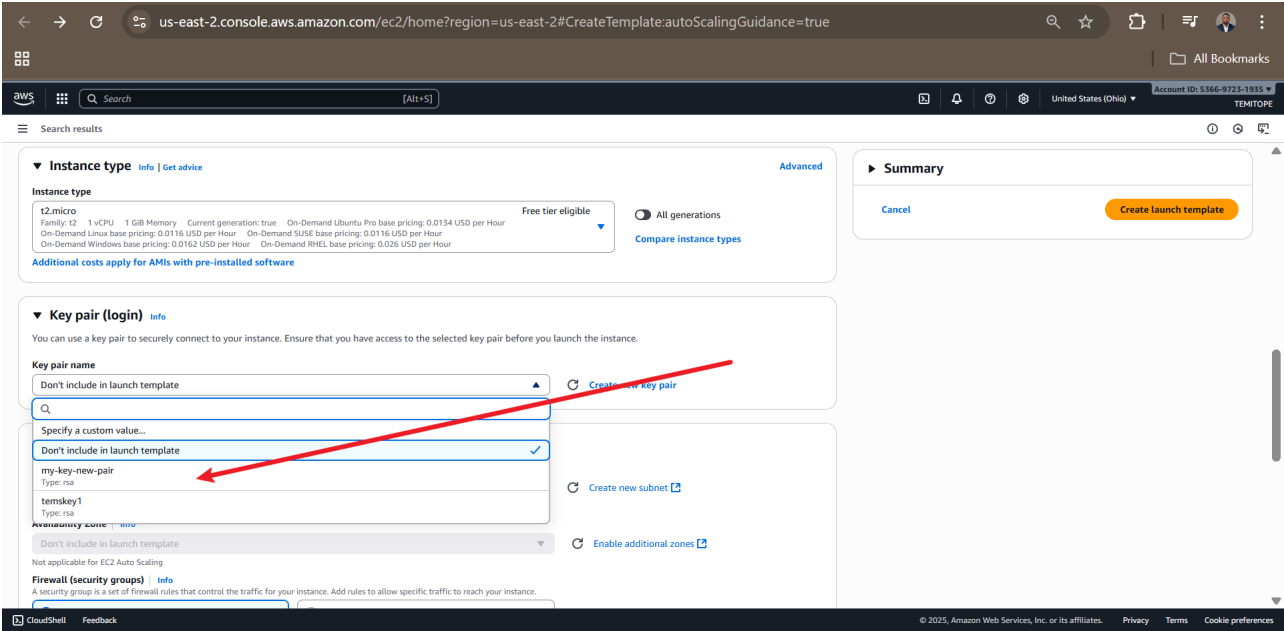
88. Image now selected scroll down.



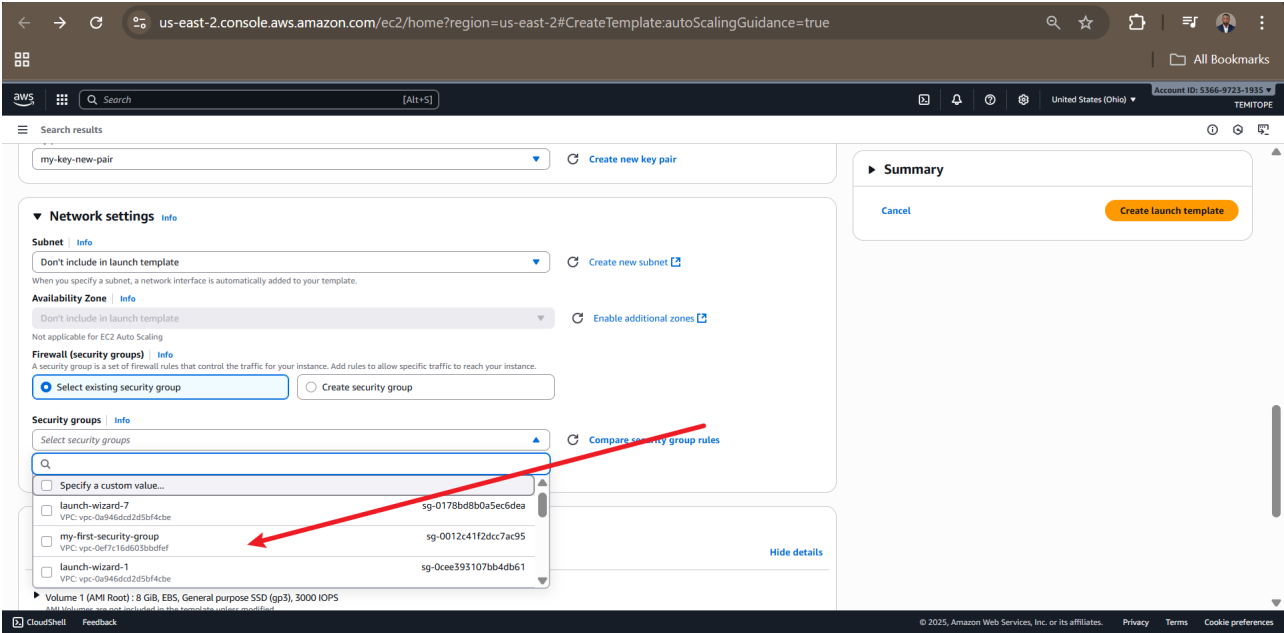
89. For instance type select t2 micro and scroll down.



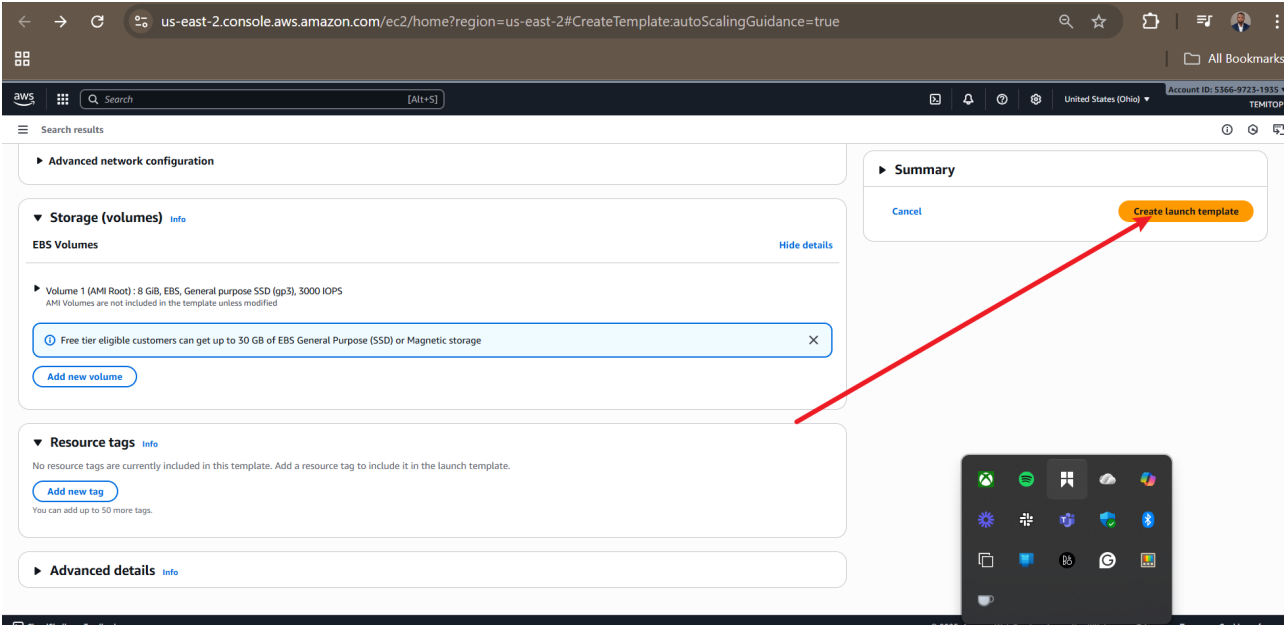
90. Select the key pair for all the instances.



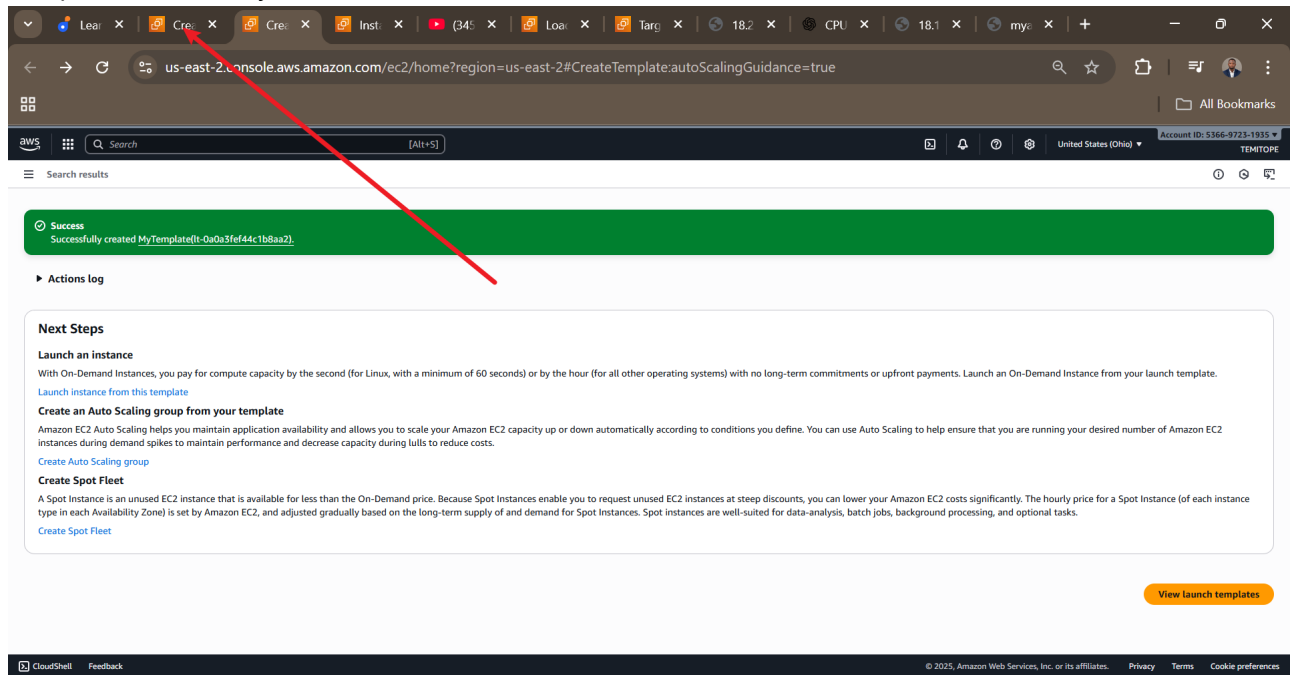
91. Select security group for all instances or create new.



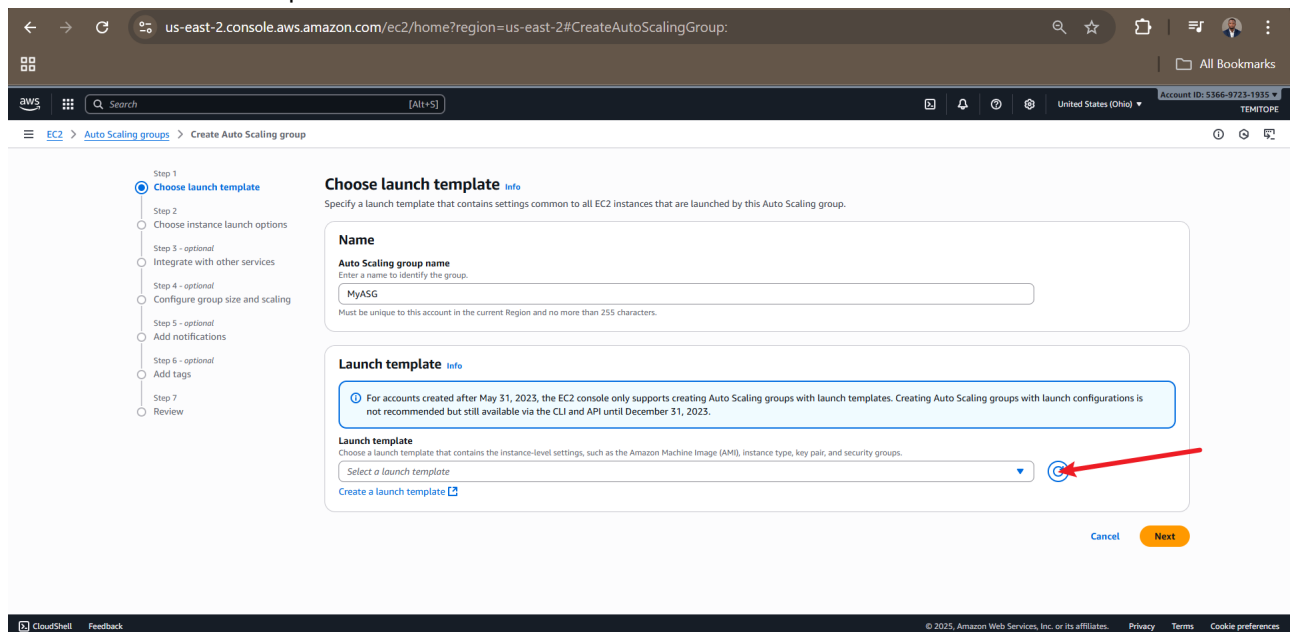
92. Leave everything and create launch template.



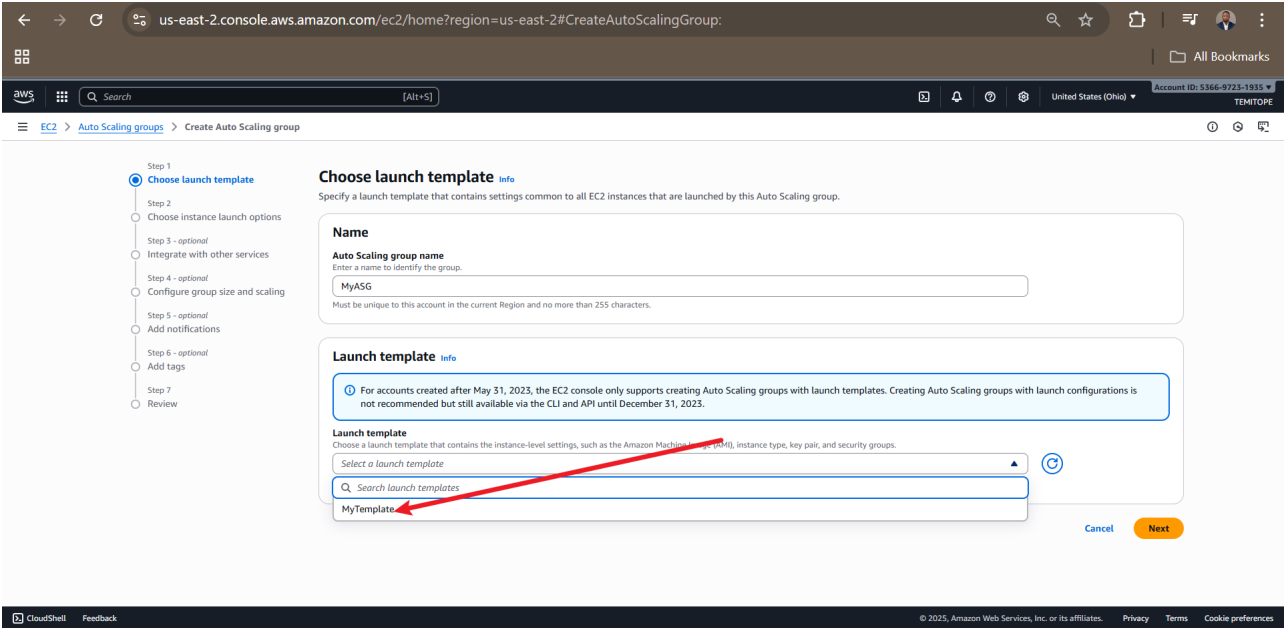
93. Template successfully created now continue to ASG creation tab.



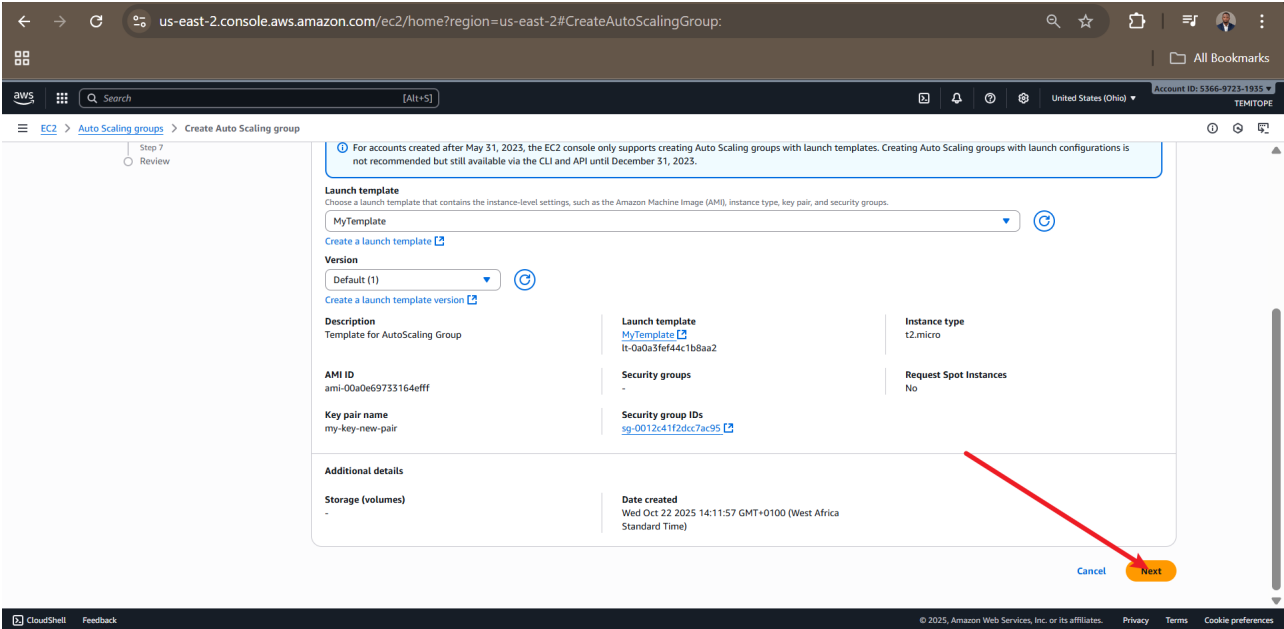
94. To choose created template click the refresh button.



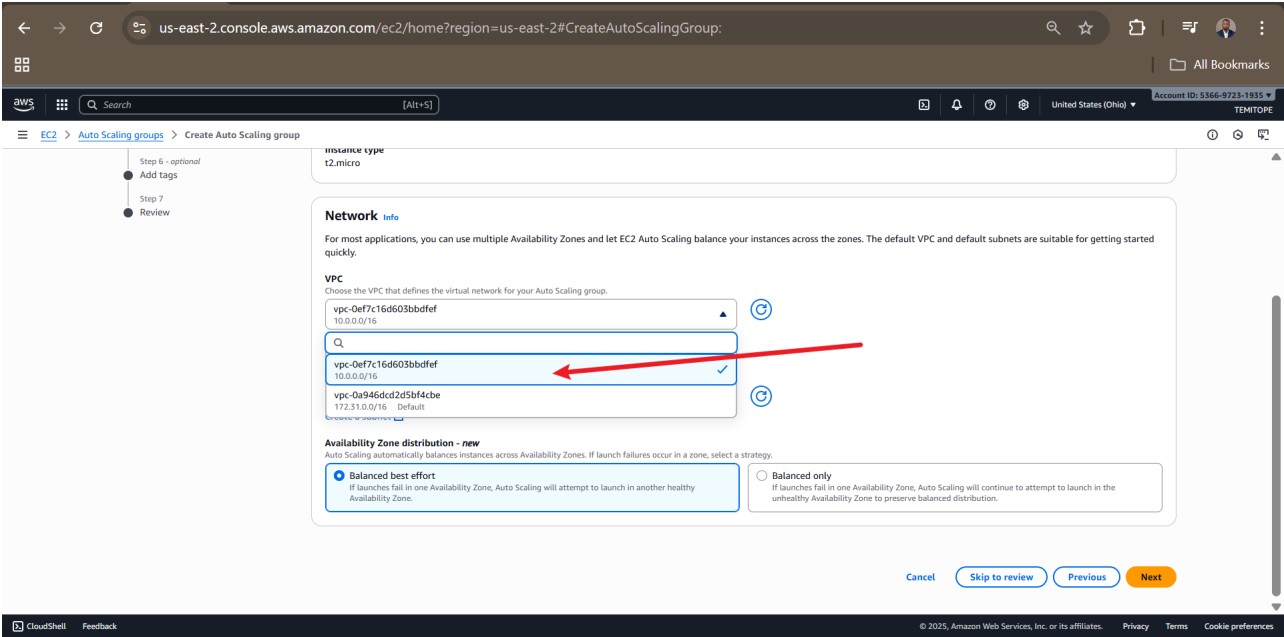
95. Search refreshed now select created template.



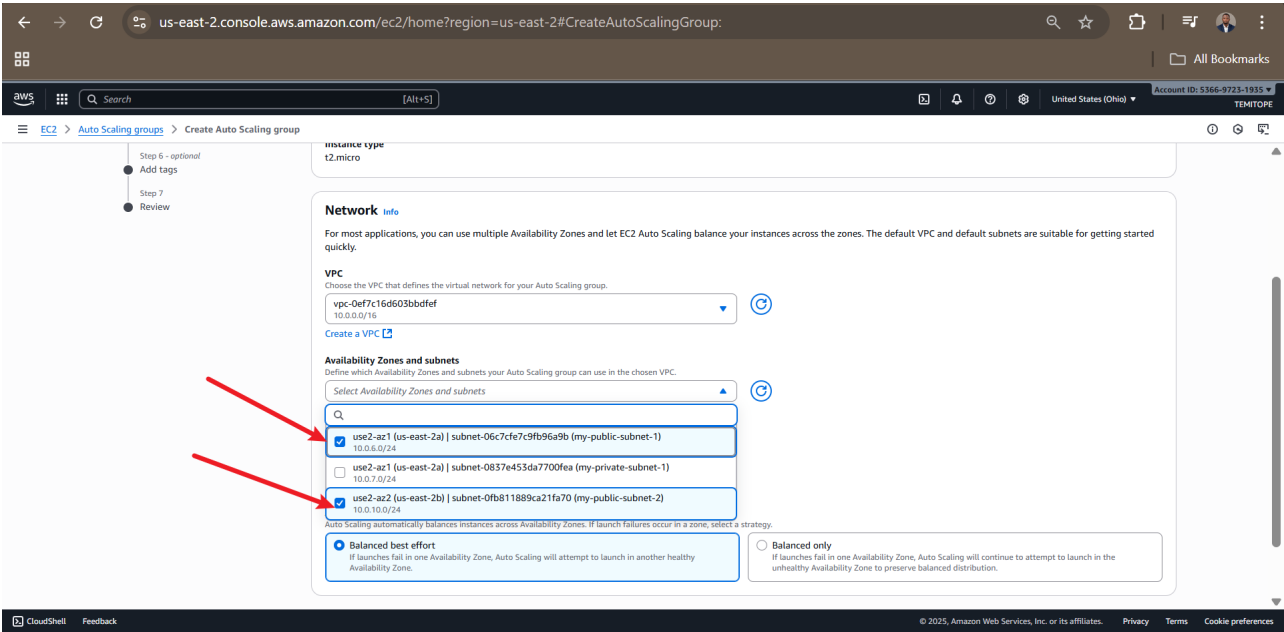
96. Template selected click next.



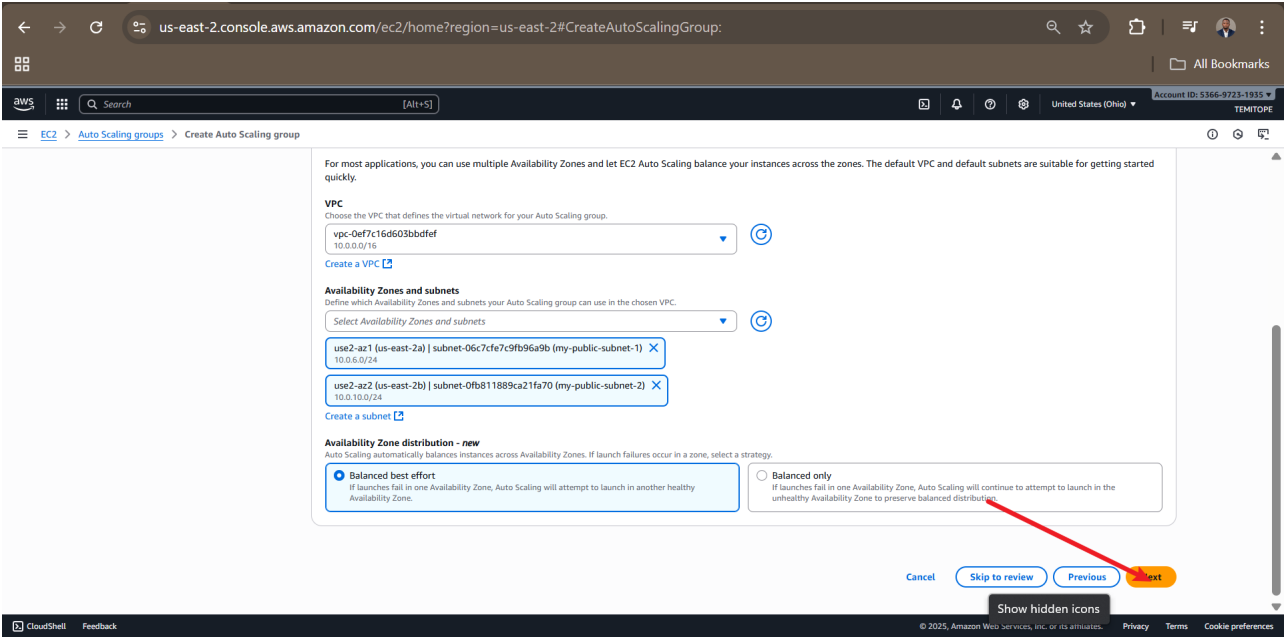
97. Choose VPC where servers are located.



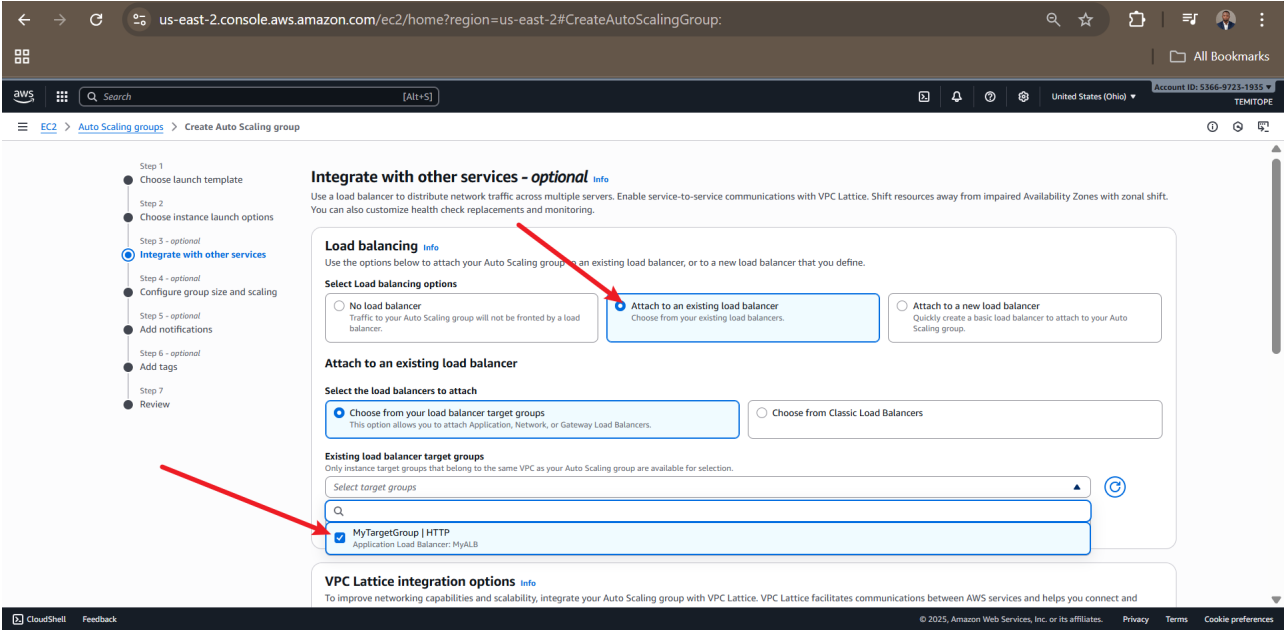
98. Select all the public availability zones if you have only one public create another one.



99. At least two public AZs selected click next.



100. Click attach to an existing load balancer and select created target group.



101. Turn on elastic load balancer healthcheck and click next.

us-east-2.console.aws.amazon.com/ec2/home?region=us-east-2#CreateAutoScalingGroup:

Application Recovery Controller (ARC) console shift - new info

During an Availability Zone impairment, target instance launches towards other healthy Availability Zones.

☐ Enable zonal shift

New instance launches will be retargeted towards healthy Availability Zones until the zonal shift is canceled.

Health checks

Health checks increase availability by replacing unhealthy instances. When you use multiple health checks, all are evaluated, and if at least one fails, instance replacement occurs.

EC2 health checks

[Always enabled](#)

Additional health check types - optional [Info](#)

☒ Turn on Elastic Load Balancing health checks **Recommended**

Elastic Load Balancing monitors whether instances are available to handle requests. When it reports an unhealthy instance, EC2 Auto Scaling can replace it on its next periodic check.

EC2 Auto Scaling will start to detect and act on health checks performed by Elastic Load Balancing. To avoid unexpected terminations, first verify the settings of these health checks in the Load Balancer console.

☐ Turn on VPC Lattice health checks

VPC Lattice can monitor whether instances are available to handle requests. If it considers a target as failed a health check, EC2 Auto Scaling replaces it after its next periodic check.

☐ Turn on Amazon EBS health checks

EBS monitors whether an instance's root volume or attached volume stalls. When it reports an unhealthy volume, EC2 Auto Scaling can replace the instance on its next periodic health check.

Health check grace period [Info](#)

This time period delays the first health check until your instances finish initializing. It doesn't prevent an instance from terminating when placed into a non-running state.

300 seconds

Cancel Skip to review Previous **Next**

102. Set desired capacity scaling limit (min and max) to manage cost.

us-east-2.console.aws.amazon.com/ec2/home?region=us-east-2#CreateAutoScalingGroup:

Step 1: Choose launch template

Step 2: Choose instance launch options

Step 3 - optional: Integrate with other services

Step 4 - optional: **Configure group size and scaling**

Step 5 - optional: Add notifications

Step 6 - optional: Add tags

Step 7: Review

Configure group size and scaling - optional [Info](#)

Define your group's desired capacity and scaling limits. You can optionally add automatic scaling to adjust the size of your group.

Group size [Info](#)

Set the initial size of the Auto Scaling group. After creating the group, you can change its size to meet demand, either manually or by using automatic scaling.

Desired capacity type

Choose the unit of measurement for the desired capacity value. vCPUs and Memory (GiB) are only supported for mixed instances groups configured with a set of instance attributes.

Units (number of instances)

Desired capacity

Specify your group size.

1

Scaling [Info](#)

You can resize your Auto Scaling group manually or automatically to meet changes in demand.

Scaling limits

Set limits on how much your desired capacity can be increased or decreased.

Min desired capacity

1

Equal or less than desired capacity

Max desired capacity

5

Equal or greater than desired capacity

103. Choose target tracking scaling policy and set name metric target value.

us-east-2.console.aws.amazon.com/ec2/home?region=us-east-2#CreateAutoScalingGroup:

Automatic scaling - optional [Info](#)

Choose whether to use a target tracking policy

☐ No scaling policies

Your Auto Scaling group will remain at its initial size and will not dynamically resize to meet demand.

☒ Target tracking scaling policy

Choose a CloudWatch metric and target value and let the scaling policy adjust the desired capacity in proportion to the metric's value.

Scaling policy name

Target Tracking Policy

Metric type [Info](#)

Monitored metric that determines if resource utilization is too low or high. If using EC2 metrics, consider enabling detailed monitoring for better scaling performance.

Average CPU utilization

Target value

75%

Instance warmup [Info](#)

300 seconds

☐ Disable scale in to create only a scale-out policy

Instance maintenance policy [Info](#)

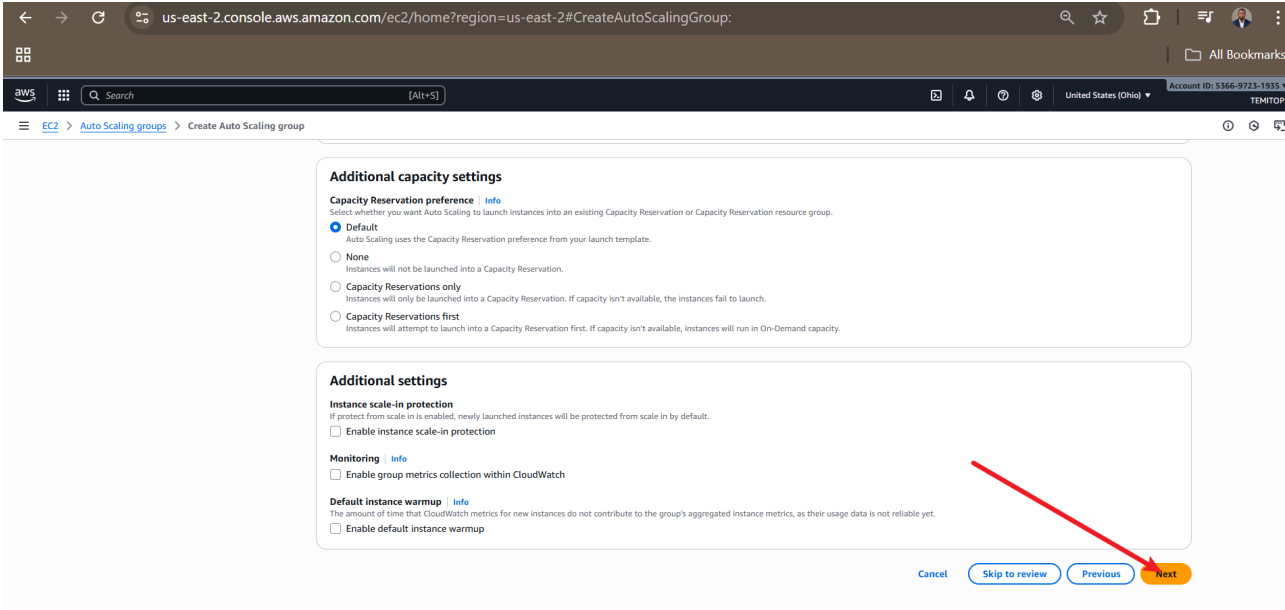
Control your Auto Scaling group's availability during instance replacement events. This includes health checks, instance refreshes, maximum instance lifetime features and events that happen automatically to keep your group balanced, called rebalancing events.

Choose a replacement behavior depending on your availability requirements

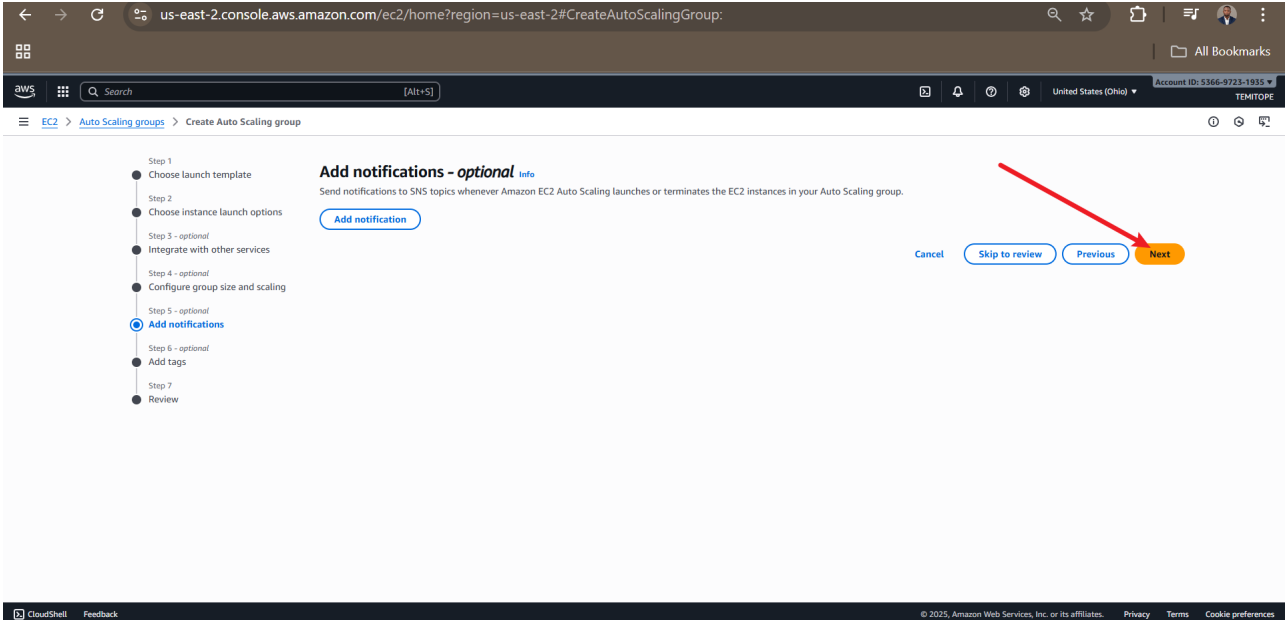
Mixed behavior Prioritize availability Control costs

© 2025, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences

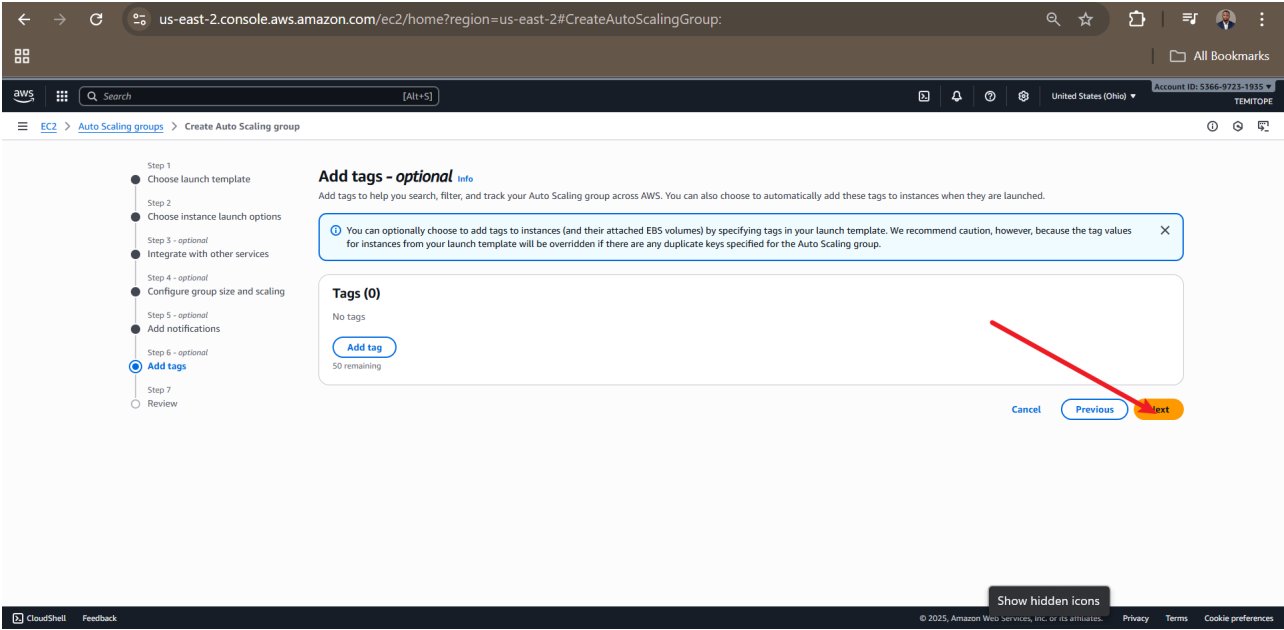
104. Leave everything at default and click next.



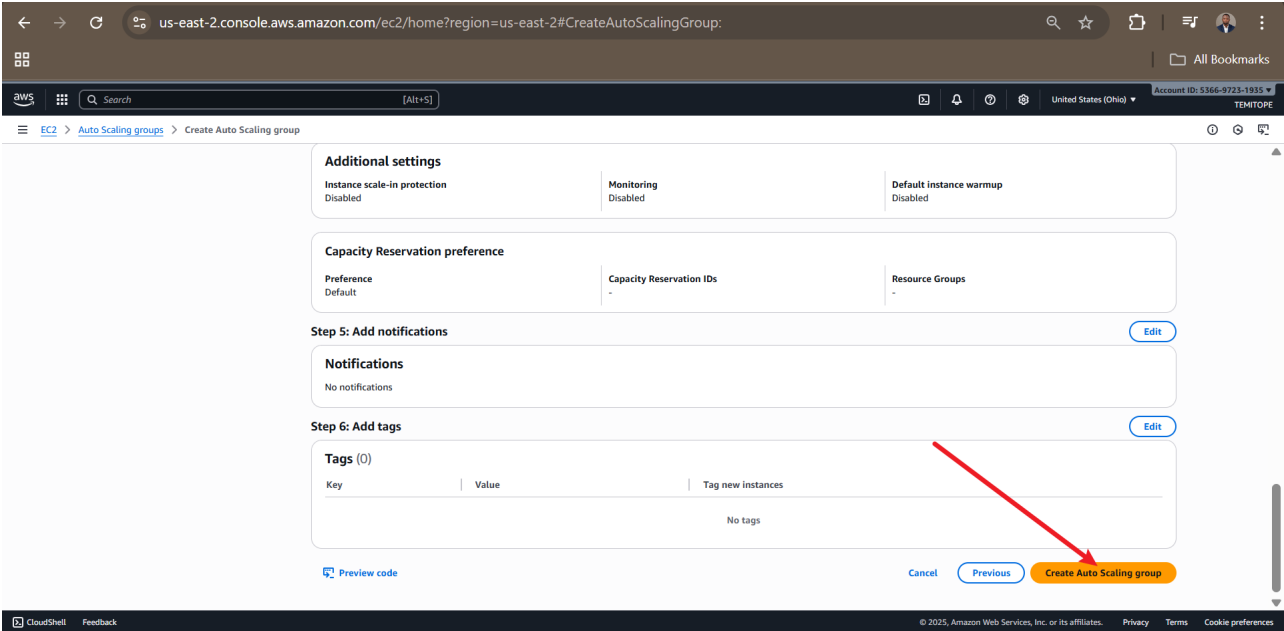
105. Leave add notification at default and click next.



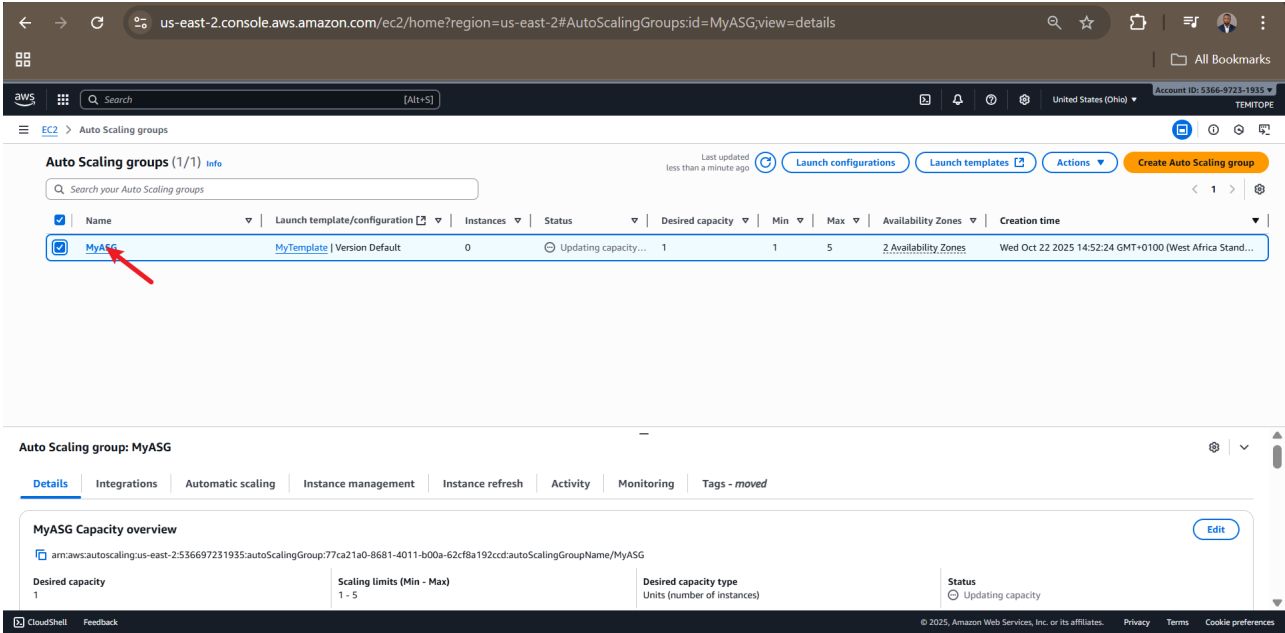
106. Leave add tags at default and click next.



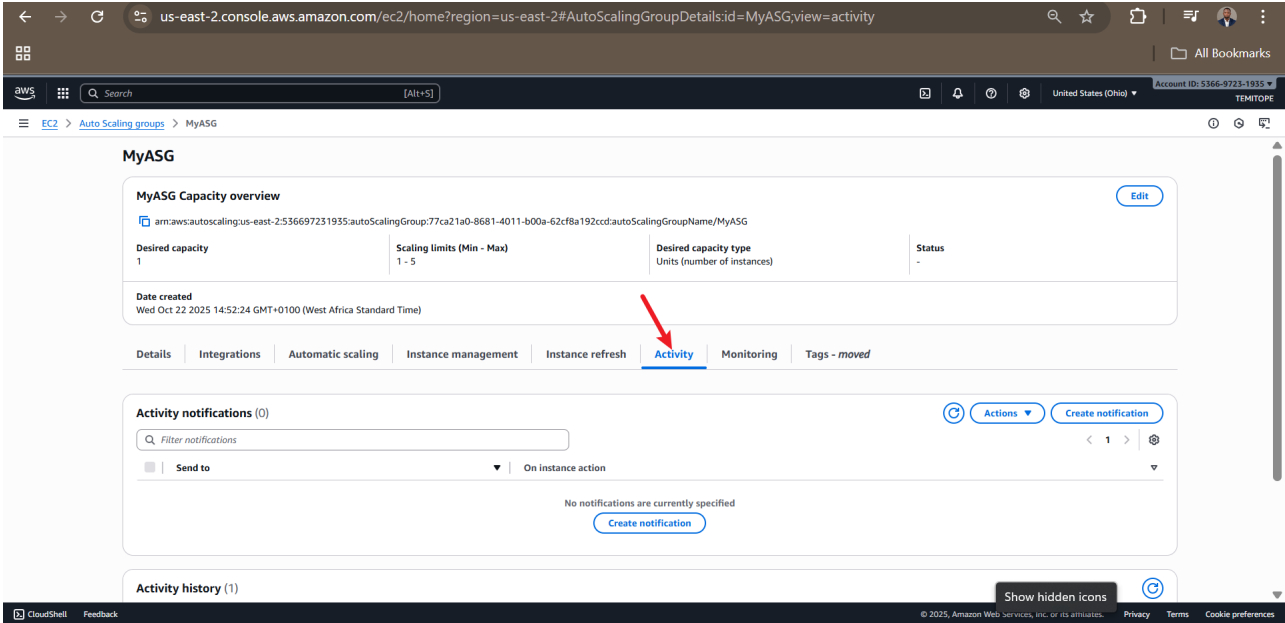
107. On the review page click Create Auto Scaling Group.



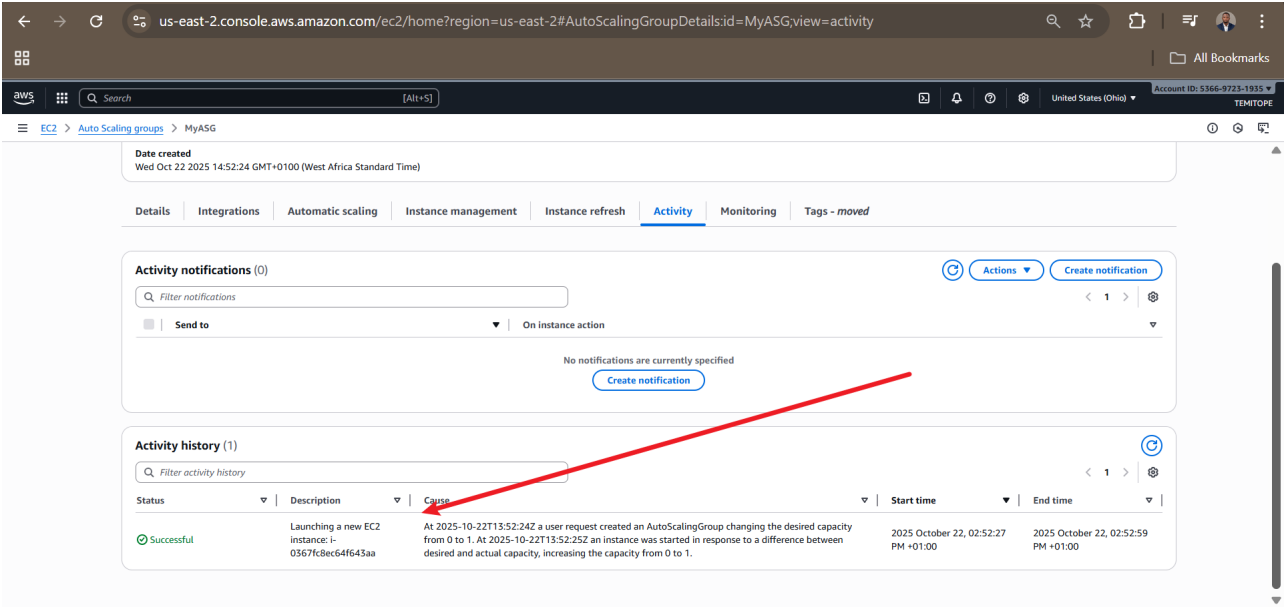
108. Auto scaling group successfully created click on it.



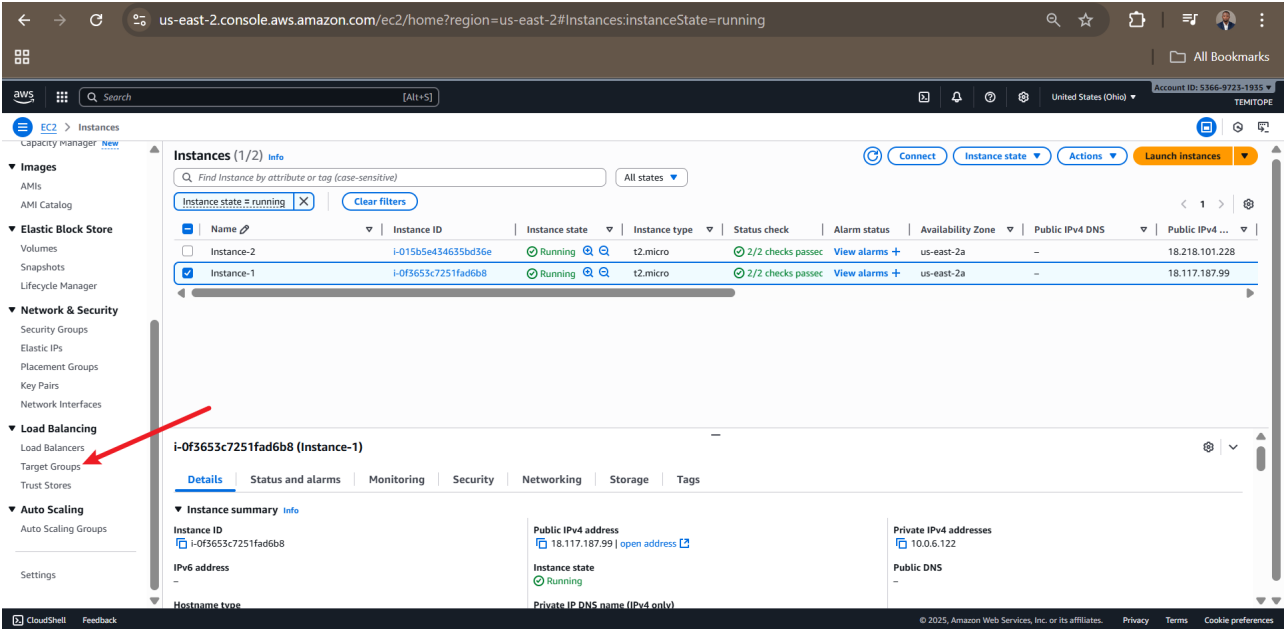
109. On created ASG dashboard click on activity tab and scroll down.



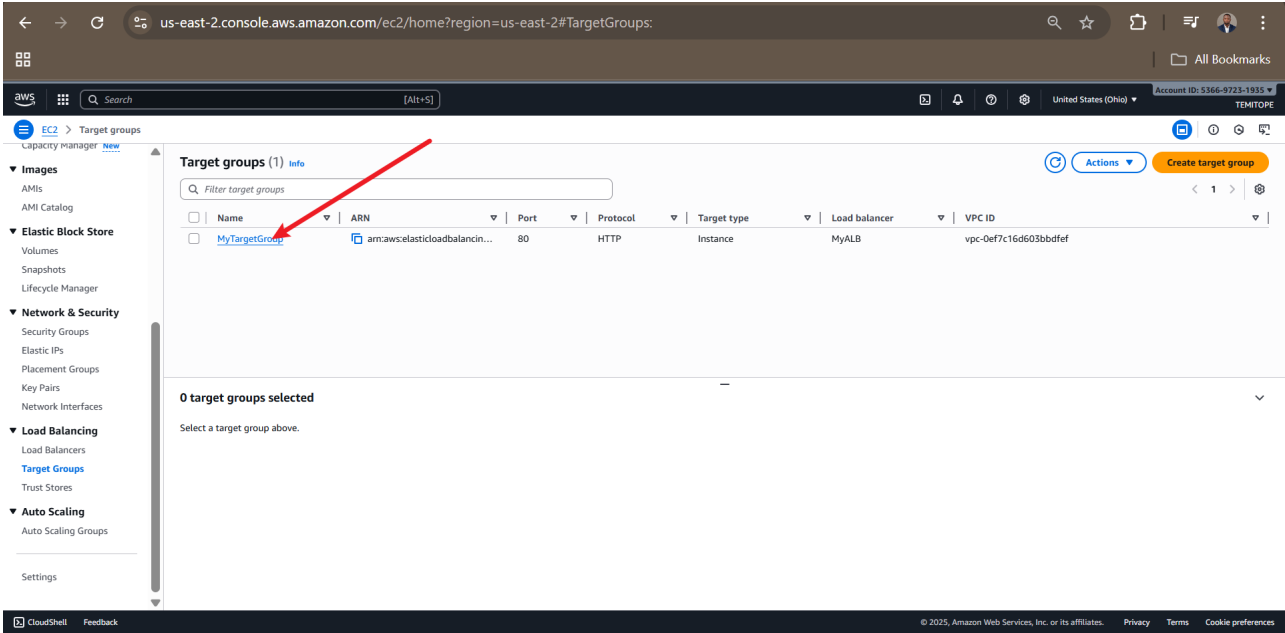
110. Activity history shows launching a new EC2 instance increasing capacity from 0-1.



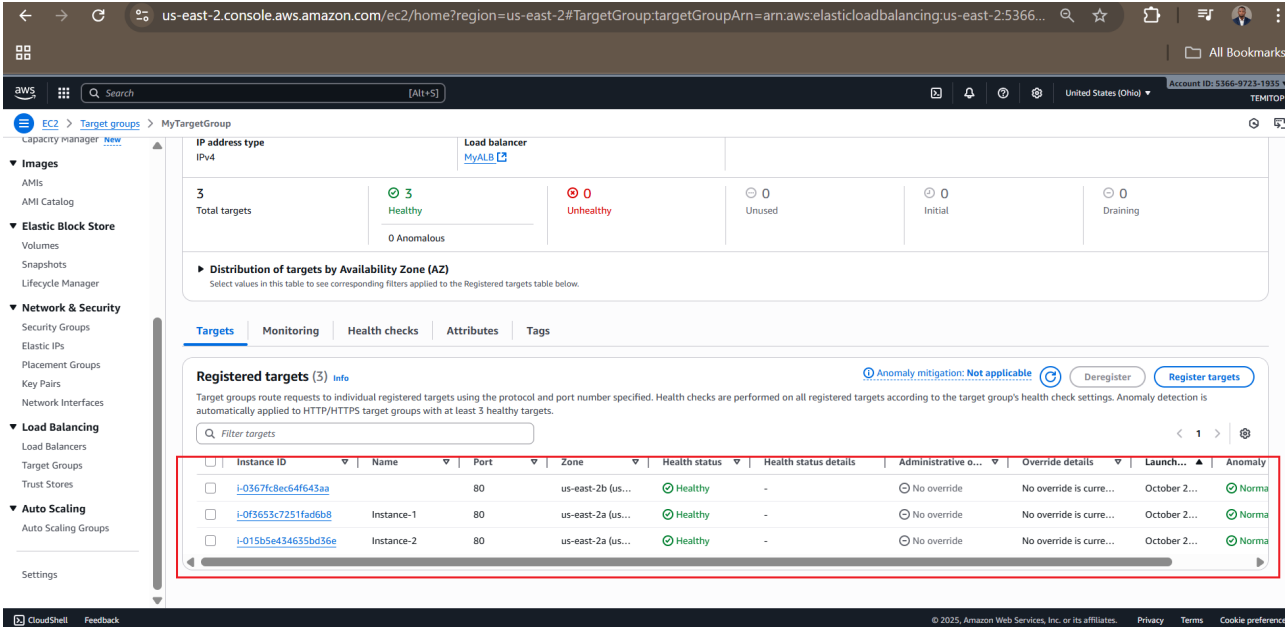
111. Navigate to instances page and under load balancing click on target group.



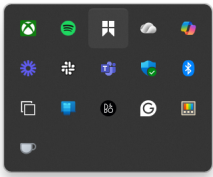
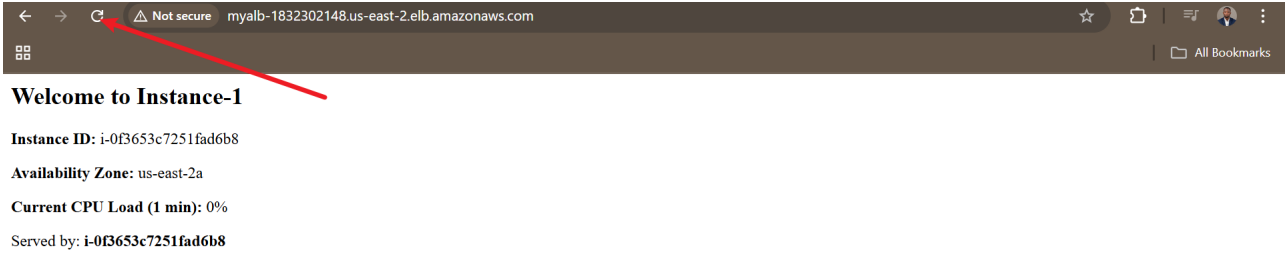
112. Click on the target group for the ALB and ASG.



113. Observe that there are now three instances under targets.



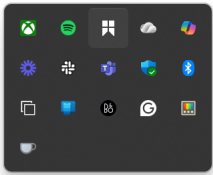
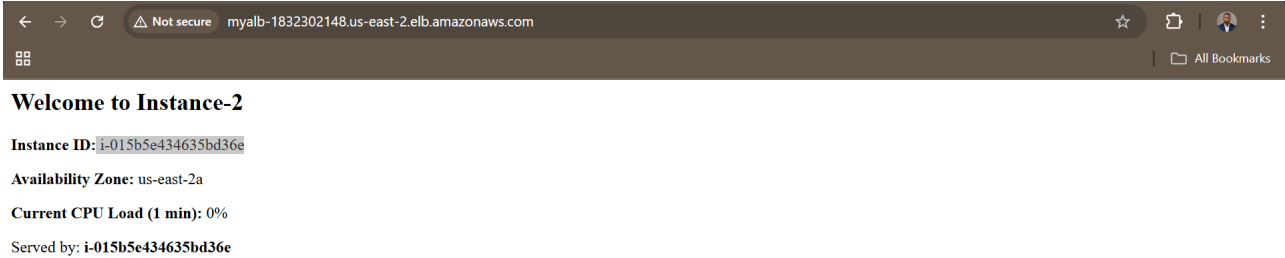
114. Proceed to the webpage and reload page.



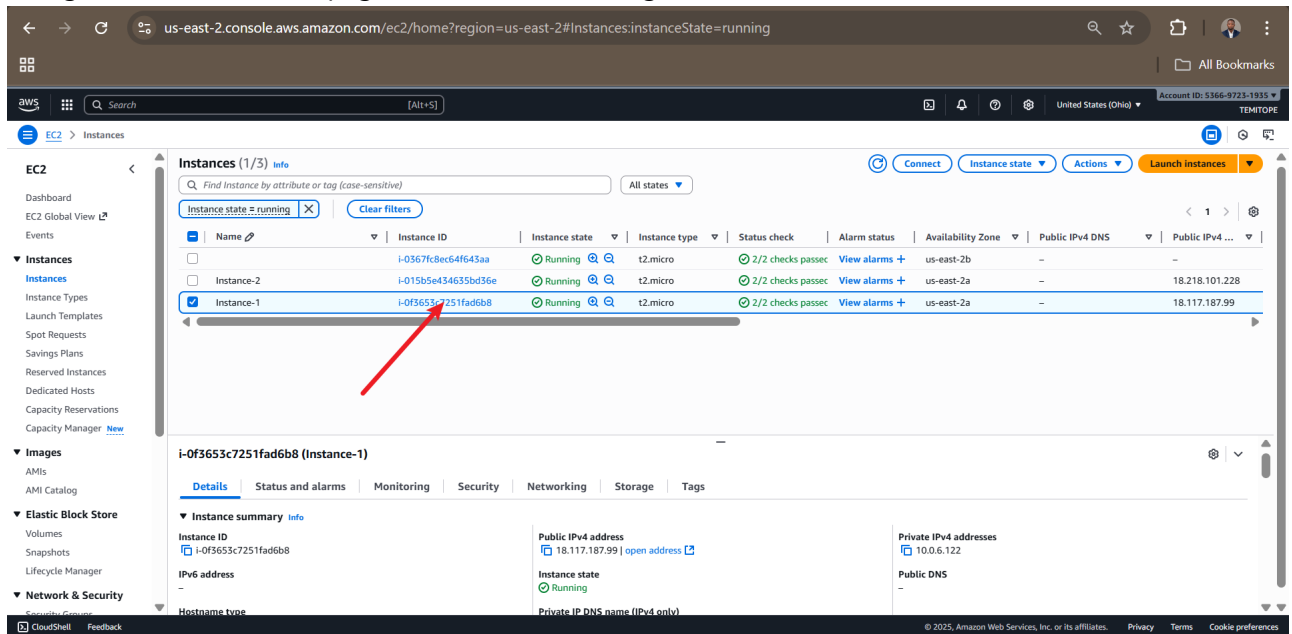
115. Instance with ID i-015b5e434635bd36e now displays reload again.
![[Instance with ID i-015b5e434635bd36e now displays reload again](./img/114.instance_with_id_i-015b5e434635bd36e_now_displays_reload_again.png)]

116. Second instance with ID i-0367fc8ec64f643aa now displays reload again.
![[Second instance with ID i-0367fc8ec64f643aa now displays reload again](./img/115.second_instance_with_id_i-0367fc8ec64f643aa_now_displays_reload_again.png)]

117. Third instance with ID i-015b5e434635bd36e now displays reload again.



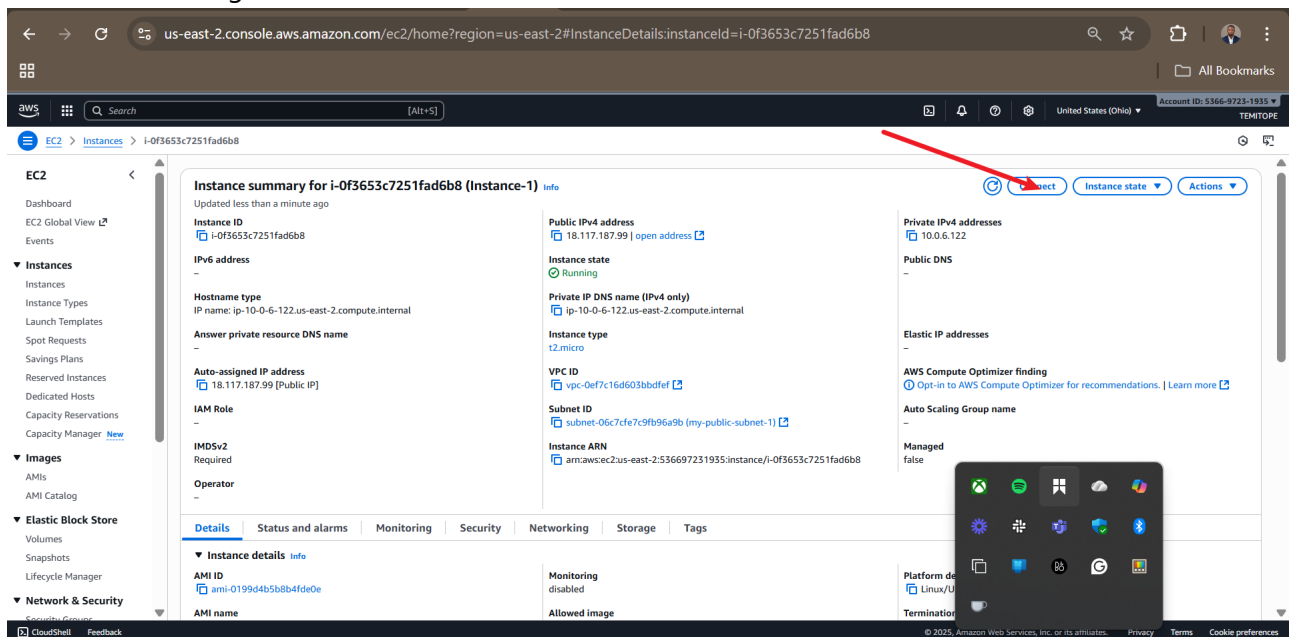
118. Navigate to the instances page select one of the original instances and click on it.



The screenshot shows the AWS Management Console for the 'us-east-2' region. The 'Instances' page is active, displaying a table of EC2 instances. A red arrow points to the row for 'Instance-1' with ID 'i-0f3653c7251fad6b8'. The table columns include Name, Instance ID, Instance state, Instance type, Status check, Alarm status, Availability Zone, Public IPv4 DNS, and Public IPv4 address.

Name	Instance ID	Instance state	Instance type	Status check	Alarm status	Availability Zone	Public IPv4 DNS	Public IPv4 address
Instance-1	i-0f3653c7251fad6b8	Running	t2.micro	2/2 checks passed	View alarms +	us-east-2a	-	18.117.187.99

119. On the chosen original instance dashboard click connect.

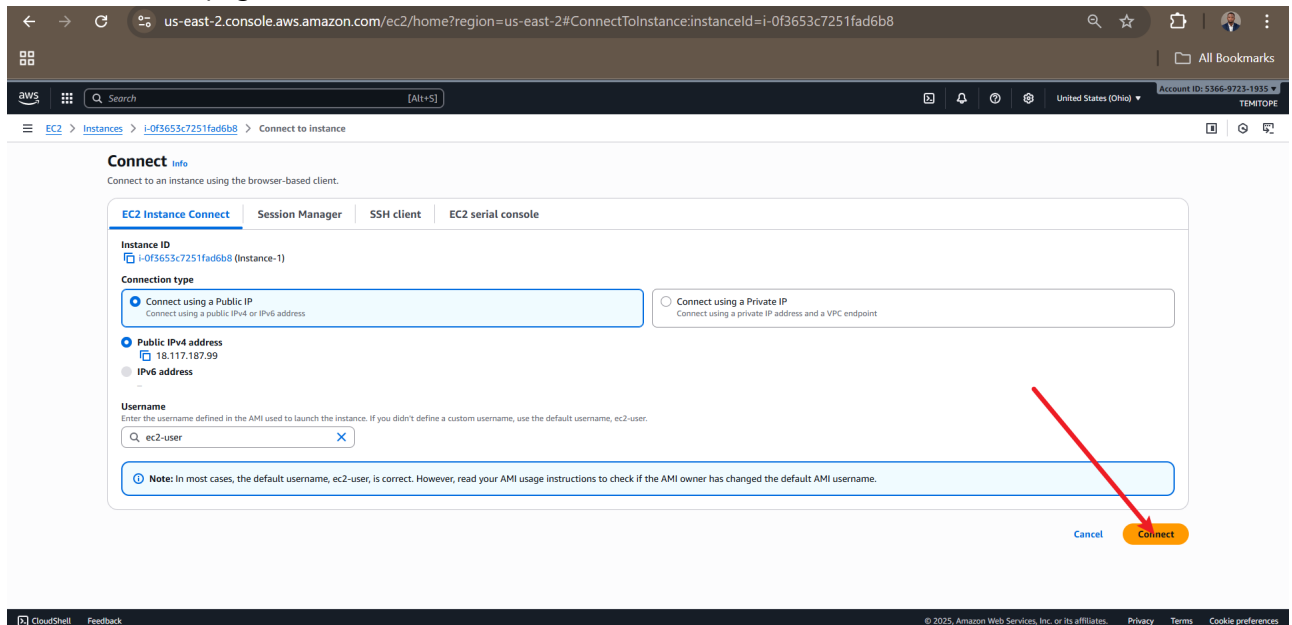


The screenshot shows the 'Instance details' page for 'Instance-1' (ID: i-0f3653c7251fad6b8). A red arrow points to the 'Connect' button in the top right corner. The page displays various instance details, including the Public IPv4 address, Instance state, Private IP DNS name, VPC ID, Subnet ID, and Instance ARN.

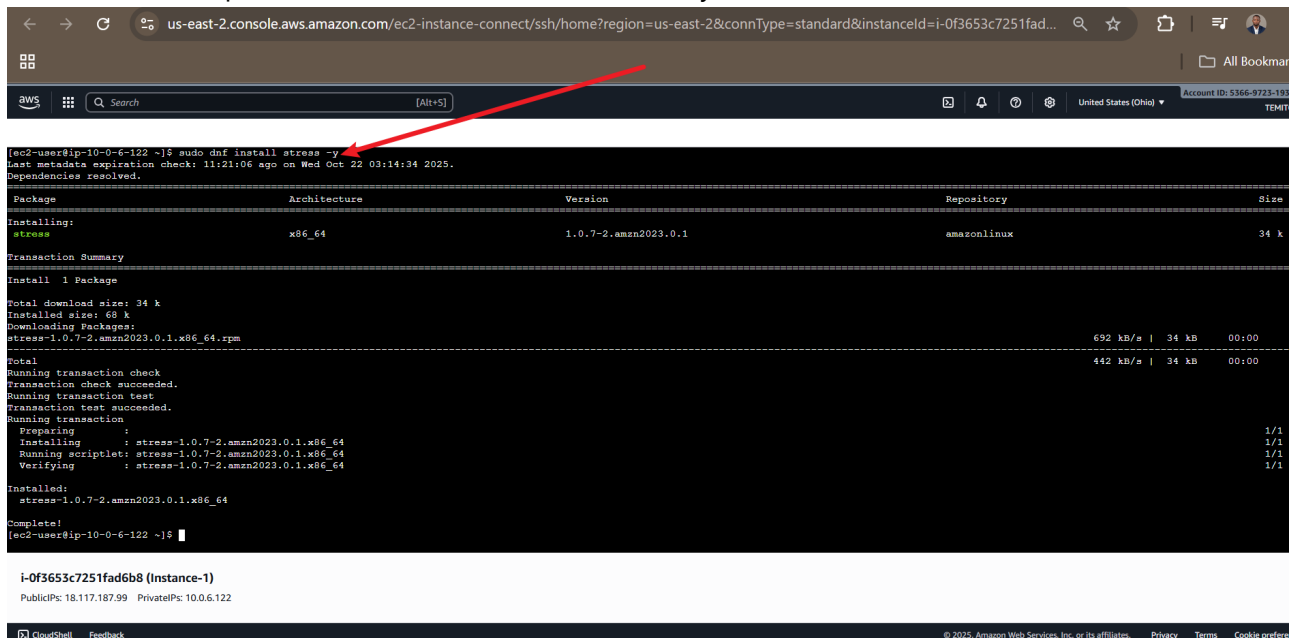
Instance summary for i-0f3653c7251fad6b8 (Instance-1)

- Instance ID: i-0f3653c7251fad6b8
- Public IPv4 address: 18.117.187.99 | open address
- Instance state: Running
- Private IP DNS name (IPv4 only): ip-10-0-6-122.us-east-2.compute.internal
- Instance type: t2.micro
- VPC ID: vpc-0ef7c16d603bbdfef
- Subnet ID: subnet-06c7cf7c9fb96a9b (my-public-subnet-1)
- Instance ARN: arn:aws:ec2:us-east-2:536697231935:instance/i-0f3653c7251fad6b8

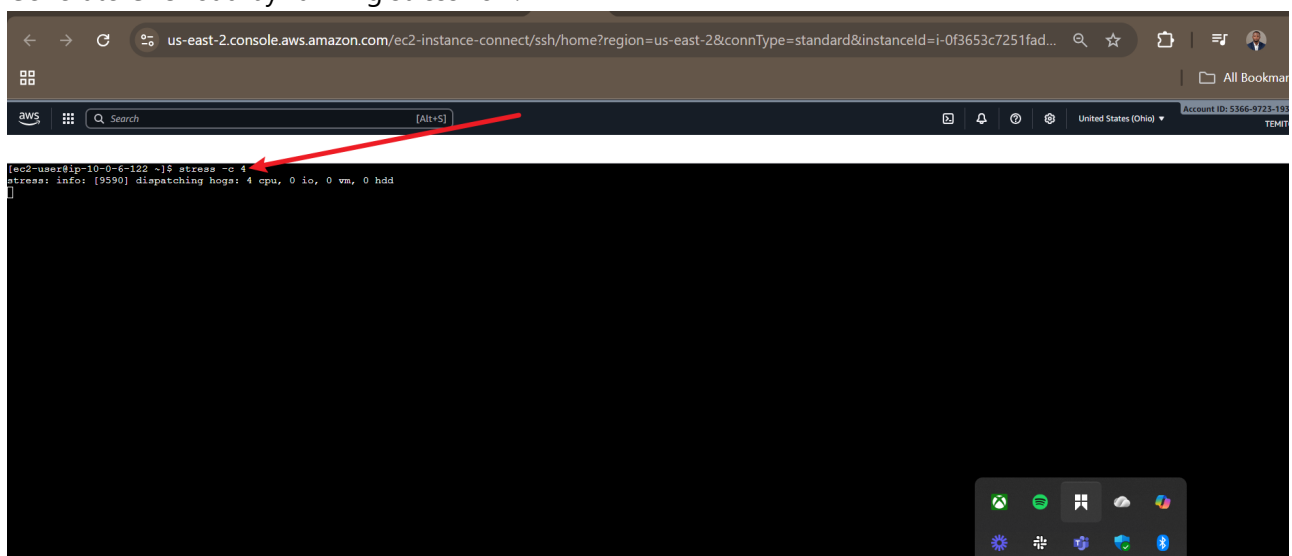
120. On the connect page under the EC2 instance connect tab click connect.



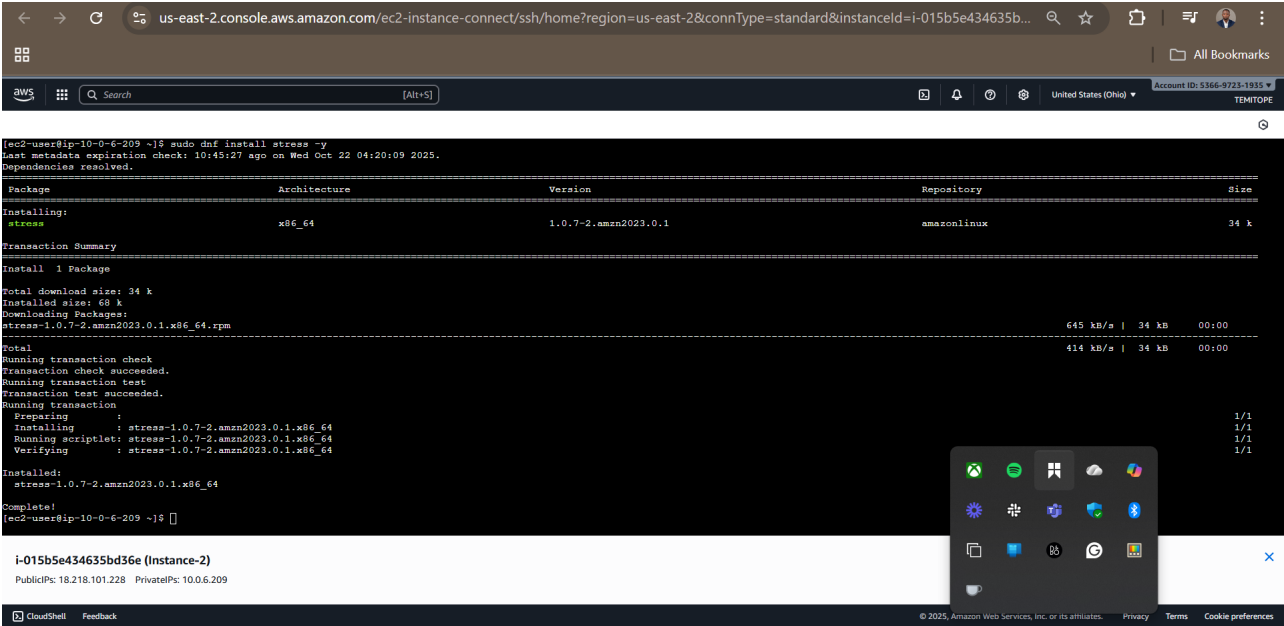
121. On the terminal update then run `sudo dnf install stress -y`.



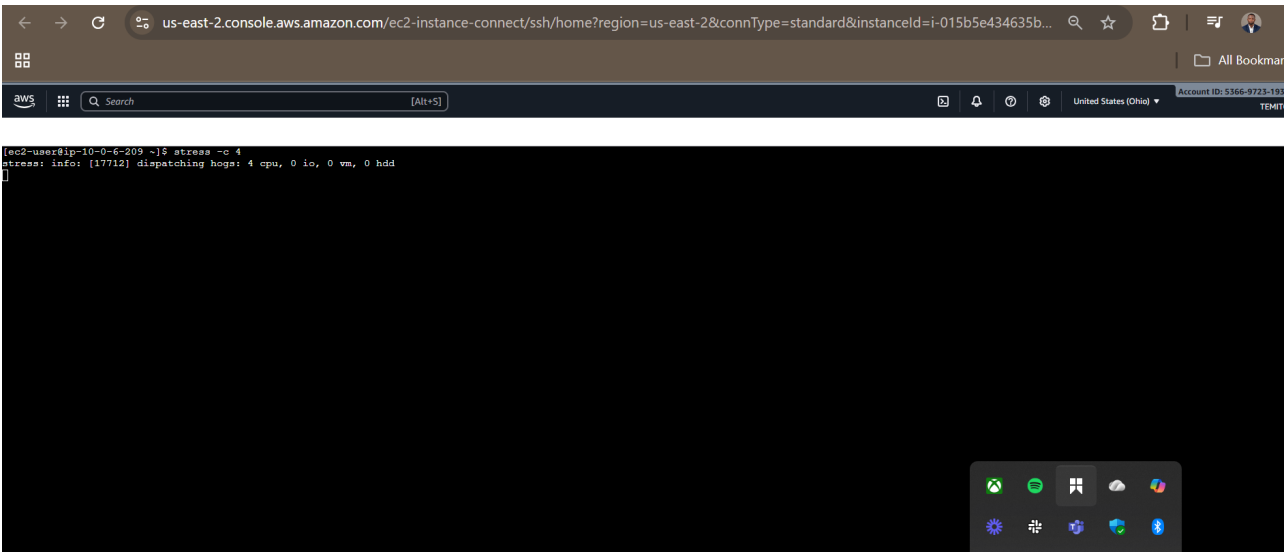
122. Generate CPU load by running `stress -c 4`.



123. Note that after reloading CPU utilization of i-0f3653c7251fad6b8 has now increased to 400 percent.
![Note that after reloading CPU utilization of i-0f3653c7251fad6b8 has now increased to 400 percent]
(./img/122.note_that_after_reloading_cpu_utilization_of_i-0f3653c7251fad6b8_has_now_increased_to_400_percent.png)
124. Install stress on the second original instance as well.



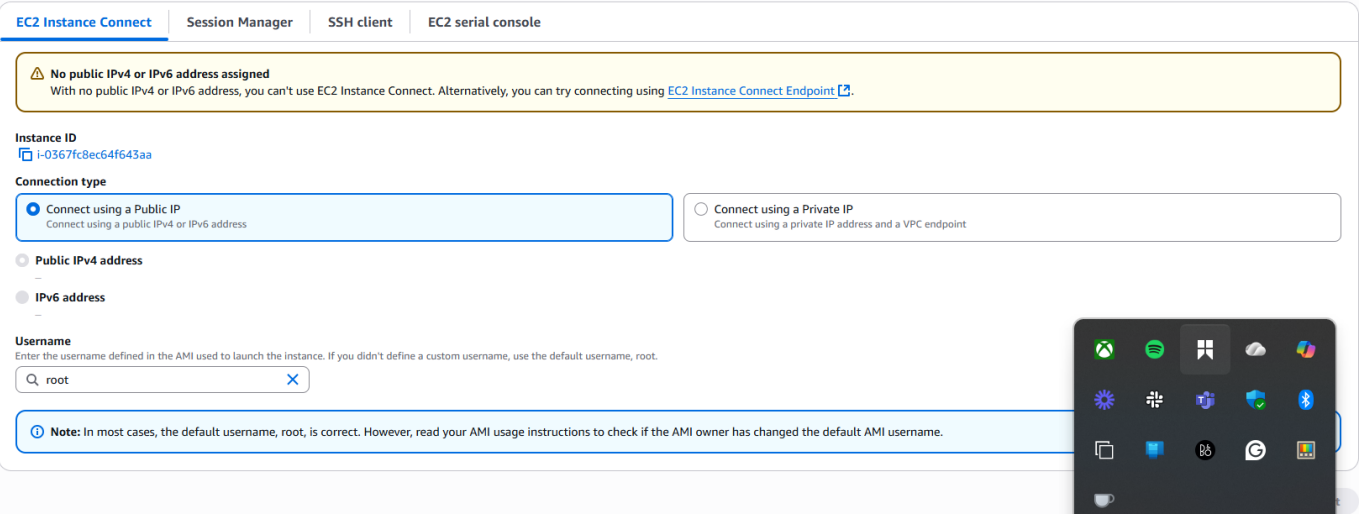
125. Stress the second original instance.



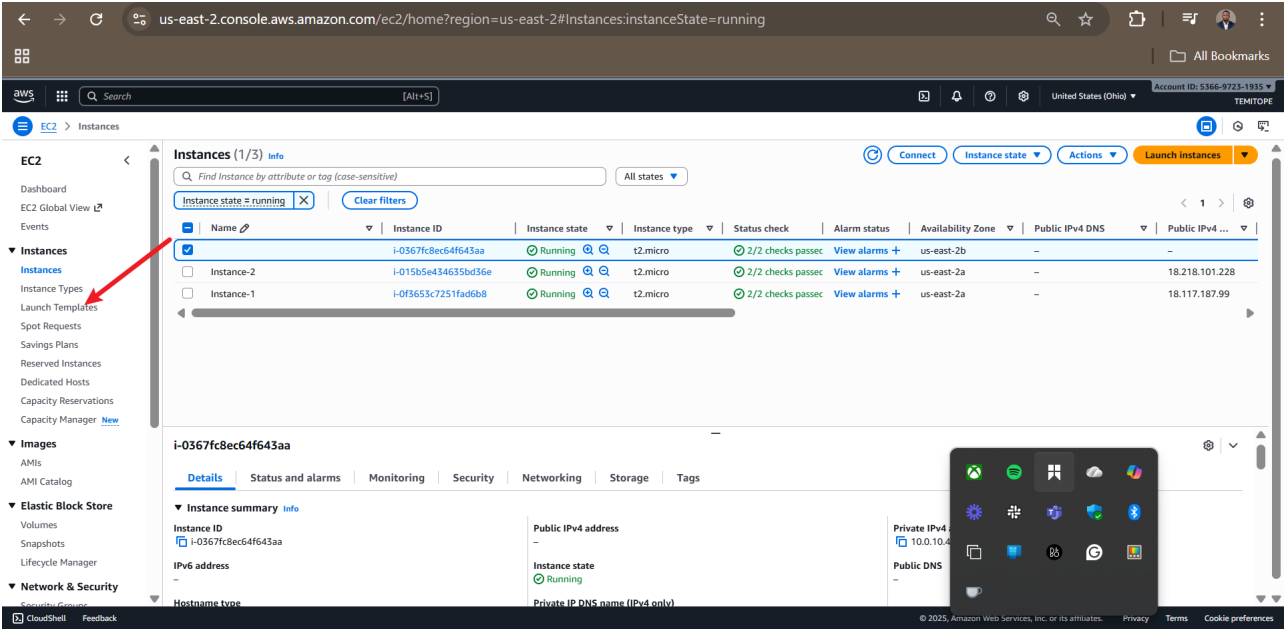
124b. Note that the ASG created instance can not connect like the originals.

Connect Info

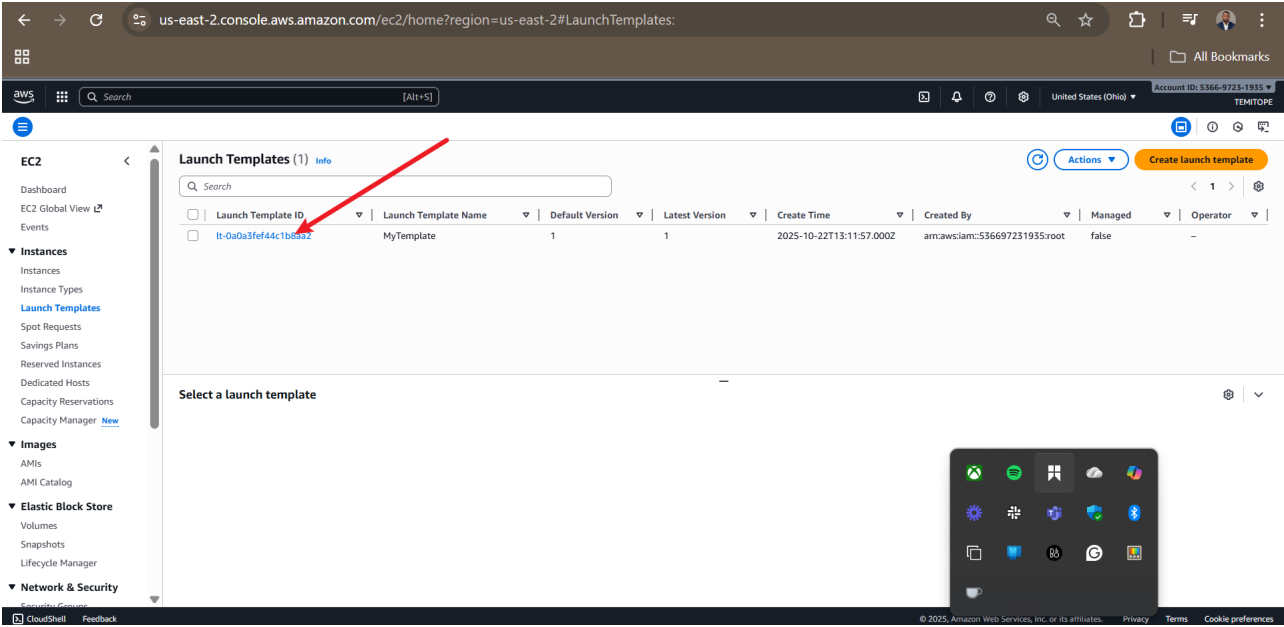
Connect to an instance using the browser-based client.



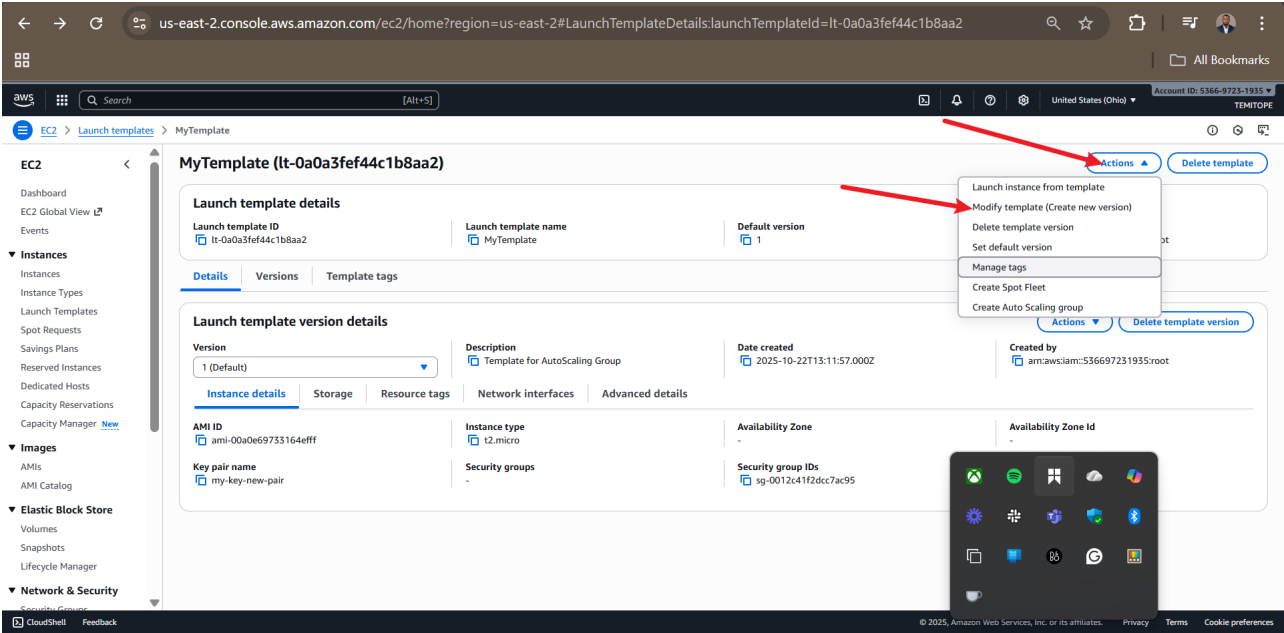
125. Navigate to instances page and click on launch template.



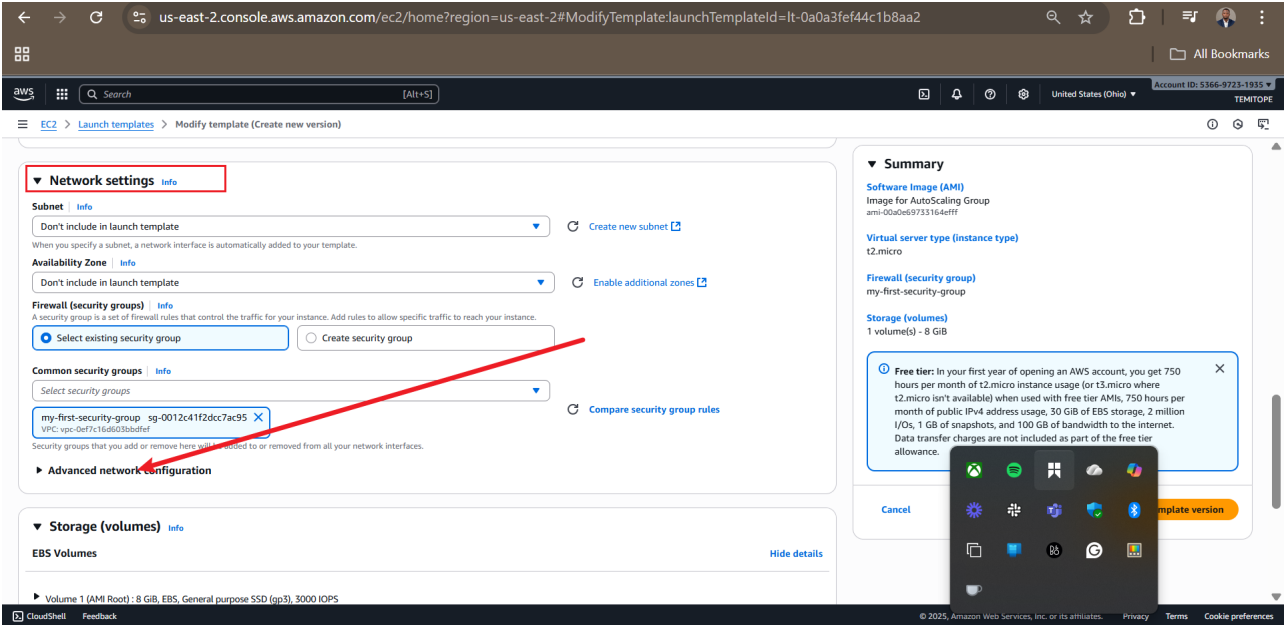
126. Click on the created launch template.



127. Click on actions then modify template (create new version).



128. Now scroll down to network setting click on advanced network configuration.



129. Under Advanced network configuration enable Auto Assign Public IP create new version.

The screenshot shows the AWS Management Console interface for modifying a launch template. The browser address bar shows the URL: `us-east-2.console.aws.amazon.com/ec2/home?region=us-east-2#ModifyTemplate:launchTemplateId=lt-0a0a3fef44c1b8aa2`. The page title is "Modify template (Create new version)".

In the "Advanced network configuration" section, the "Auto-assign public IP" dropdown is set to "Enable". A red box highlights this dropdown, and a red arrow points to the "Create template version" button in the Summary panel on the right.

The Summary panel on the right shows the following details:

- Software Image (AMI):** Image for AutoScaling Group `ami-0b0e4d9735164aff7`
- Virtual server type (instance type):** `t2.micro`
- Firewall (security group):** `my-first-security-group`
- Storage (volumes):** 1 volume(s) - 8 GB

A free tier notification is also visible: "Free tier: In your first year of opening an AWS account, you get 750 hours per month of t2.micro instance usage (or t3.micro where t2.micro isn't available) when used with free tier AMIs, 750 hours per month of public IPv4 address usage, 30 GiB of EBS storage, 2 million I/Os, 1 GiB of snapshots, and 100 GiB of bandwidth to the Internet. Data transfer charges are not included as part of the free tier allowance."

130. Successfully modified template click view launch templates.

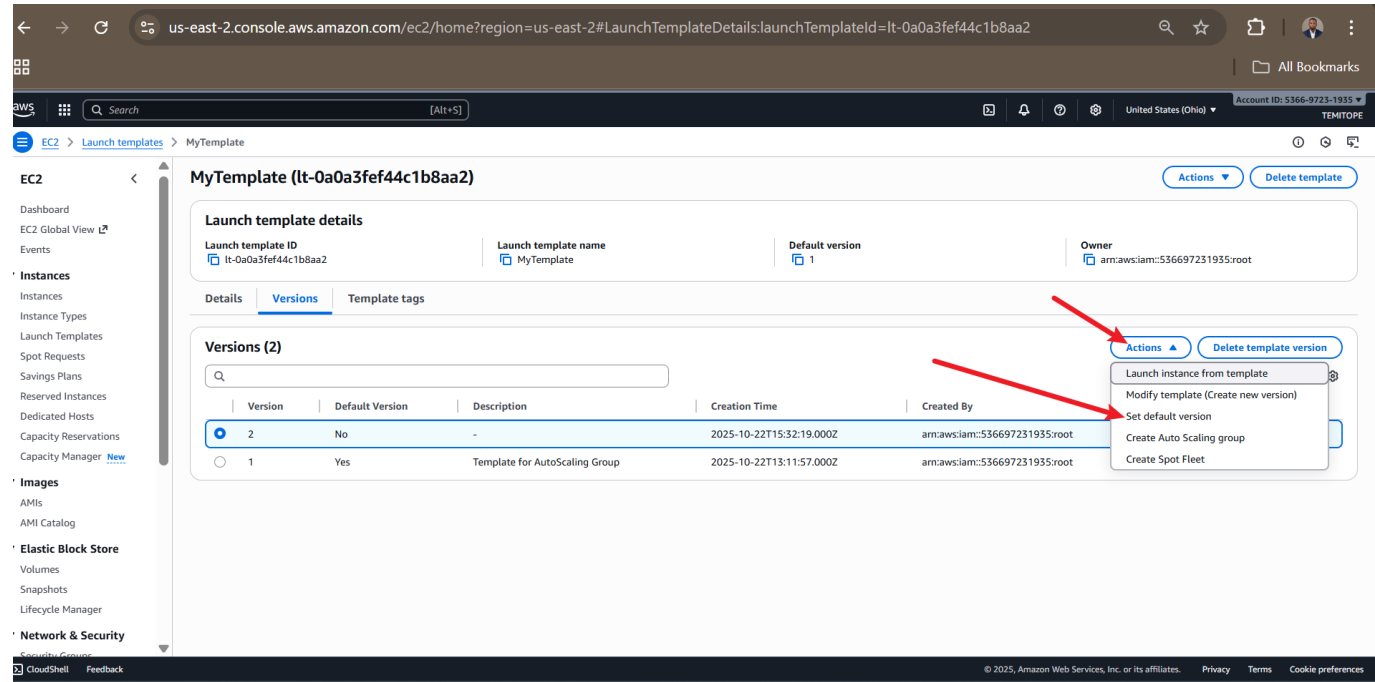
The screenshot shows the AWS Management Console interface for the "Next Steps" page after successfully modifying a launch template. The browser address bar shows the URL: `us-east-2.console.aws.amazon.com/ec2/home?region=us-east-2#ModifyTemplate:launchTemplateId=lt-0a0a3fef44c1b8aa2`. The page title is "Modify template (Create new version)".

The "Next Steps" section includes the following actions:

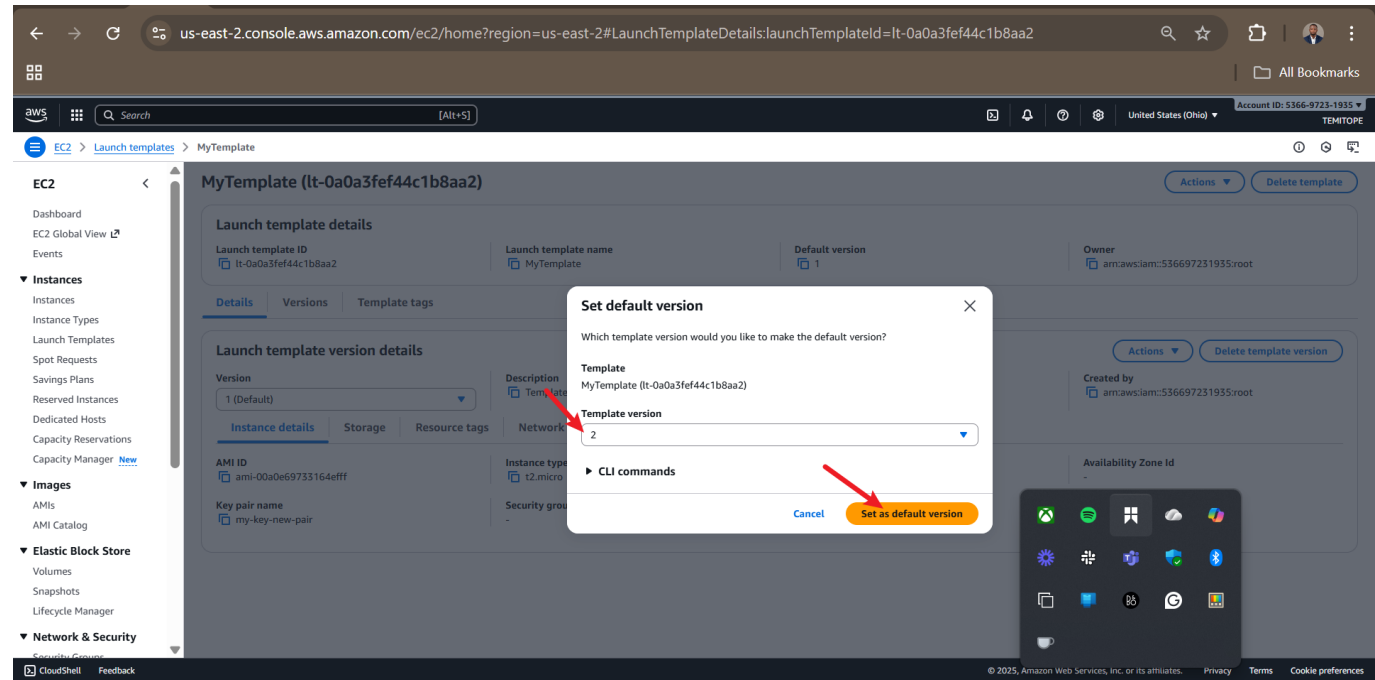
- Launch an instance:** With On-Demand Instances, you pay for compute capacity by the second (or by the hour for all other operating systems) with no long-term commitments or upfront payments. Launch an On-Demand Instance from your launch template. [Launch instance from this template](#)
- Create an Auto Scaling group from your template:** Amazon EC2 Auto Scaling helps you maintain application availability and allows you to scale your Amazon EC2 capacity up or down automatically according to conditions you define. You can use Auto Scaling to help ensure that you are running your desired number of Amazon EC2 instances during demand spikes to maintain performance and decrease capacity during lulls to reduce costs. [Create Auto Scaling group](#)
- Create Spot Fleet:** A Spot Instance is an unused EC2 instance that is available for less than the On-Demand price. Because Spot Instances enable you to request unused EC2 instances at steep discounts, you can lower your Amazon EC2 costs significantly. The hourly price for a Spot Instance in each instance type in each Availability Zone is set by Amazon EC2, and adjusted gradually based on the long-term supply of and demand for Spot Instances. Spot instances are well-suited for data-analysis, batch jobs, background processing, and optional tasks. [Create Spot Fleet](#)

A red arrow points to the "View launch templates" button at the bottom right of the page.

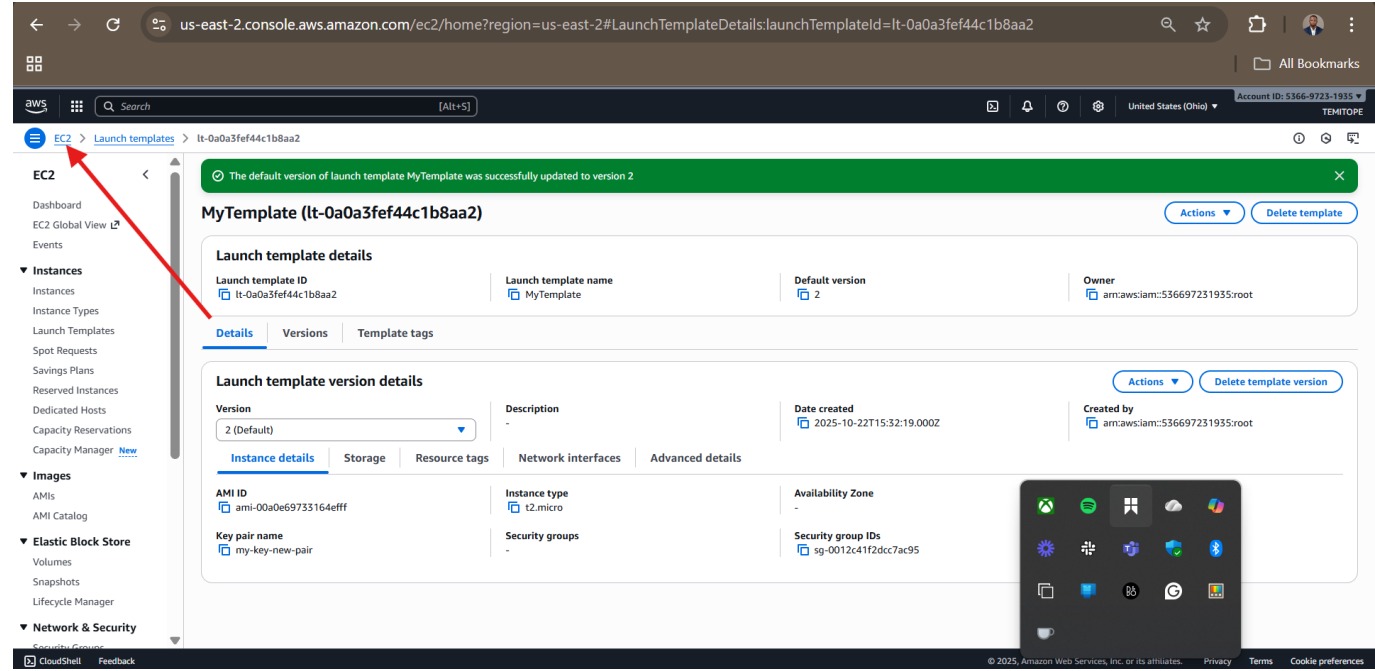
130b. Click versions tab then actions button then set default version.



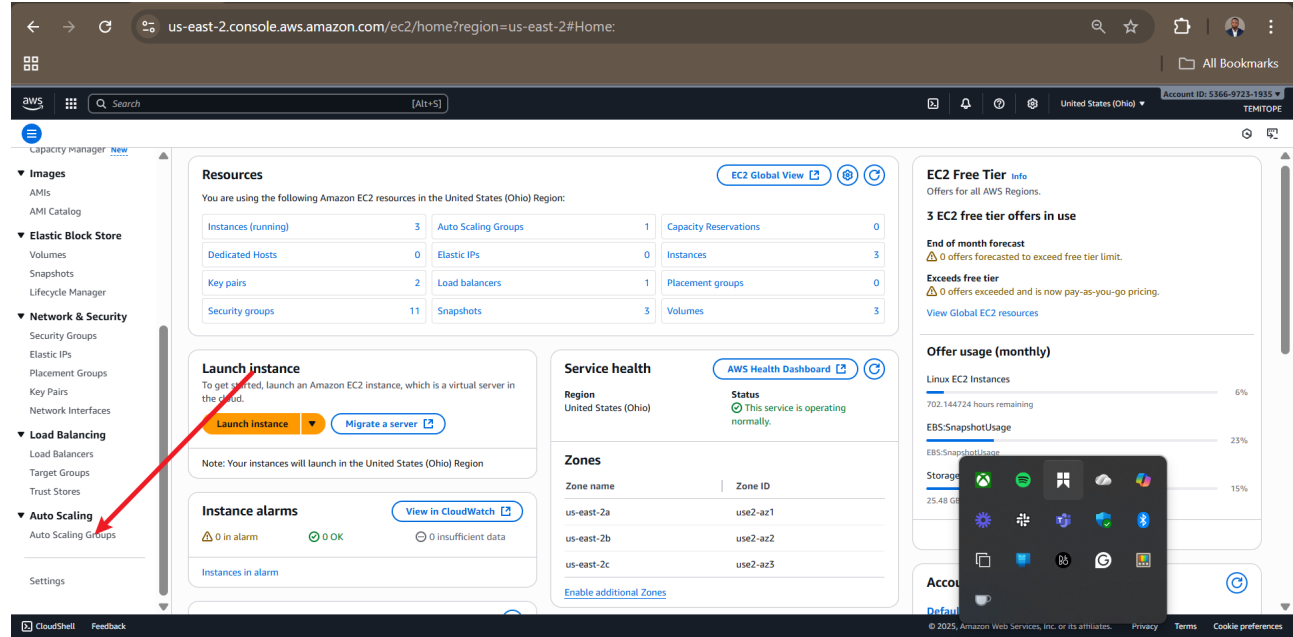
130c. Choose version 2 and click set as default version.



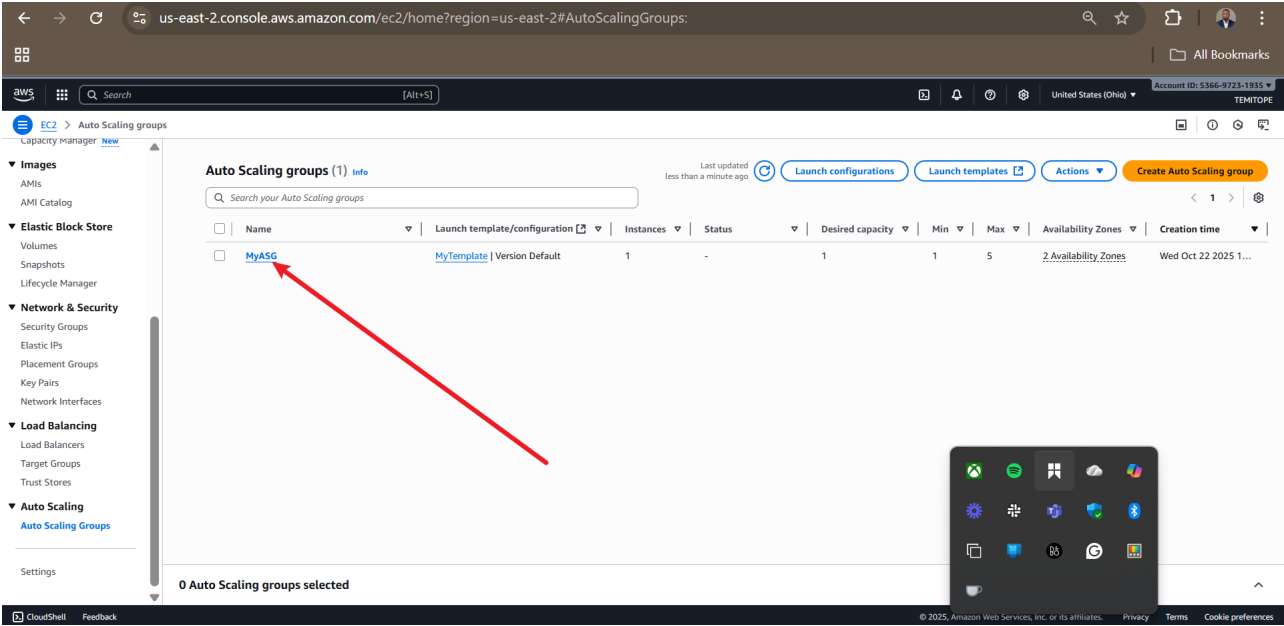
130d. Version 2 successfully set as default version click EC2 to navigate to EC2.



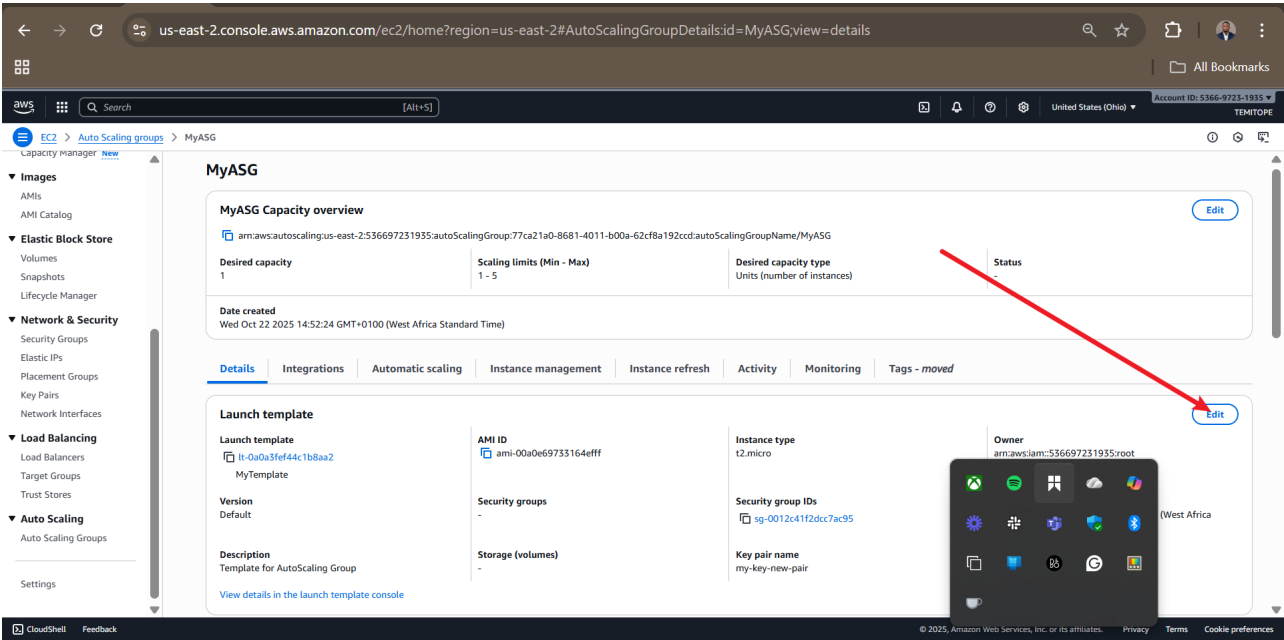
131. Navigate to EC2 page scroll to Auto scaling and click on auto scaling group.



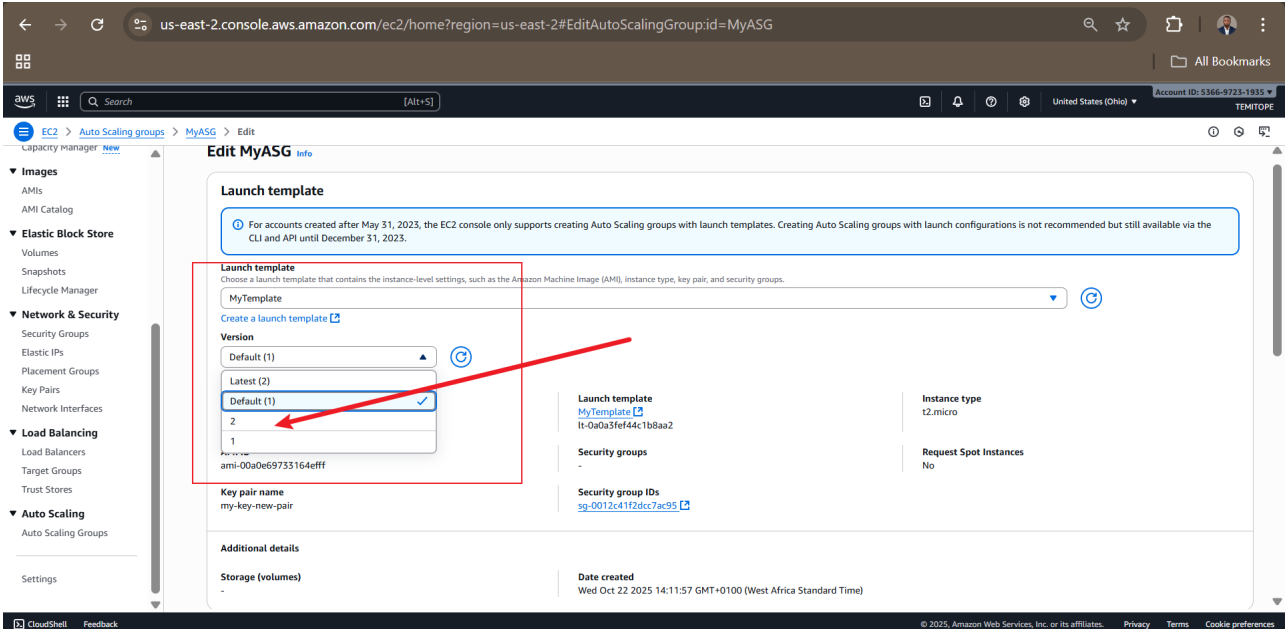
132. Click on the created AutoScaling Group.



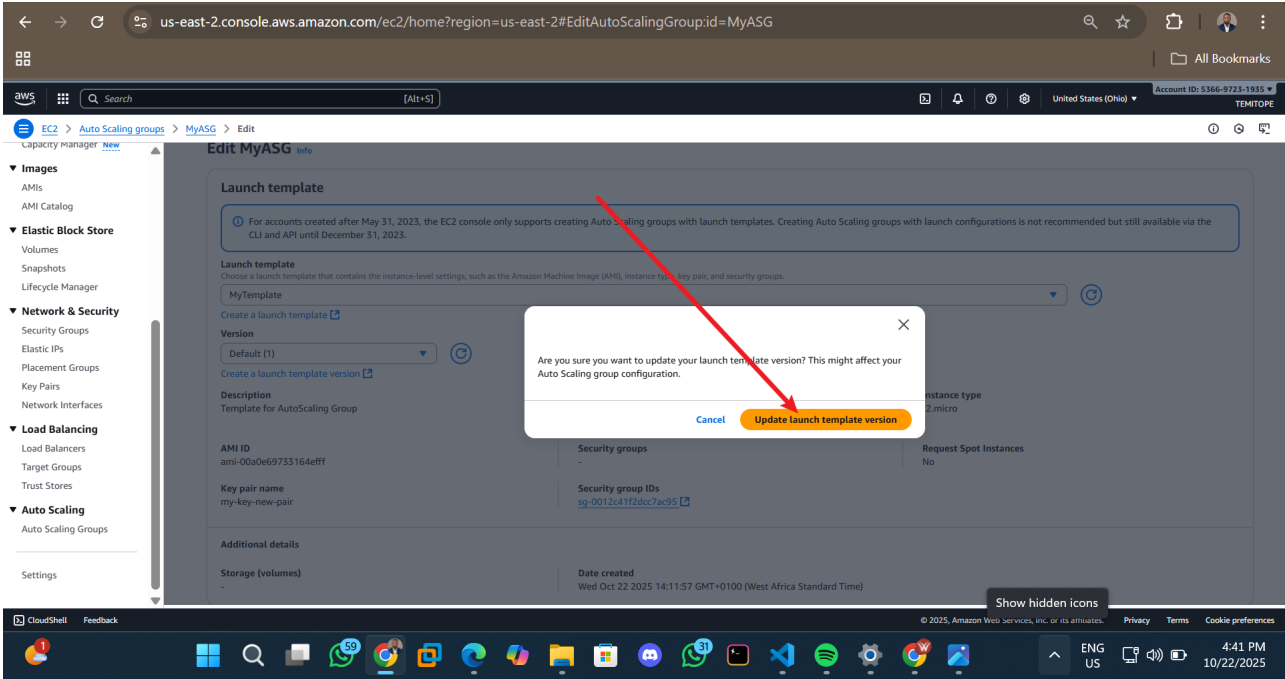
133. Click edit we are doing this to make scales public.



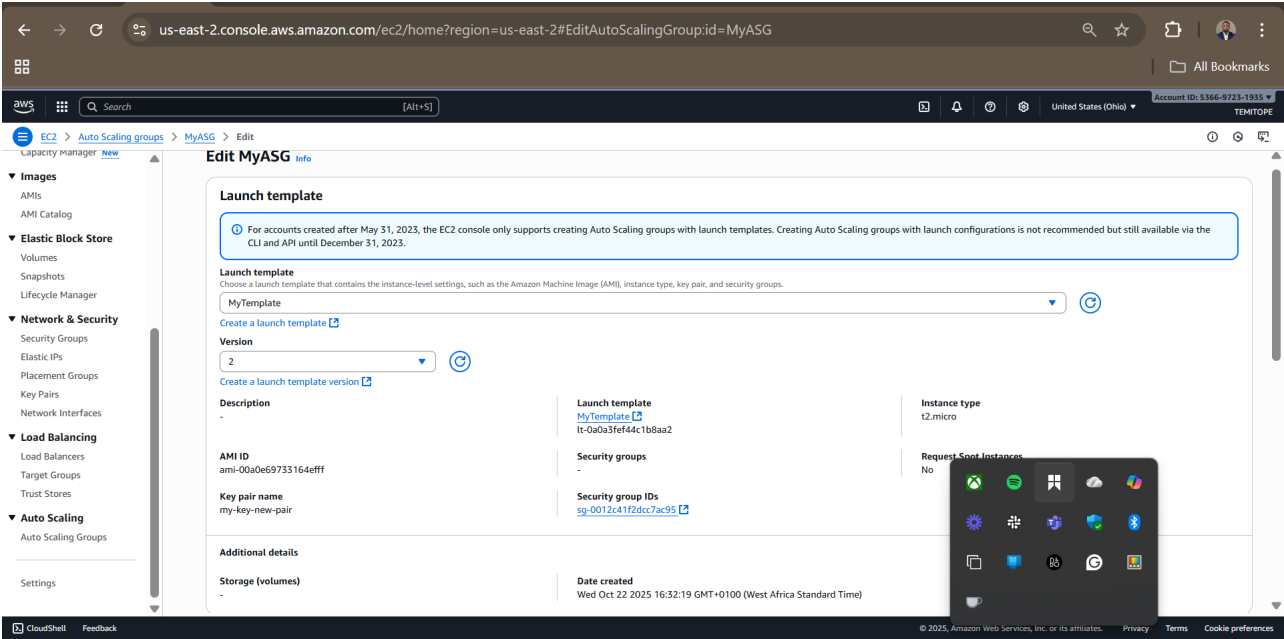
134. Under launch template version select 2 (new version).



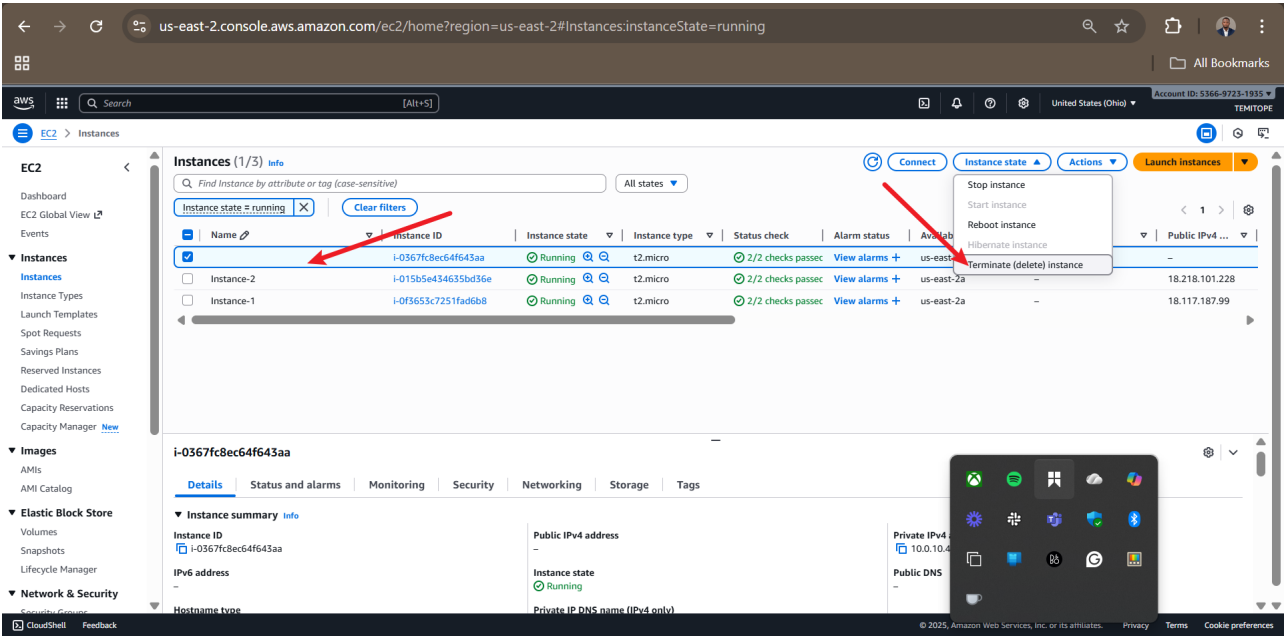
135. Click update launch template version on the pop-up.



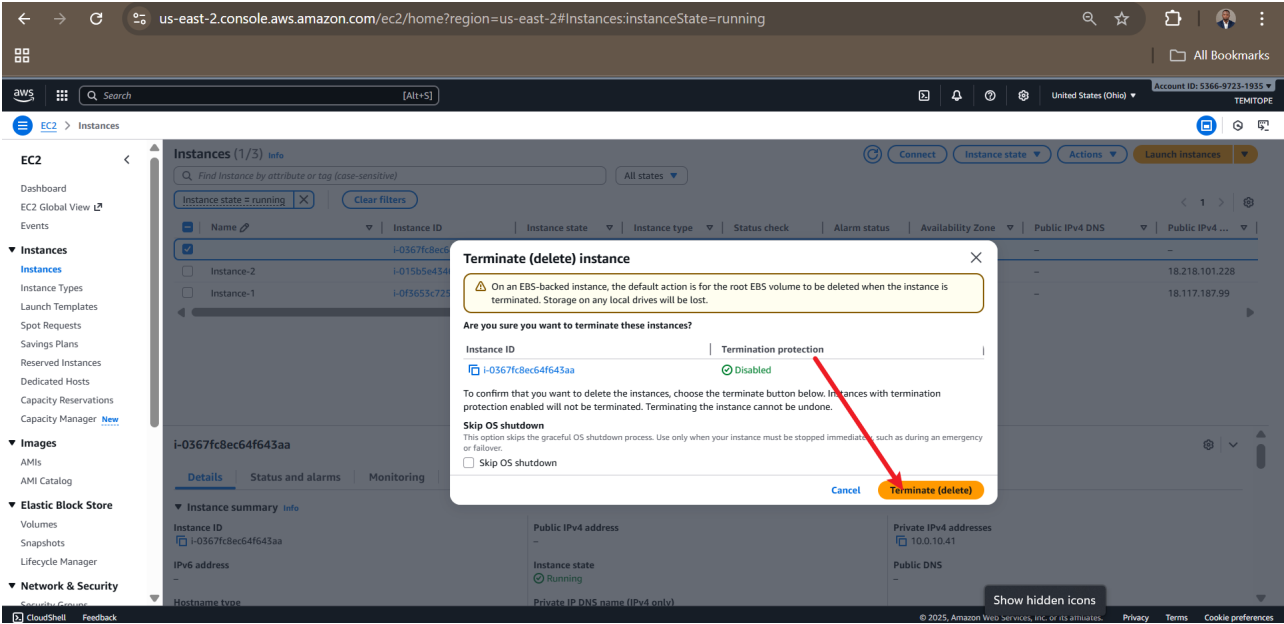
136. Version now updated so we can use EC2 instance connect to fuzz our scales.



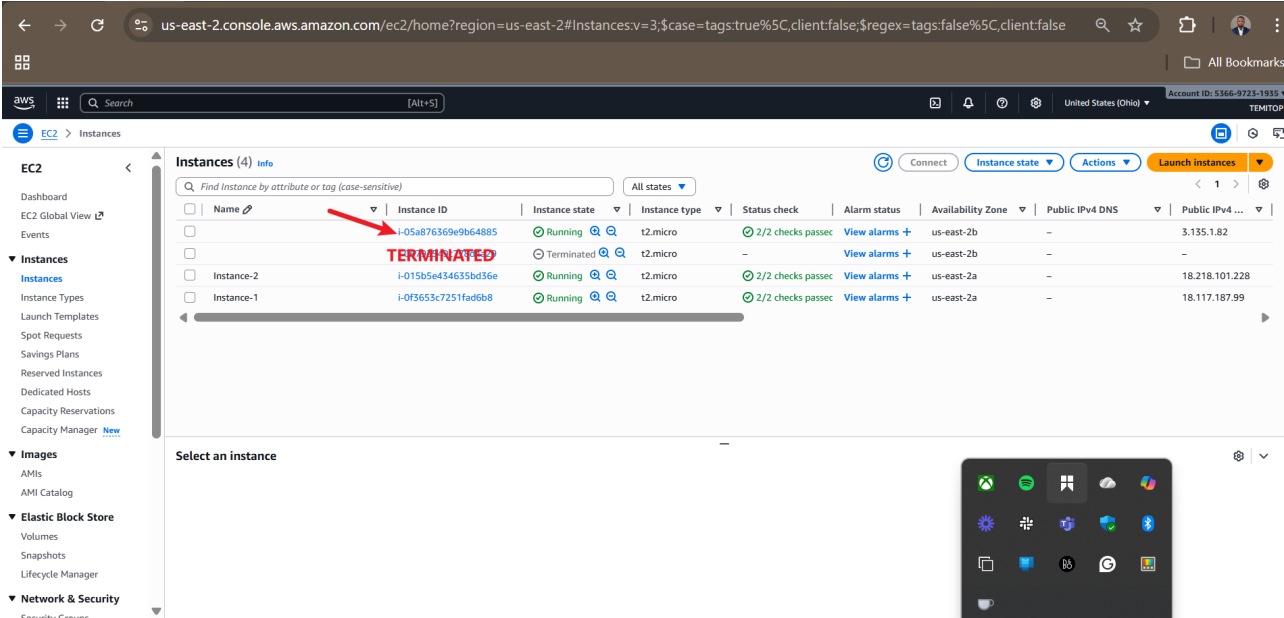
137. Select the EC2 created by ASG and terminate it.



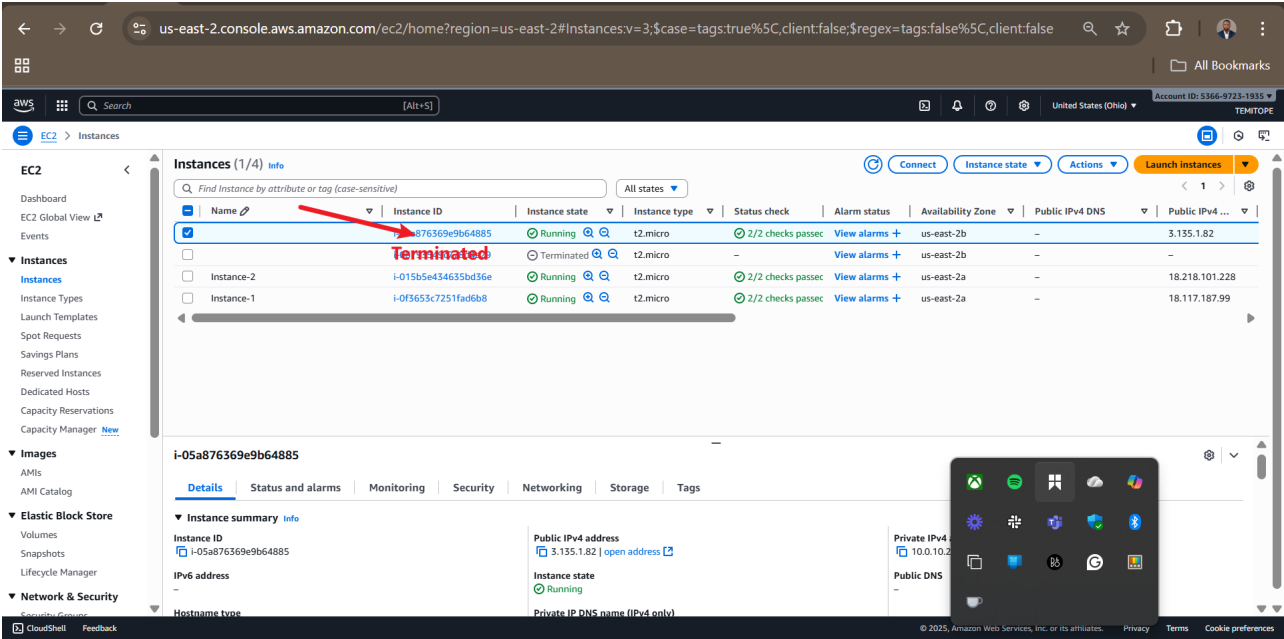
138. Click terminate ASG will create a new one that we can SSH into.



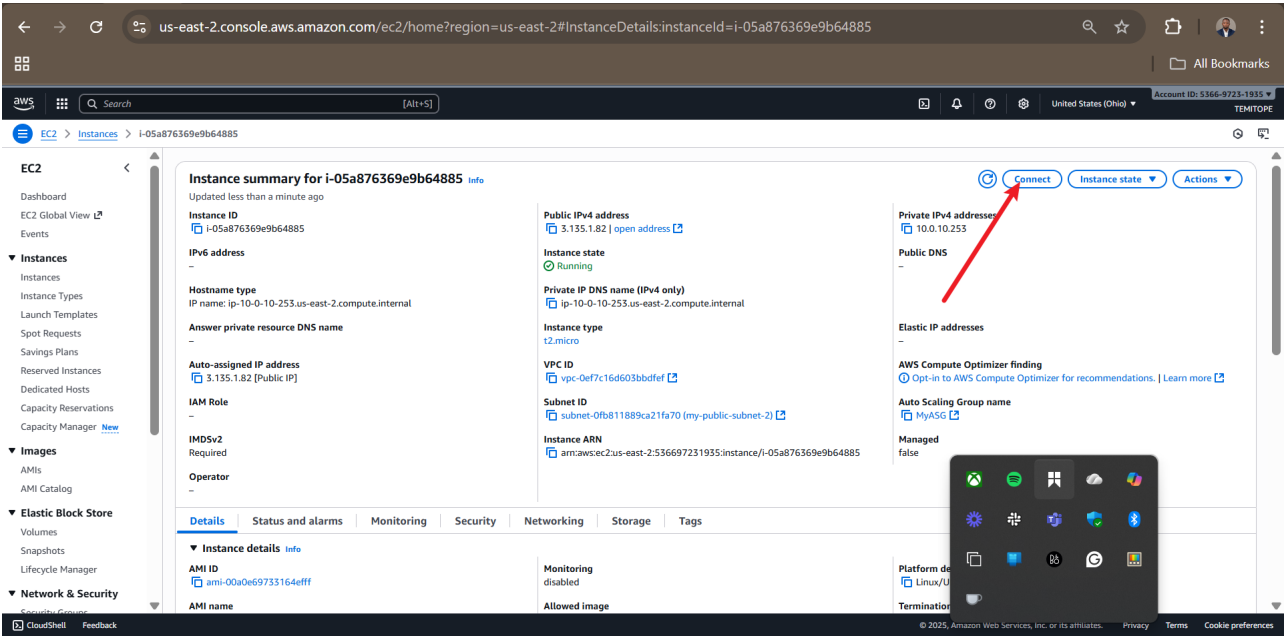
139. ASG has now spun up a new instance according to our policy (min of 1).



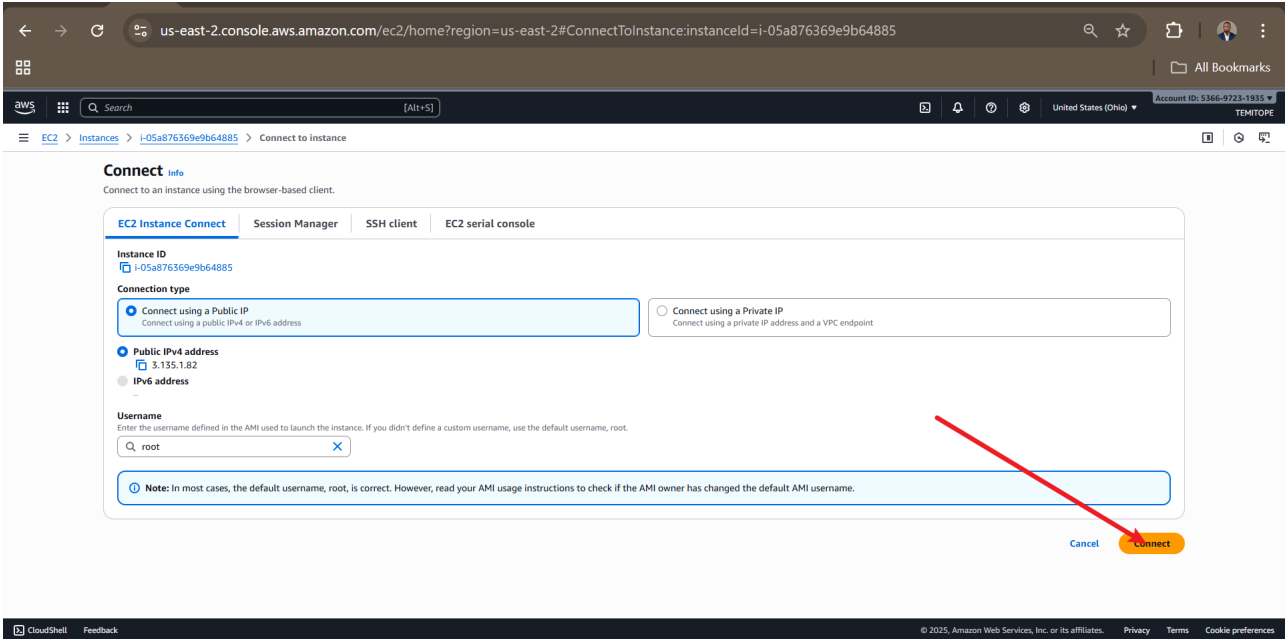
140. Click on the new instance to see if it can be connected to via SSH (internet).



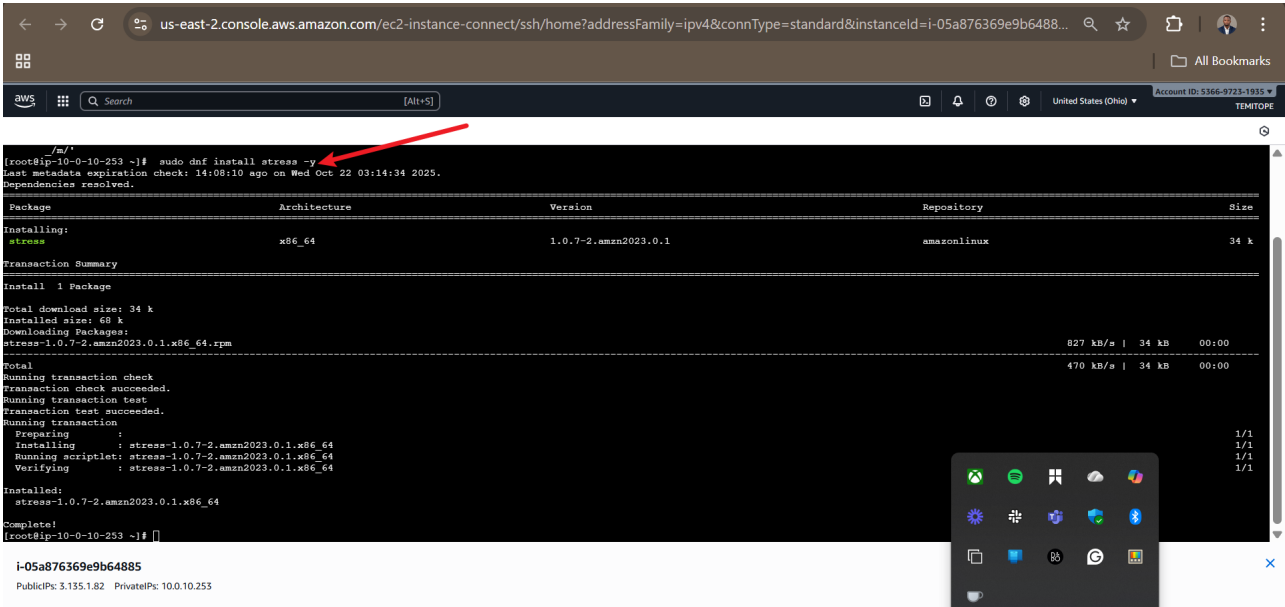
141. Now click connect to go to connect page.



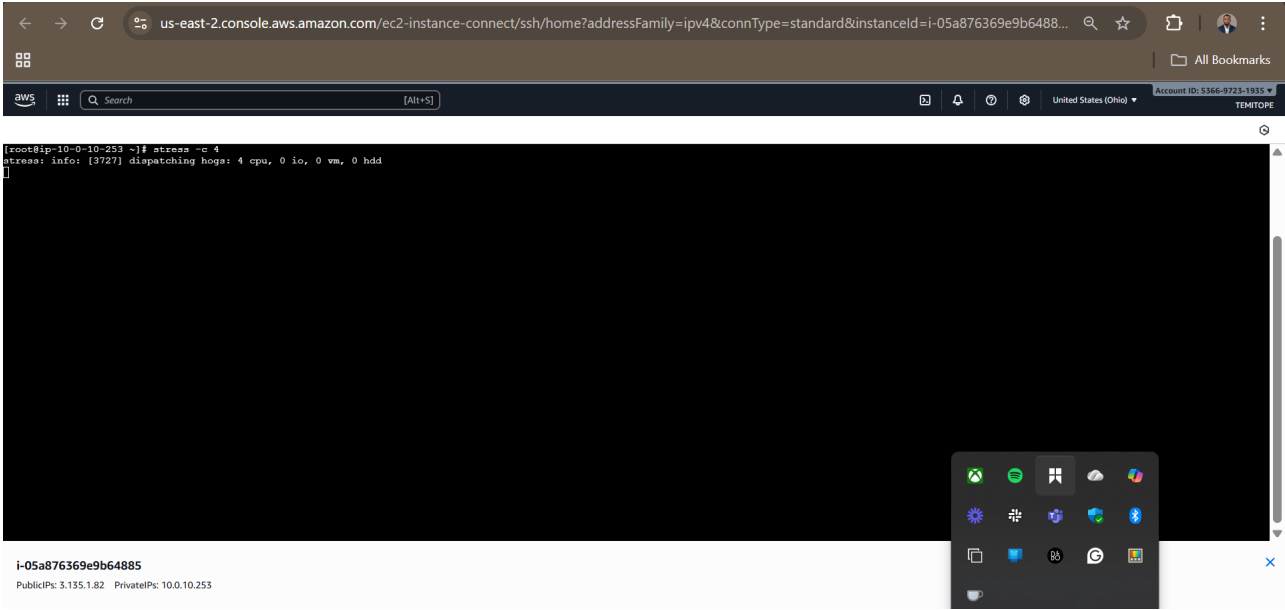
142. New auto scaling instance routable over the internet click connect.



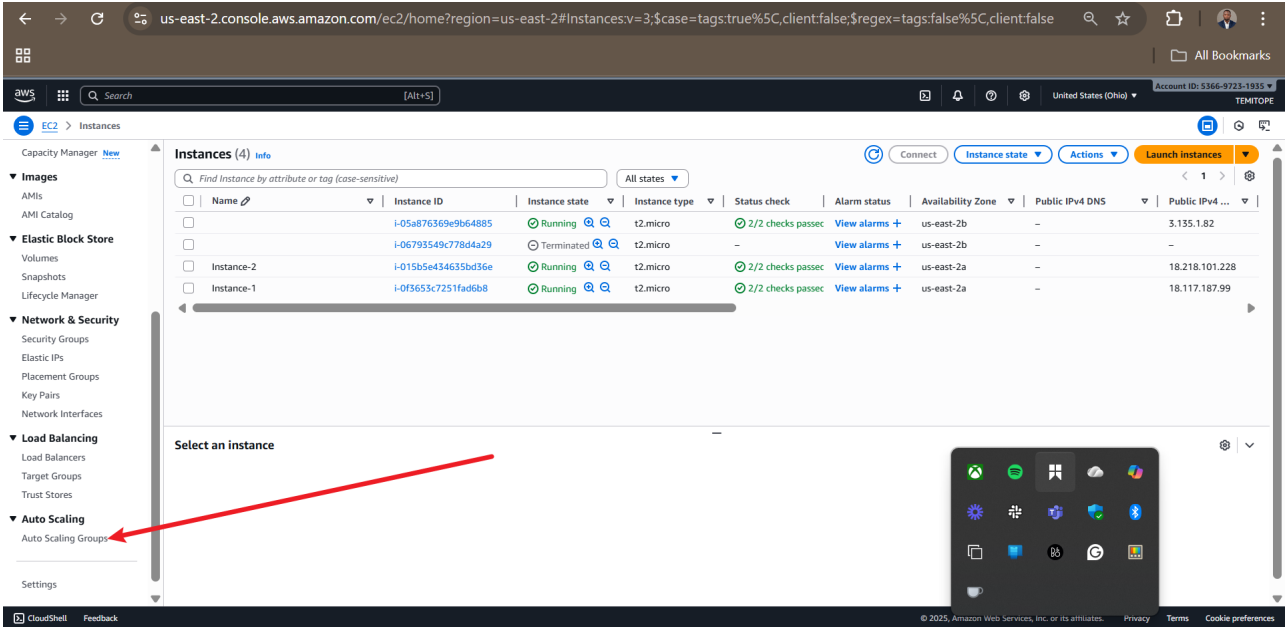
143. Install stress on new auto scaling instance.



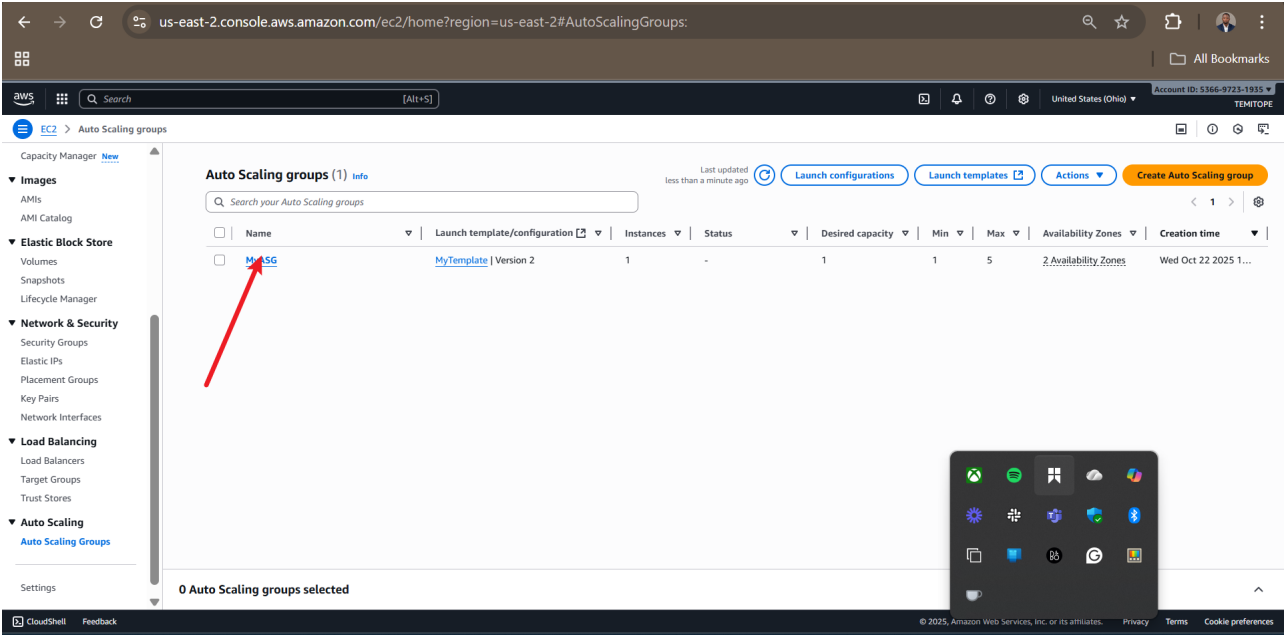
144. New auto scaling instance now stressed too.



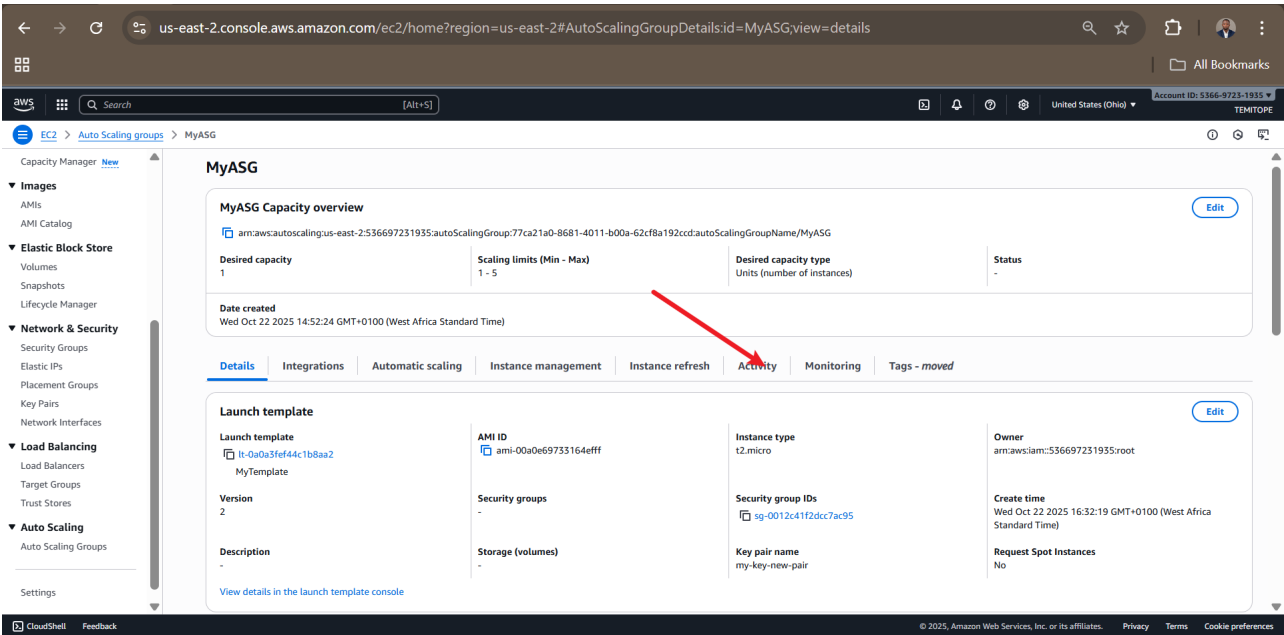
145. Navigate to EC2 instances page and under Auto Scaling click on ASG.



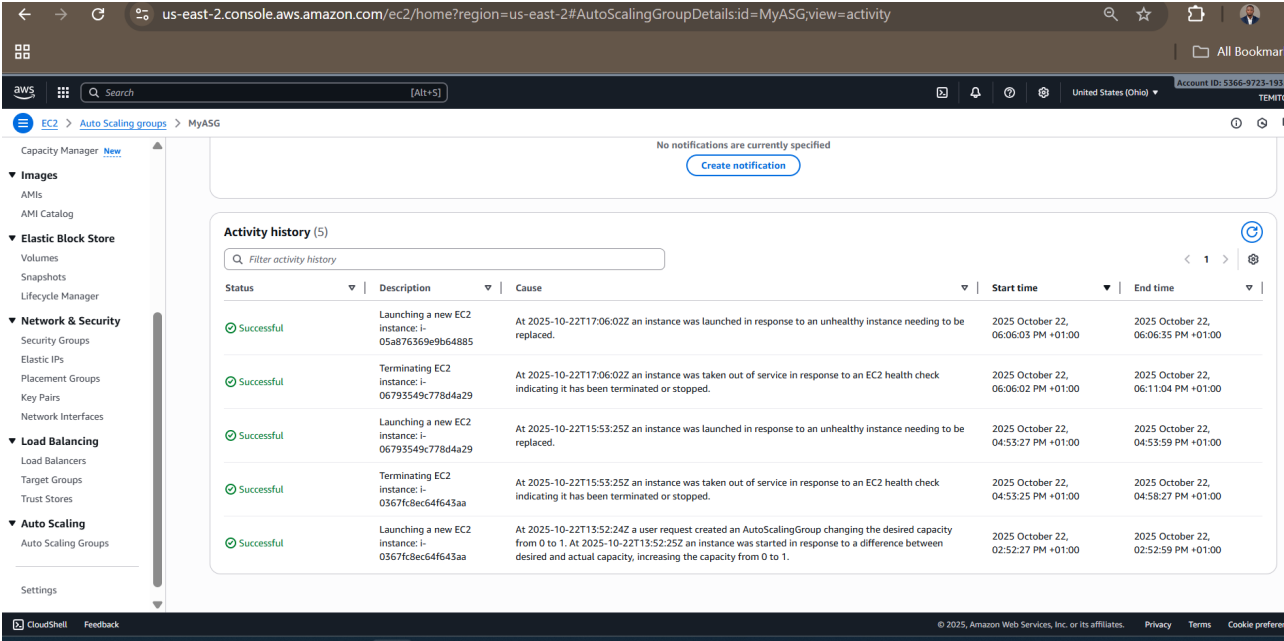
146. Click on the created auto scaling group.



147. On ASG dashboard click on activity.



148. Observe that there are now more instances created due to the surge in CPU consumption.



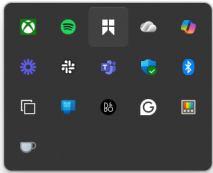
149. After a reload I now have instance with ID i-05a876369e9b64885.
!After a reload I now have instance with ID i-05a876369e9b64885]
(/img/149.after_a_reload_i_now_have_instace_with_id_ i-05a876369e9b64885.png)

150. After a reload I now have instance 2 with ID i-0f3653c7251fad6b8.
!After a reload I now have instance 2 with ID i-0f3653c7251fad6b8]
(/img/150.after_a_reload_i_now_have_instance_2_with_id_ i-0f3653c7251fad6b8.png)

151. After a reload I now have instance 5 with ID i-0c52afc9edb60330f.
!After a reload I now have instance 5 with ID i-0c52afc9edb60330f]
(/img/150.after_a_reload_i_now_have_instance_5_with_id_ i-0c52afc9edb60330f.png)

152. After a reload I now have instance 3 with ID i-015b5e434635bd36e.
!After a reload I now have instance 3 with ID i-015b5e434635bd36e]
(/img/151.after_a_reload_i_now_have_instance_3_with_id_ i-015b5e434635bd36e.png)

153. After a reload I now have instance 4 with ID i-015b5e434635bd36e.



154. Observe that all registered targets on target group are healthy.

us-east-2.console.aws.amazon.com/ec2/home?region=us-east-2#TargetGroup:targetGroupArn=arn:aws:elasticloadbalancing:us-east-2:53669723...

Search [Alt+S]

United States (Ohio) Account ID: 5366-9723-15

EC2 > Target groups > MyTargetGroup

Capacity Manager

▼ Images

- AMIs
- AMI Catalog

▼ Elastic Block Store

- Volumes
- Snapshots
- Lifecycle Manager

▼ Network & Security

- Security Groups
- Elastic IPs
- Placement Groups
- Key Pairs
- Network Interfaces

▼ Load Balancing

- Load Balancers
- Target Groups
- Trust Stores

▼ Auto Scaling

- Auto Scaling Groups

Settings

5
Total targets

5
Healthy

0
Unhealthy

0
Unused

0
Initial

0
Draining

Distribution of targets by Availability Zone (AZ)

Select values in this table to see corresponding filters applied to the Registered targets table below.

Targets

Monitoring

Health checks

Attributes

Tags

Registered targets (5)

Anomaly mitigation: Not applicable

Deregister

Register targets

Filter targets

	Instance ID	Name	Port	Zone	Health status	Health status details	Administrative o...	Override details	Launch...	Anomaly
☐	i-0f4edec5d4acc85		80	us-east-2b (us...	Healthy	-	No override	No override is curre...	October 2...	Normal
☐	i-0c52afc9edb60330f		80	us-east-2a (us...	Healthy	-	No override	No override is curre...	October 2...	Normal
☐	i-05a876369e9b64885		80	us-east-2b (us...	Healthy	-	No override	No override is curre...	October 2...	Normal
☐	i-0f3653c7251fad6b8	Instance-1	80	us-east-2a (us...	Healthy	-	No override	No override is curre...	October 2...	Normal
☐	i-015b5e434635bd36e	Instance-2	80	us-east-2a (us...	Healthy	-	No override	No override is curre...	October 2...	Normal